

FCN - Metodi iterativi 1 passo

ORA che abbiamo portato le librerie ad un punto stabile, possiamo riprendere lo sviluppo di argomenti nuovi. Cosa ne pensi se implementassimo qualche metodo iterativo tipico, iniziando da quelli ad un passo, espliciti, da Eulero in su?

Mi sembra un'ottima idea: partire da Eulero esplicito e costruire una gerarchia di metodi one-step espliciti (Runge-Kutta di ordine crescente, metodi di Taylor, ecc.) è il modo più naturale per strutturare bene sia la teoria (ordine, consistenza, stabilità) sia l'API numerica.¹²

Cornice del problema

Direi che il setting naturale per i metodi “ad un passo” espliciti è il problema di Cauchy per ODE:

$$y'(t) = f(t, y(t)), \quad y(t_0) = y_0, \quad t \in [t_0, T].$$

Un metodo one-step esplicito con passo h produce una successione $y_n \approx y(t_n)$ con $t_{n+1} = t_n + h$ della forma generale

$$y_{n+1} = \Phi(t_n, y_n, h, f),$$

dove Φ dipende solo da valori già noti di y e da valutazioni di f in (t_n, y_n) , $(t_n + \alpha h, y_n + \dots)$, ecc., ma mai in (t_{n+1}, y_{n+1}) (questo li distingue dagli impliciti).³⁴

Per la parte “metodi iterativi ad un passo” in senso astratto, è la classica successione $x_{n+1} = \varphi(x_n)$ per la soluzione di un'equazione o di un problema fisso, con convergenza garantita localmente se $|\varphi'(\alpha)| < 1$ in un intorno del punto fisso α .⁵

Metodo di Eulero esplicito

Per cominciare l'implementazione, il candidato ovvio è Eulero in avanti:

$$y_{n+1} = y_n + h f(t_n, y_n), \quad t_{n+1} = t_n + h.$$

È il più basilare dei metodi espliciti per ODE ed è un Runge-Kutta di ordine 1. Si interpreta come approssimazione della soluzione tramite il primo polinomio di Taylor, sostituendo $y'(t_n)$ con $f(t_n, y_n)$.⁶⁷⁸

Per organizzare il codice della tua libreria, potremmo:

- Definire una funzione template del tipo

```
template<class F, class State>
void euler_step(F&& f, double t, double h, State& y) {
    y = y + h * f(t, y);
}
```

dove **State** può essere uno scalare, un `std::vector`, una tua struttura di stato, ecc., assumendo che siano definite le operazioni di somma e prodotto scalare; questo mantiene il codice generico e coerente con il resto della FCN. - Incapsulare poi un “integratore” come oggetto che tiene **t**, **h**, **y** e chiama ripetutamente **step**.

Come test immediato, classico oscillatore armonico, oppure un decadimento esponenziale per avere un caso stiff dove far vedere subito i limiti di stabilità di Eulero.⁹¹⁰

Passi successivi “da Eulero in su”

Per costruire una progressione naturale di metodi one-step espliciti:

- Metodi di Taylor espliciti:
 - Usare l'espansione in serie di $y(t+h)$ e derivare f rispetto a t e y ; utili più come esercizio concettuale per ordine/consistenza che come metodo pratico.¹¹
- Runge-Kutta a 2 stadi (Heun, “improved Euler”):
 - Forma standard:

$$k_1 = f(t_n, y_n), \quad k_2 = f(t_n + h, y_n + hk_1), \quad y_{n+1} = y_n + \frac{h}{2}(k_1 + k_2).$$

- Ordine 2, ancora esplicito e semplice, buono come secondo passo didattico dopo Eulero.¹² - Runge-Kutta classico a 4 stadi: - Il solito RK4, che in pratica è il “workhorse” generale; a questo punto avrebbe senso strutturare anche l’infrastruttura per i Butcher tableaux.¹³¹⁴

Tutti questi metodi sono one-step espliciti e rientrano nella stessa interfaccia di `step(f,t,h,y)`; differiscono solo per il numero di valutazioni di `f` e per la combinazione lineare.¹⁵¹⁶

Struttura concettuale da codificare

Se vogliamo far crescere la libreria in modo “didatticamente pulito”, io proporrei:

- Namespace (o modulo) dedicato alle ODE con:
 - concetto astratto di “stepper” one-step;
 - implementazioni concrete: `euler_explicit`, `rk2_heun`, `rk4_classic`, ecc.;
 - eventuale concetto di “controller” per variare il passo in base a stime d’errore.¹⁷
- Documentazione che esplicita per ogni metodo:
 - ordine di consistenza;
 - proprietà di stabilità (magari, per Eulero, regione di stabilità sul problema lineare test $y' = \lambda y$).¹⁸¹⁹

Se ti va, nel prossimo passo possiamo fissare con precisione l’interfaccia C++ desiderata (template, concetti, tipo di `State`, gestione di funzioni lambda vs functor) e scrivere insieme una prima implementazione completa di Eulero esplicito più qualche test di convergenza sull’oscillatore armonico o su un pendolo linearizzato.

Ti interessa che questi metodi one-step siano subito predisposti anche per sistemi Hamiltoniani (per poi discutere metodi simmetrici/simpletici), o per il momento li pensiamo come integratori “generici” e rimandiamo la parte geometricamente consapevole a una sezione dedicata? ²⁰²¹²²²³²⁴²⁵²⁶²⁷²⁸²⁹³⁰

Io pensavo ad un’ implementazione didattica, abbastanza inconsapevole delle implicazioni incrociate con altre materie, quindi fine all’analisi numerica ma soprattutto in stile imperativo per questioni di propedeuticità didattica. Compiliamo con g++ ma senza costrutti OOP, come se fosse anzi C ma con le librerie standard di C++. Mi sono spiegato bene?

Ti sei spiegato benissimo: un’implementazione didattica, “imperativa pura”, niente OOP, come se fosse C con

Obiettivo dell’implementazione

L’idea, se capisco bene, è:

- Concentrarsi su:
 - formulazione dei metodi ad un passo;
 - implementazione “lineare” degli algoritmi;
 - osservazione sperimentale di errore, ordine, stabilità.
- Rinviare:
 - qualsiasi design a classi/gerarchie;
 - considerazioni “avanzate” (Hamiltoniani, metodi simpletici, ecc.);
 - astrazioni generiche complesse.

Quindi, file `.cpp` con funzioni libere del tipo:

```
double f(double t, double y) {
    return /* RHS dell'ODE */;
}

void euler_step(double t, double h, double* y) {
    *y = *y + h * f(t, *y);
}
```

e un `main()` che gestisce loop, I/O, e magari stampa su file i risultati per poterli plottare.

Stile di codice proposto

Direi di fissare uno “stile base” a cui attenersi per tutti i metodi:

- Firma delle funzioni di passo:
 - Variante per scalare:

```
void euler_step(double t, double h, double* y);
```

– Variante per sistema (dimensione fissata `n`):

```
void euler_step_system(double t, double h, double* y, int n);
```

dove `y` è un array `C` di lunghezza `n` e la RHS della ODE è incapsulata in una funzione tipo

```
void f_system(double t, const double* y, double* dydt, int n);
```

- Niente template, niente `std::function`, solo puntatori a funzione:

```
void euler_step_system(double t, double h,
                      double* y, int n,
                      void (*f)(double, const double*, double*, int));
```

- Un `main()` che:
 - legge o definisce `t0`, `T`, `h`, `y0`;
 - calcola il numero di passi `N`;
 - fa un loop `for (int k = 0; k < N; ++k) { step(...); }`;
 - stampa `t_k`, `y_k` in formato testo.

Questo rende l’implementazione molto leggibile per chi viene da C/imperativo e permette di vedere chiaramente dove sta la logica numerica.

Roadmap “imperativa” dei metodi

In questo stile, potremmo procedere così:

1. **Eulero esplicito**
 - Funzione di passo per scalare e per sistema.
 - Esempio: decadimento esponenziale, oscillatore armonico linearizzato.
2. **“Eulero migliorato” (Heun / RK2)**
 - Funzione di passo che usa due chiamate a `f`.
 - Stessa interfaccia, cambia solo l’algoritmo interno.
3. **RK4 classico**
 - Quattro valutazioni di `f`, ancora stessa interfaccia.
 - Comparazione numerica empirica di errore vs passo.
4. (Volendo) **Metodo di Taylor di ordine 2 per un caso semplice**
 - Solo dove `f` è nota in forma che permette di derivarla a mano, per mostrare la relazione con lo sviluppo di Taylor.

Tutto rigorosamente in stile “C con un po’ di C++”.

Se ti va, nel prossimo messaggio posso proporti direttamente il primo file: `euler_explicit.cpp` con:

- definizione di un problema di test (es. $y' = \lambda y$);

- implementazione di `f`, `euler_step`;
- `main()` completo, compilabile con `g++ euler_explicit.cpp -O2 -Wall`.

Ti interesserebbe partire dal caso scalare o vuoi subito un esempio su sistema di dimensione 2 (tipo oscillatore armonico scritto come sistema del primo ordine)?

Ragioniamo un attimo su cio' di cui disponiamo per capire da dove ripartire.

Abbiamo già in forma matriciale una soluzione per sistemi di cauchy lineari del primo ordine, la nostra `matrix_build_cauchy` alla quale possiamo assegnare anche funzioni non lineare tramite i due coefficienti, variabile di tipo funzione, $a(t)$ e $f(t)$.

Questo ci copre già i problemi lineari e possiamo passare al secondo ordine, tipo oscillatore armonico, oppure pensavi ad un' implementazione puramente iterativa, estranea all' approccio matriciale, che potrebbe essere anche un banco di confronto con quello che abbiamo già e da cui varrebbe la pena ripartire?

Ha senso distinguere nettamente i due filoni: tenere la parte matriciale come “solutore strutturato” per sistemi lineari, e affiancarle ora un nucleo di metodi iterativi one-step puramente procedurali, che usiamo sia come banco di confronto sia come palestra didattica su ordine/stabilità/errore.³¹³²

Cosa copre già la parte matriciale

La tua `matrix_build_cauchy` è, di fatto, un'implementazione in forma matriciale della soluzione di un problema di Cauchy lineare del primo ordine:

$$y'(t) = A(t) y(t) + g(t), \quad y(t_0) = y_0,$$

dove $A(t)$ e $g(t)$ sono assegnati anche tramite funzioni $a(t)$, $f(t)$. In termini teorici, questo corrisponde all'uso della matrice di transizione (esponenziale di matrice nel caso a coefficienti costanti) o comunque a una formulazione globale del sistema lineare.³³³⁴

Questo significa:

- hai già una soluzione robusta e concettualmente “alta” per problemi lineari di primo ordine;
- puoi gestire anche forzamenti non lineari nel termine noto se lo incapsuli in $f(t)$ (nel senso di non linearità in t , non in y).

Quindi sul primo ordine lineare sei coperto.

Perché vale la pena aggiungere un nucleo iterativo “grezzo”

I metodi one-step espliciti come Eulero, Heun, RK4, ecc., sono per loro natura iterativi nel senso $y_{n+1} = \Phi(t_n, y_n, h)$. Implementarli:³⁵³⁶

- ti permette di lavorare direttamente sulla discretizzazione temporale (griglia t_n) e sulle nozioni di:
 - consistenza (errore locale),
 - convergenza (errore globale),
 - stabilità (equazione test $y' = \lambda y$);³⁷³⁸
- ti dà un riferimento “minimale” per confrontare:
 - la soluzione matriciale (quando applicabile);
 - i metodi espliciti ad un passo sulla stessa ODE;
- è propedeutico a tutto il resto:
 - metodi impliciti;
 - metodi stiff-stable;
 - metodi symplettici su Hamiltoniani (più avanti).

Inoltre, questa parte iterativa può uscire esplicitamente dal framework matriciale e lavorare direttamente con funzioni $f(t, y)$ non lineari in y , cosa che la soluzione matriciale lineare non copre naturalmente.³⁹⁴⁰

Proposta di ripartenza: nucleo iterativo indipendente

Io ti proporrei quindi di:

1. **Tenere `matrix_build_cauchy` come “modulo parallelo”**
 - Lo useremo dopo come benchmark:
 - per un sistema lineare 1D o 2D, puoi confrontare la soluzione ottenuta con `matrix_build_cauchy` con quella di Eulero/RK4 sulla stessa griglia temporale.
2. **Definire un’API minimale iterativa “alla C” per ODE scalare**

Qualcosa del tipo (pseudocodice C-like):

```
double f(double t, double y); // RHS

void euler_step(double t, double h, double* y,
                double (*f)(double, double)) {
    *y = *y + h * f(t, *y);
}
```

Questa è completamente estranea all’approccio matriciale: prende un valore y_n e fa un passo. Il `main` fa il loop:⁴¹⁴²

```
int main() {
    double t = t0;
    double y = y0;
    for (int k = 0; k < N; ++k) {
        euler_step(t, h, &y, f);
        t += h;
        // stampa t, y
    }
}
```

3. **Estendere poi ad un sistema del primo ordine “alla C”**

Per gestire un sistema:

```
void f_system(double t, const double* y, double* dydt, int n);

void euler_step_system(double t, double h,
                       double* y, int n,
                       void (*f)(double, const double*, double*, int)) {
    std::vector<double> dydt(n);
    f(t, y, dydt.data(), n);
    for (int i = 0; i < n; ++i)
        y[i] += h * dydt[i];
}
```

Anche qui, tutto è imperativo, niente matrici “alte”, ma puoi scegliere f che provenga da un sistema lineare (e confrontare col modulo matriciale) o da uno non lineare. 4. **Oscillatore armonico come “secondo ordine”**

L’oscillatore armonico $q'' + \omega^2 q = 0$ viene riscritto come sistema del primo ordine:

$$q' = p, \quad p' = -\omega^2 q,$$

e integrato con `euler_step_system` ecc. Questo ti permette di:⁴³ - riciclare la stessa infrastruttura; - iniziare a discutere di conservazione dell’energia e vedere empiricamente che Eulero esplicito la “spara via” (introducendo il desiderio di metodi symplettici più avanti, ma senza toccarli ora).

Collegamento esplicito con la parte matriciale

Una volta che hai:

- `matrix_build_cauchy` per un sistema lineare $y' = Ay$;
- `euler_step_system` (o `rk4_step_system`) che lavora su $f(t, y) = Ay$;

puoi costruire un esempio di confronto didattico:

- scegli A costante e un y_0 ;
- convergi con `matrix_build_cauchy` alla soluzione “esatta” (o quasi) basata sulla matrice di transizione;⁴⁴⁴⁵
- integra con Eulero esplicito sulla stessa griglia;
- misura la norma della differenza e mostra la dipendenza dall’ampiezza di h coerente con l’ordine del metodo.⁴⁶⁴⁷

Questo rende molto tangibile il legame tra approccio globale matriciale e iterativo locale.

Io partirei dunque proprio da un file “grezzo” che implementa Eulero esplicito *indipendente* da `matrix_build_cauchy`, e poi costruirei un esempio dove per un sistema lineare semplice li mettiamo a confronto.

Ti va se nel prossimo passo ti scrivo uno scheletro concreto di `euler_scalar.cu` (diciamo un solo `.cpp`) in stile C, con:

- definizione del problema $y' = \lambda y$,
- Eulero esplicito,
- stampa dei dati e commento di come affiancarlo al tuo modulo matriciale? ⁴⁸⁴⁹⁵⁰⁵¹⁵²⁵³⁵⁴⁵⁵

si, procediamo così’ in modo da poter affrontare gli stessi problemi che sappiamo risolvere con il matriciale e poi confrontare le soluzioni. Credo che questa bozza sia poi espandibile al secondo ordine se ho capito bene cosa mi stai proponendo.

Esatto, hai capito perfettamente: impostiamo un nucleo iterativo esplicito, indipendente dal modulo matriciale, che risolve gli stessi problemi (almeno nel caso lineare) e poi lo useremo come banco di confronto; il passaggio al secondo ordine (oscillatore) viene naturalmente tramite riscrittura come sistema del primo ordine.⁵⁶

Schema di partenza: caso scalare, problema noto

Prendiamo come “hello world” l’ODE scalare lineare che puoi già trattare a livello matriciale:

$$y'(t) = \lambda y(t), \quad y(t_0) = y_0,$$

che ha soluzione esatta $y(t) = y_0 e^{\lambda(t-t_0)}$. Sul piano iterativo, il metodo di Eulero esplicito si scrive:⁵⁷

$$y_{n+1} = y_n + h f(t_n, y_n) = y_n + h \lambda y_n = (1 + h\lambda) y_n. \square \square$$

Qui il “core” algoritmico è chiarissimo e completamente legato da matrici; il tuo modulo matriciale può invece farti ottenere la soluzione esatta (o la matrice di transizione) per confronto.⁵⁸⁵⁹

Bozza imperativa in stile C

In stile puramente procedurale:

```
#include <iostream>
#include <cmath>

double f(double t, double y) {
    const double lambda = -1.0; // per esempio
    (void)t;                     // t non usato in questo esempio
    return lambda * y;
}
```

```

void euler_step(double t, double h, double* y,
               double (*rhs)(double, double)) {
    *y = *y + h * rhs(t, *y);
}

int main() {
    const double t0 = 0.0;
    const double T = 5.0;
    const double h = 0.1;
    const int N = static_cast<int>((T - t0) / h);

    double t = t0;
    double y = 1.0; // y0

    for (int n = 0; n <= N; ++n) {
        double y_exact = std::exp(-1.0 * (t - t0)); // soluzione esatta
        std::cout << t << " " << y << " " << y_exact << "\n";

        if (n < N) {
            euler_step(t, h, &y, f);
            t += h;
        }
    }
}

```

Questo è il prototipo minimale: nessun OOP, un solo puntatore a funzione per la RHS, e puoi già osservare empiricamente che l'errore globale va come $O(h)$ e la stabilità in funzione di λh .⁶⁰⁶¹⁶²

A questo punto, per un sistema lineare 1D che già tratti con `matrix_build_cauchy`, puoi:

- usare la soluzione “matriciale” come riferimento (fondamentalmente la stessa $e^{\lambda t}$),⁶³⁶⁴
- confrontare numericamente la sequenza y_n prodotta da Eulero con i valori sulla stessa griglia temporale.

Estensione naturale al sistema del primo ordine

Per preparare il passaggio al secondo ordine, basta generalizzare a un sistema $y' = f(t, y)$ con $y \in \mathbb{R}^n$.⁶⁵

Schema C-like:

```

#include <vector>

void f_system(double t, const double* y, double* dydt, int n) {
    // Esempio lineare: y' = A y, con A 2x2
    (void)t;
    const double a11 = 0.0, a12 = 1.0;
    const double a21 = -1.0, a22 = 0.0;
    dydt[4_0] = a11 * y[4_0] + a12 * y[4_1];
    dydt[4_1] = a21 * y[4_0] + a22 * y[4_1];
}

void euler_step_system(double t, double h,
                      double* y, int n,
                      void (*rhs)(double, const double*, double*, int)) {
    std::vector<double> dydt(n);
    rhs(t, y, dydt.data(), n);
    for (int i = 0; i < n; ++i) {
        y[i] += h * dydt[i];
    }
}

```

Qui l'oscillatore armonico standard $q'' + \omega^2 q = 0$ è semplicemente:

$$\begin{cases} q' = p, \\ p' = -\omega^2 q, \end{cases}$$

che si inserisce con `n=2` e un `f_system` lineare. È esattamente il tipo di sistema che la tua infrastruttura matriciale sa trattare, quindi il confronto è immediato.⁶⁶

Come usare questo per confrontare col matriciale

Per ogni problema lineare che sai già risolvere con `matrix_build_cauchy`:

1. Scegli un intervallo $[t_0, T]$ e un passo h , quindi una griglia $t_n = t_0 + nh$.⁶⁷
2. Usa il modulo matriciale per ottenere $y(t_n)$ “esatti” (o molto accurati) sui nodi.^{68,69}
3. Usa il nucleo iterativo Eulero/RK per ottenere y_n sulla stessa griglia.
4. Confronta $y_n - y(t_n)$ e verifica sperimentalmente:
 - dipendenza dell'errore da h (stima dell'ordine);
 - stabilità (caso $\lambda < 0$ vs $\lambda > 0$, ecc.).^{70,71,72}

Questo schema resta identico quando passi al “secondo ordine”: riduci a sistema del primo ordine e cambi solo $f(t, y)$.⁷³

Se ti va, nel prossimo passo posso prendere proprio uno dei sistemi lineari che hai già in mente per `matrix_build_cauchy` (per esempio un oscillatore con matrice costante 2×2) e scrivere un esempio completo dove:

- definisci la stessa matrice A in `f_system`;
- integri con Eulero esplicito;
- indico come affiancare (a livello concettuale/codice) la chiamata alla tua `matrix_build_cauchy` per ottenere il riferimento sulla stessa griglia.

Quale problema lineare “di casa tua” ti piacerebbe usare come primo banco di prova: il decadimento esponenziale $\lambda < 0$ o subito l'oscillatore armonico 2D? ^{74,75,76,77,78,79,80,81,82}

beh partiamo col decadimento esponenziale

Perfetto, allora fissiamo bene il “caso di scuola” decadimento esponenziale e la sua implementazione Eulero-style in stile C, già pronta per confrontarsi con il tuo modulo matriciale.

Problema: decadimento esponenziale

Consideriamo l'ODE scalare

$$y'(t) = \lambda y(t), \quad y(t_0) = y_0,$$

con $\lambda < 0$ per avere un decadimento. La soluzione esatta è⁸³

$$y(t) = y_0 e^{\lambda(t-t_0)}.$$

Il metodo di Eulero esplicito per il problema di Cauchy $y' = f(t, y)$ è

$$y_{n+1} = y_n + h f(t_n, y_n), \quad t_{n+1} = t_n + h,$$

quindi nel nostro caso:

$$f(t, y) = \lambda y \Rightarrow y_{n+1} = (1 + h\lambda) y_n.$$

La stabilità assoluta richiede $|1 + h\lambda| < 1$, cioè, per $\lambda < 0$,

$$-1 < 1 + h\lambda < 1 \Rightarrow 0 < h < \frac{2}{|\lambda|}.$$

Questo è un ottimo esempio per illustrare sia l'errore di ordine 1, sia la regione di stabilità di Eulero.⁸⁴⁸⁵

Implementazione imperativa minimale (scalare)

Ti propongo una bozza “C-like” compilabile con g++ -O2 -Wall che:

- definisce la RHS $f(t, y)$;
- implementa un singolo passo di Eulero;
- fa un loop e stampa anche la soluzione esatta sullo stesso nodo, in modo che tu possa poi affiancare o sostituire la colonna “esatta” con quella ottenuta dal modulo matriciale.

```
#include <iostream>
#include <cmath>

// RHS della ODE: y' = lambda * y
double rhs(double t, double y) {
    (void)t; // t non usato in questo esempio
    const double lambda = -1.0; // decadimento con costante 1
    return lambda * y;
}

// Un passo di Eulero esplicito: y_{n+1} = y_n + h * f(t_n, y_n)
void euler_step(double t, double h, double* y,
                double (*f)(double, double)) {
    *y = *y + h * f(t, *y);
}

int main() {
    const double t0 = 0.0;
    const double T = 5.0;
    const double h = 0.1; // soddisfa |1 + h*lambda| < 1 per lambda=-1
    const int N = static_cast<int>((T - t0) / h);

    double t = t0;
    double y = 1.0; // y0

    for (int n = 0; n <= N; ++n) {
        // soluzione esatta sul nodo (per ora da formula chiusa)
        double y_exact = std::exp(-1.0 * (t - t0));

        std::cout << t << " " << y << " " << y_exact << "\n";

        if (n < N) {
            euler_step(t, h, &y, rhs);
            t += h;
        }
    }

    return 0;
}
```

Questa struttura è volutamente:

- senza `std::vector`, senza OOP, solo un puntatore a funzione per la RHS;
- facilmente generalizzabile (basta cambiare `rhs`);
- pronta per essere “allineata” al tuo approccio matriciale: se al posto di `y_exact` metti la soluzione che ti arriva da `matrix_build_cauchy` valutata allo stesso `t`, hai un confronto diretto.⁸⁶⁸⁷

Collegamento con la soluzione matriciale

Per un problema lineare scalarissimo come questo, il tuo modulo matriciale dovrebbe ridursi a qualcosa che implementa $e^{\lambda(t-t_0)}$ o l'equivalente costruito via matrice di transizione in dimensione 1. A livello “banco di prova”, puoi:⁸⁸⁸⁹

- lasciare la colonna “soluzione esatta” come da formula, oppure
- farla produrre dal percorso `matrix_build_cauchy` su una griglia di tempi t_n identica a quella del loop di Eulero, in modo da non usare la formula chiusa e far vedere che l'approccio matriciale e quello iterativo convergono allo stesso oggetto.

L'importante, per la didattica, è tenere chiaro che:

- `euler_step` implementa un metodo ad un passo di ordine 1 (errore globale $O(h)$);⁹⁰⁹¹
- il modulo matriciale rappresenta la “soluzione strutturale” del problema lineare;
- il confronto numerico concretizza le definizioni di errore globale e ordine.

Se ti torna questo schema, il passo successivo potrebbe essere:

- stessa ODE $y' = \lambda y$, ma implementiamo anche Heun o RK2 in forma analoga a `euler_step`, per mostrare immediatamente il salto di ordine con lo stesso driver `main`; oppure
- passare già a un sistema 2×2 lineare (con A costante) con un'API `euler_step_system` analoga, per preparare l'oscillatore armonico.

Quale dei due passi preferisci affrontare subito: aggiungere un secondo metodo (Heun/RK2) sullo scalare o generalizzare già ad un sistema 2D con Eulero? ⁹²⁹³⁹⁴⁹⁵⁹⁶⁹⁷⁹⁸⁹⁹¹⁰⁰¹⁰¹¹⁰²

ho fatto un collage del codice che mi hai passato e l'ho montato così in libreria con il main che è diventata una funzione test. solo che non vedo dove vengono chiamate le prime due funzioni

```
//
// Funzioni per soluzioni di ODE con metodi iterativi
//
void f_system(double t, const double* y, double* dydt, int n) {
    // Esempio lineare: y' = A y, con A 2x2
    (void)t;
    const double a11 = 0.0, a12 = 1.0;
    const double a21 = -1.0, a22 = 0.0;
    dydt[0] = a11 * y[0] + a12 * y[1];
    dydt[1] = a21 * y[0] + a22 * y[1];
}

void euler_step_system(double t, double h,
                      double* y, int n,
                      void (*rhs)(double, const double*, double*, int)) {
    std::vector<double> dydt(n);
    rhs(t, y, dydt.data(), n);
    for (int i = 0; i < n; ++i) {
        y[i] += h * dydt[i];
    }
}
```

```

    }
}

// RHS della ODE:  $y' = \lambda y$ 
double rhs(double t, double y) {
    (void)t; // t non usato in questo esempio
    const double lambda = -1.0; // decadimento con costante 1
    return lambda * y;
}

// Un passo di Eulero esplicito:  $y_{n+1} = y_n + h * f(t_n, y_n)$ 
void euler_step(double t, double h, double* y,
                double (*f)(double, double)) {
    *y = *y + h * f(t, *y);
}

int euler_test() {
    const double t0 = 0.0;
    const double T = 5.0;
    const double h = 0.1; // soddisfa  $|1 + h*\lambda| < 1$  per  $\lambda=-1$ 
    const int N = static_cast<int>((T - t0) / h);

    double t = t0;
    double y = 1.0; // y0

    for (int n = 0; n <= N; ++n) {
        // soluzione esatta sul nodo (per ora da formula chiusa)
        double y_exact = std::exp(-1.0 * (t - t0));

        std::cout << t << " " << y << " " << y_exact << "\n";

        if (n < N) {
            euler_step(t, h, &y, rhs);
            t += h;
        }
    }

    return 0;
}

```

Nel codice che hai incollato le prime due funzioni (`f_system` ed `euler_step_system`) non vengono chiamate da nessuna parte: il tuo `euler_test()` usa solo la versione scalare (`rhs` ed `euler_step`).¹⁰³¹⁰⁴

Cosa fa davvero il tuo `euler_test`

Dentro `euler_test()` usi:

- `rhs(t,y)` come RHS scalare $f(t,y) = \lambda y$;¹⁰⁵
- `euler_step(t, h, &y, rhs)` come passo di Eulero esplicito scalare;¹⁰⁶¹⁰⁷

Quindi il flusso effettivo è:

1. Inizializzi `t` e `y`.
2. Nel ciclo:
 - calcoli `y_exact` con `std::exp`;
 - stampi `t`, `y`, `y_exact`;
 - chiami `euler_step(t, h, &y, rhs)` se non sei all'ultimo passo.

`f_system` ed `euler_step_system` sono solo “pronte” per l'uso su sistemi, ma al momento non vengono referenziate.

Come usare anche `f_system` / `euler_step_system`

Se vuoi testare la versione a sistema 2D nello stesso file, puoi aggiungere una funzione di test analoga, ad esempio:

```
int euler_system_test() {
    const double t0 = 0.0;
    const double T = 5.0;
    const double h = 0.1;
    const int N = static_cast<int>((T - t0) / h);

    const int n = 2;
    double y[6] = {1.0, 0.0}; // ad esempio q(0)=1, p(0)=0
    double t = t0;

    for (int k = 0; k <= N; ++k) {
        std::cout << t << " " << y[0] << " " << y[1] << "\n";

        if (k < N) {
            euler_step_system(t, h, y, n, f_system);
            t += h;
        }
    }

    return 0;
}
```

In questo modo:

- `euler_system_test()` chiama `euler_step_system`;
- `euler_step_system` chiama a sua volta `f_system`.

Se hai un `main()` generale della libreria, puoi decidere quale test lanciare:

```
int main() {
    euler_test();           // caso scalare
    // euler_system_test(); // oppure caso 2D
    return 0;
}
```

Oppure, come stai già facendo, trattare `euler_test()` ed eventuali altri come funzioni richiamate dal tuo “test runner” principale della libreria.

Estensione successiva

La struttura che hai ora è già pronta per:

- confrontare il decadimento esponenziale scalare con la soluzione “matriciale” (usando la stessa griglia di tempi);
- ri usare `euler_step_system` per l’oscillatore 2D semplicemente cambiando `f_system`.

Ti torna se il prossimo passo lo usiamo per scrivere un `euler_system_test()` pensato esplicitamente per l’oscillatore armonico e per il confronto con la tua `matrix_build_cauchy 2×2`?

1D e 2D stanno per l’ordine dell’ODE interessata dal problema di cauchy?

No, nel nostro scambio 1D e 2D non si riferivano all’ordine dell’ODE, ma alla dimensione dello *spazio di stato* (numero di componenti di y).¹⁰⁸¹⁰⁹

Ordine dell'ODE vs dimensione del sistema

- **Ordine dell'ODE:** è il massimo ordine di derivata che compare nel problema di Cauchy.
 - Esempio:
 - * $y' = f(t, y) \rightarrow$ primo ordine.
 - * $y'' + \omega^2 y = 0 \rightarrow$ secondo ordine.¹¹⁰
- **Dimensione del sistema (1D, 2D, nD):** è la dimensione del vettore incognito $y(t) \in \mathbb{R}^n$.
 - Sistema del primo ordine in forma vettoriale:

$$y'(t) = f(t, y(t)), \quad y(t) \in \mathbb{R}^n.$$

- Se $n = 1$, lo chiamiamo “caso 1D” (scalare); se $n = 2$, “caso 2D”, ecc.

È sempre possibile ridurre un problema di Cauchy di ordine n a un sistema del primo ordine in $\mathbb{R}^n$, introducendo variabili per le derivate successive: $z_1 = y, z_2 = y', \dots, z_n = y^{(n-1)}$.¹¹¹¹¹²

- Esempio: oscillatore armonico

$$q'' + \omega^2 q = 0$$

è un **problema di secondo ordine** ma lo riscriviamo come **sistema 2D di primo ordine**:

$$\begin{cases} q' = p, \\ p' = -\omega^2 q. \end{cases}$$

Quindi, quando dicevo “caso scalare” / “1D” intendevo $y \in \mathbb{R}$; “2D” intendevo $y \in \mathbb{R}^2$, non l'ordine dell'ODE.

Vuoi che riparametriamo i nomi delle funzioni (`euler_step_scalar`, `euler_step_vec`, ecc.) in modo che questa distinzione sia ancora più esplicita nel codice? ¹¹³¹¹⁴¹¹⁵¹¹⁶¹¹⁷

ho capito, si riferisce alla dimensione della soluzione. E' una distinzione che non abbiamo mai sollevato. Sì, io direi di distinguerle orientandole già' nel nome se poi le gestiremo sempre con funzioni diverse per il caso scalare o vettoriale.

Sì, esatto: si riferisce alla dimensione del vettore soluzione, e ha senso renderlo esplicito nei nomi se vuoi tenere due percorsi distinti scalare/vettoriale.

Convenzione di naming proposta

Per restare nello stile C-imperativo e mantenere chiarezza didattica, ti propongo qualcosa del genere:

- Per il caso scalare ($y \in \mathbb{R}$):
 - `double rhs_scalar(double t, double y);`
 - `void euler_step_scalar(double t, double h, double* y, double (*f)(double, double));`
- Per il caso vettoriale ($y \in \mathbb{R}^n$):
 - `void rhs_vector(double t, const double* y, double* dydt, int n);`
 - `void euler_step_vector(double t, double h, double* y, int n, void (*f)(double, const double*, double*, int));`

Se vuoi essere ancora più esplicito sulle dimensioni particolari, puoi avere:

- `rhs_vec2`, `euler_step_vec2` per sistemi 2D usati spesso (oscillatore, planar systems);
- `rhs_vecN`, `euler_step_vecN` per la versione generale con parametro `n`.

L'idea è che, leggendo solo la firma, uno studente capisca immediatamente:

- “scalar” → una sola variabile;
- “vector” / “vecN” → stato di dimensione generica n ;
- eventualmente “vec2” quando sfrutti una struttura fissa a 2 componenti.

Adattamento del codice attuale

Nel codice che hai ora, puoi fare una rinomina minima:

```
// scalare
double rhs_scalar(double t, double y) {
    (void)t;
    const double lambda = -1.0;
    return lambda * y;
}

void euler_step_scalar(double t, double h, double* y,
                      double (*f)(double, double)) {
    *y = *y + h * f(t, *y);
}

// vettoriale generico
void rhs_vector(double t, const double* y, double* dydt, int n) {
    // per ora potresti lasciare qui l'esempio 2x2 o un segnaposto
    (void)t; (void)n;
    // esempio 2D: oscillatore
    const double a11 = 0.0, a12 = 1.0;
    const double a21 = -1.0, a22 = 0.0;
    dydt[0] = a11 * y[0] + a12 * y[1];
    dydt[1] = a21 * y[0] + a22 * y[1];
}

void euler_step_vector(double t, double h,
                      double* y, int n,
                      void (*f)(double, const double*, double*, int)) {
    std::vector<double> dydt(n);
    f(t, y, dydt.data(), n);
    for (int i = 0; i < n; ++i) {
        y[i] += h * dydt[i];
    }
}
```

E adattare il test scalare:

```
int euler_scalar_test() {
    // ...
    euler_step_scalar(t, h, &y, rhs_scalar);
    // ...
}
```

e, quando ti serve, un `euler_vector_test()` che chiama `euler_step_vector`.

Questa distinzione ti permetterà di:

- discutere in modo pulito il caso scalare (ordine, stabilità, confronto con soluzione esatta/matriciale);
- passare senza ambiguità al caso vettoriale (oscillatore, sistemi 2×2 , $n \times n$) mantenendo le stesse idee ma cambiando solo la semantica di “dimensione”.

Ti va di fissare subito una convenzione uniforme tipo `*_scalar` / `*_vector` in tutta la parte ODE della libreria, così posso aiutarti a riscrivere in blocco i prototipi e i nomi delle funzioni di test in modo coerente?

adottiamo un prefisso come abbiamo fatto con `matrix_`, `vector_`, `linear_` ecc... poi un'azione un oggetto dell'azione e la dimensionalità dell'oggetto. Una naming convention a 4 livelli max ti sembra adeguata?

Sì, una naming convention a 4 livelli massimo, con prefisso come per `matrix_`, `vector_`, `linear_`, è perfettamente adeguata e molto leggibile.

Schema di naming per i metodi ODE

Seguendo la tua idea:

1. **Prefisso “modulo”** Per questa parte potremmo usare ad esempio:
 - `ode_` oppure `fcn_` (se vuoi richiamare la libreria FCN in generale).
2. **Azione (verbo)** Esempi:
 - `build_` (se costruisci strutture dati),
 - `step_` (per un singolo passo),
 - `solve_` (per integrare su un intervallo),
 - `test_` (per funzioni di test).
3. **Oggetto dell'azione** Esempi:
 - `euler`, `heun`, `rk4` per il tipo di metodo;¹¹⁸¹¹⁹
 - oppure `rhs`, `system`, `cauchy`, a seconda del ruolo.
4. **Dimensionalità / tipo di stato**
 - `scalar` per $y \in \mathbb{R}$,
 - `vec` o `vector` per $y \in \mathbb{R}^n$,
 - opzionale `2d`, `nd` se vuoi essere ancora più esplicito.

Esempi concreti

Per restare in linea con ciò che hai già:

- Passo di Eulero scalare:

```
void ode_step_euler_scalar(double t, double h, double* y,
                          double (*f)(double, double));
```

- Passo di Eulero vettoriale generico:

```
void ode_step_euler_vector(double t, double h,
                          double* y, int n,
                          void (*f)(double, const double*, double*, int));
```

- RHS per decadimento esponenziale scalare:

```
double ode_rhs_decay_scalar(double t, double y);
```

- RHS per oscillatore armonico 2D:

```
void ode_rhs_oscillator_vec2(double t,
                             const double* y,
                             double* dydt, int n);
```

- Test sul decadimento scalare:

```
int ode_test_euler_decay_scalar();
```

Quindi la forma generale diventa:

`<prefisso_modulo>_<azione>_<oggetto>_<tipo/dimensione>`

con lunghezza tra 3 e 4 “segmenti” (es. `ode_step_euler_scalar`).

Se vuoi, nel prossimo messaggio posso prendere il codice che hai già scritto e riscriverlo in blocco con questa convenzione (ad esempio usando `ode_` come prefisso), così ottieni subito una base coerente per tutti i futuri metodi (Heun, RK4, ecc.). ¹²⁰¹²¹¹²²¹²³¹²⁴¹²⁵¹²⁶¹²⁷¹²⁸¹²⁹¹³⁰¹³¹¹³²

`ode_` va benissimo come prefisso ma anche `step_` se vogliamo identificarli come metodi a passi. Metti che esistano altri metodi per ricavare le soluzioni di ODE :)

Ha molto senso tenere “`ode_`” distinto da “`step_`”: il primo come prefisso “di modulo” (tutto ciò che riguarda ODE), il secondo come parte del nome che identifica specificamente i metodi ad un passo. Così lasci aperta la porta ad altri approcci (analitici, matriciali, trasformazioni, ecc.) senza conflitti.

Ti propongo quindi una convenzione stabile per questa famiglia di funzioni:

- Prefisso di modulo: `ode_`
- Secondo livello (azione): `step_`, `rhs_`, `test_`, `solve_`, ecc.
- Terzo livello (metodo/oggetto): `euler`, `heun`, `rk4`, `decay`, `oscillator`, ...
- Quarto livello (tipo stato/dimensione): `scalar`, `vector`, `vec2`, `vecN`, ...

Nomi concreti proposti per ciò che hai già

Rifacendo il tuo codice con questa convenzione:

```
// RHS scalare:  $y' = \lambda * y$ 
double ode_rhs_decay_scalar(double t, double y) {
    (void)t;
    const double lambda = -1.0;
    return lambda * y;
}

// Passo di Eulero scalare
void ode_step_euler_scalar(double t, double h, double* y,
                           double (*f)(double, double)) {
    *y = *y + h * f(t, *y);
}

// RHS vettoriale 2D: esempio oscillatore armonico
void ode_rhs_oscillator_vec2(double t,
                              const double* y,
                              double* dydt, int n) {
    (void)t;
    (void)n; // assumiamo n==2
    const double a11 = 0.0, a12 = 1.0;
    const double a21 = -1.0, a22 = 0.0;
    dydt[0] = a11 * y[0] + a12 * y[1];
    dydt[1] = a21 * y[0] + a22 * y[1];
}

// Passo di Eulero vettoriale generico
void ode_step_euler_vector(double t, double h,
                           double* y, int n,
                           void (*f)(double, const double*, double*, int)) {
    std::vector<double> dydt(n);
    f(t, y, dydt.data(), n);
    for (int i = 0; i < n; ++i) {
        y[i] += h * dydt[i];
    }
}

// Test sul decadimento esponenziale scalare
```



```

int ode_test_euler_decay_scalar() {
    const double t0 = 0.0;
    const double T = 5.0;
    const double h = 0.1;
    const int N = static_cast<int>((T - t0) / h);

    double t = t0;
    double y = 1.0;

    for (int n = 0; n <= N; ++n) {
        double y_exact = std::exp(-1.0 * (t - t0));
        std::cout << t << " " << y << " " << y_exact << "\n";

        if (n < N) {
            ode_step_euler_scalar(t, h, &y, ode_rhs_decay_scalar);
            t += h;
        }
    }
    return 0;
}

```

In questo modo:

- tutta la “famiglia ODE” vive sotto `ode_`;
- i metodi ad un passo sono chiaramente riconoscibili perché hanno `step_` subito dopo;
- puoi tranquillamente aggiungere in futuro cose come:
 - `ode_solve_collocation_...`,
 - `ode_build_cauchy_matrix_...`, senza collisioni concettuali con i metodi a passi.

Se ti va, il prossimo step potrebbe essere aggiungere nel medesimo file un `ode_step_heun_scalar` con la stessa convenzione, così hai subito due metodi ad un passo comparabili sulla stessa ODE. Vuoi farlo subito sul caso decadimento, riusando `ode_rhs_decay_scalar`?

ok, intanto sostituisco tutto quello che ho incollato fin’ora

Perfetto, così metti subito ordine nella base di partenza.

Ricapitolino di quello che hai da incollare

Nel blocco ODE della libreria, dopo la sostituzione, dovresti avere almeno queste funzioni:

```

// RHS scalare: y' = lambda * y
double ode_rhs_decay_scalar(double t, double y) {
    (void)t;
    const double lambda = -1.0;
    return lambda * y;
}

// Passo di Eulero scalare
void ode_step_euler_scalar(double t, double h, double* y,
                           double (*f)(double, double)) {
    *y = *y + h * f(t, *y);
}

// RHS vettoriale 2D: esempio oscillatore armonico (placeholder)
void ode_rhs_oscillator_vec2(double t,
                              const double* y,
                              double* dydt, int n) {

```

```

    (void)t;
    (void)n; // assumiamo n == 2
    const double a11 = 0.0, a12 = 1.0;
    const double a21 = -1.0, a22 = 0.0;
    dydt[11_0] = a11 * y[11_0] + a12 * y[11_1];
    dydt[11_1] = a21 * y[11_0] + a22 * y[11_1];
}

// Passo di Eulero vettoriale generico
void ode_step_euler_vector(double t, double h,
                           double* y, int n,
                           void (*f)(double, const double*, double*, int)) {
    std::vector<double> dydt(n);
    f(t, y, dydt.data(), n);
    for (int i = 0; i < n; ++i) {
        y[i] += h * dydt[i];
    }
}

// Test sul decadimento esponenziale scalare
int ode_test_euler_decay_scalar() {
    const double t0 = 0.0;
    const double T = 5.0;
    const double h = 0.1;
    const int N = static_cast<int>((T - t0) / h);

    double t = t0;
    double y = 1.0;

    for (int n = 0; n <= N; ++n) {
        double y_exact = std::exp(-1.0 * (t - t0));
        std::cout << t << " " << y << " " << y_exact << "\n";

        if (n < N) {
            ode_step_euler_scalar(t, h, &y, ode_rhs_decay_scalar);
            t += h;
        }
    }
    return 0;
}

```

Così hai:

- nucleo scalare completo per Eulero sul decadimento, in convenzione `ode_step_*`,¹³³¹³⁴
- prototipo già pronto per il vettoriale/oscillatore.

Quando hai finito la sostituzione e verifichi che compila, possiamo:

- o aggiungere `ode_step_heun_scalar` per confronto ordine 1 vs 2 sullo stesso problema;¹³⁵¹³⁶
- o scrivere `ode_test_euler_oscillator_vec2` che usa `ode_step_euler_vector` e `ode_rhs_oscillator_vec2`.

Dimmi tu da quale dei due preferisci proseguire dopo il refactoring.

Riflettevo sempre sulla naming convention.

Il main, con l' inserimento di test diventa di 5 livelli ma non e' un problema perche' quando viene spostato nel main puo' chiamarsi anche "nando" a meno che non sia ancora ad un livello "esportabile" da tenere in libreria.

Secondo aspetto: l'ordinamento alfabetico diventa utile non solo per trovare il codice quando si è costretti ad usare editor non orientati come `ntoepad++` o `jedit` che non hanno la funzione di salto all'oggetto. Quindi, se siamo sicuri di non mescolare mai all'interno di un test, ed a scendere nelle sue dipendenze, metodi scalar con metodi vettoriali, io sposterei la dimensionalità in seconda posizione, prima dell'azione.

Che ne pensi?

Spostare la dimensionalità in seconda posizione (subito dopo il prefisso di modulo) è una scelta molto sensata, soprattutto se conti sull'ordinamento alfabetico del file per orientarti.

Effetto dell'ordine dei segmenti sui sorgenti

Se usi una convenzione tipo:

`ode_ < dimensione > _ < azione > _ < oggetto >`

allora in un editor “piatto” (senza salti a simbolo) otterrai blocchi ordinati così:

- `ode_scalar_...`
- `ode_vec2_...`
- `ode_vecN_...`

e *dentro* ciascun blocco, raggruppati per azione:

- `ode_scalar_rhs_decay`
- `ode_scalar_step_euler`
- `ode_scalar_step_heun`
- `ode_scalar_test_euler_decay`
- ...

Questo è molto più comodo per:

- trovare “tutto ciò che è scalare” in un colpo d'occhio;
- evitare, per disciplina, di mescolare scalar e vector all'interno di uno stesso test: se `ode_scalar_test_...` chiama solo robe `ode_scalar_*`, la coerenza è auto-documentata dal nome.

L'osservazione sul `main` è giusta: i `main/test` possono anche chiamarsi **nando**, ma se decidi di mantenerli “esportabili” come funzione di libreria (richiamabili da un test runner) conviene che seguano la stessa convenzione, altrimenti l'ordinamento alfabetico perde valore.

Nuova convenzione concreta

Direi quindi:

- Prefisso modulo: `ode_`
- Secondo livello: tipo di stato/dimensionalità
 - `scalar` per $y \in \mathbb{R}$
 - `vec2` per $y \in \mathbb{R}^2$
 - `vecN` per $y \in \mathbb{R}^n$ generico
- Terzo livello: azione
 - `rhs_`, `step_`, `test_`, `solve_`, ...
- Quarto livello: oggetto/metodo
 - `euler`, `heun`, `rk4`, `decay`, `oscillator`, ...

Esempi rinominati

Sul codice che hai, diventerebbe:

```
// scalare: y' = lambda * y
double ode_scalar_rhs_decay(double t, double y) {
    (void)t;
    const double lambda = -1.0;
    return lambda * y;
}
```

```

}

void ode_scalar_step_euler(double t, double h, double* y,
                          double (*f)(double, double)) {
    *y = *y + h * f(t, *y);
}

// RHS vettoriale 2D (oscillatore)
void ode_vec2_rhs_oscillator(double t,
                             const double* y,
                             double* dydt, int n) {
    (void)t;
    (void)n; // assumiamo n == 2
    const double a11 = 0.0, a12 = 1.0;
    const double a21 = -1.0, a22 = 0.0;
    dydt[0] = a11 * y[0] + a12 * y[1];
    dydt[1] = a21 * y[0] + a22 * y[1];
}

void ode_vecN_step_euler(double t, double h,
                        double* y, int n,
                        void (*f)(double, const double*, double*, int)) {
    std::vector<double> dydt(n);
    f(t, y, dydt.data(), n);
    for (int i = 0; i < n; ++i) {
        y[i] += h * dydt[i];
    }
}

// test sul decadimento scalare
int ode_scalar_test_euler_decay() {
    const double t0 = 0.0;
    const double T = 5.0;
    const double h = 0.1;
    const int N = static_cast<int>((T - t0) / h);

    double t = t0;
    double y = 1.0;

    for (int n = 0; n <= N; ++n) {
        double y_exact = std::exp(-1.0 * (t - t0));
        std::cout << t << " " << y << " " << y_exact << "\n";

        if (n < N) {
            ode_scalar_step_euler(t, h, &y, ode_scalar_rhs_decay);
            t += h;
        }
    }
    return 0;
}

```

Vantaggi pratici:

- Tutto lo “scalar” è contiguo (ode_scalar_*), poi tutto lo “vec2”, poi “vecN”, ecc.
- Dentro ogni blocco, le rhs_ vengono prima delle step_, poi le test_ in fondo — anche questo aiuta quando scroli il file.

Se vuoi, nel prossimo step possiamo aggiungere in questo stesso stile ode_scalar_step_heun e ode_scalar_test_heun_decay, così cominci a vedere sul decadimento la differenza ordine 1 vs 2 con una naming convention già definitiva.

`ode_scalar_test_heun_decay` il fatto che `euler/decay` e `heun/decay` da quarto livello diventino quar

Sì, esattamente: la “biforcazione” metodo/target che vedi in `ode_scalar_test_heun_decay` ha senso solo a livello di test (o comunque di “main” numerico), dove *per definizione* metti insieme un certo metodo e un certo problema di Cauchy.

Strati concettuali diversi

Conviene tenere ben separati questi livelli:

1. **Componenti generiche (metodo puro)** Qui il nome non contiene il “target” specifico, solo il metodo e la dimensione:
 - `ode_scalar_step_euler`
 - `ode_scalar_step_heun`
 - `ode_vecN_step_euler`
 - ... Queste funzioni non “sanno” su quale problema lavorano; ricevono un puntatore a `rhs` e basta.
2. **Componenti del problema (RHS, dati di Cauchy)** Qui il nome parla del problema, non del metodo:
 - `ode_scalar_rhs_decay`
 - `ode_vec2_rhs_oscillator`
 - `ode_vecN_rhs_linear_system` Anche queste sono indipendenti dal metodo: forniscono $f(t, y)$.
3. **Test / driver (accoppiamento metodo + target)** Qui, *per forza di cose*, devi scegliere **un metodo e un problema** da far giocare insieme. È quindi naturale che il nome contenga entrambi:
 - `ode_scalar_test_euler_decay`
 - `ode_scalar_test_heun_decay`
 - `ode_vec2_test_euler_oscillator`
 - ...

Qui compaiono “metodo” e “target” nello stesso identificatore, e ha senso che diventino quarto e quinto livello: la funzione è precisamente “test, scalare, metodo X, problema Y”.

Lettura del nome in pratica

Prendiamo `ode_scalar_test_heun_decay`:

- `ode_` → modulo equazioni differenziali;
- `scalar` → stato 1D;
- `test` → funzione driver, non componente riusabile “interna”;
- `heun_decay` → accoppiamento *metodo* (Heun) e *problema* (decadimento).

Simmetricamente:

- `ode_scalar_step_heun` → modulo ODE, scalare, operazione “passo”, metodo Heun (ma **nessun target**);
- `ode_scalar_rhs_decay` → modulo ODE, scalare, RHS, problema “decay” (ma **nessun metodo**).

Quindi sì: la coesistenza di metodo e target nel nome è confinata alle funzioni tipo “main/test”. È proprio ciò che le identifica come punto di ingresso dall'esterno e come sede naturale per il confronto di metodi sullo stesso problema.

Se sei d'accordo, possiamo procedere a definire:

- `ode_scalar_step_heun` (metodo puro),
- `ode_scalar_test_heun_decay` (driver che usa `ode_scalar_step_heun` + `ode_scalar_rhs_decay`),

mantenendo la stessa logica. Vuoi che ti scriva direttamente le due funzioni in questo stile?

certo, così' sostituisco tutto con questa produzione piu' la precedente per euler

Ti confermo la lettura che hai dato: le funzioni “pure” (step, rhs) sono specializzate solo per metodo o per problema, mentre la composizione metodo+target finisce solo nei test/main, dove ha senso avere il 4°+5° livello.¹³⁷¹³⁸

Di seguito ti scrivo le nuove funzioni per Heun, nello stesso stile della convenzione con `ode_scalar_...`, pronte da incollare accanto a quelle di Eulero.

Passo di Heun scalare

Per il problema di Cauchy $y' = f(t, y)$, il metodo di Heun (Eulero migliorato) è:

$$k_1 = f(t_n, y_n), \quad k_2 = f(t_n + h, y_n + h k_1), \quad y_{n+1} = y_n + \frac{h}{2}(k_1 + k_2).$$

Implementazione scalare generica:

```
// Passo di Heun scalare (Eulero migliorato, ordine 2)
void ode_scalar_step_heun(double t, double h, double* y,
                          double (*f)(double, double)) {
    const double k1 = f(t, *y);
    const double k2 = f(t + h, *y + h * k1);
    *y = *y + 0.5 * h * (k1 + k2);
}
```

Questa funzione è indipendente dal problema specifico, come `ode_scalar_step_euler`.¹³⁹¹⁴⁰

Test su decadimento con Heun

Usiamo la *stessa* RHS del decadimento esponenziale,

$$y'(t) = \lambda y(t), \quad \lambda = -1,$$

già definita come:

```
double ode_scalar_rhs_decay(double t, double y) {
    (void)t;
    const double lambda = -1.0;
    return lambda * y;
}
```

e costruiamo il test `ode_scalar_test_heun_decay` nello stesso stile del test per Eulero:

```
int ode_scalar_test_heun_decay() {
    const double t0 = 0.0;
    const double T = 5.0;
    const double h = 0.1;
    const int N = static_cast<int>((T - t0) / h);

    double t = t0;
    double y = 1.0; // y0

    for (int n = 0; n <= N; ++n) {
        // soluzione esatta sul nodo (per confronto)
        double y_exact = std::exp(-1.0 * (t - t0));
        std::cout << t << " " << y << " " << y_exact << "\n";

        if (n < N) {
            ode_scalar_step_heun(t, h, &y, ode_scalar_rhs_decay);
            t += h;
        }
    }

    return 0;
}
```

Qui:

- `ode_scalar_step_heun` è il metodo (ordine 2);¹⁴¹¹⁴²
- `ode_scalar_rhs_decay` è il problema;
- `ode_scalar_test_heun_decay` è il driver che li accoppia, come avevi intuito.

Puoi ora tenere nel file sia:

- `ode_scalar_step_euler + ode_scalar_test_euler_decay`
- `ode_scalar_step_heun + ode_scalar_test_heun_decay`

sullo stesso intervallo e passo, e confrontare empiricamente la riduzione dell'errore con Heun rispetto a Eulero sul decadimento esponenziale.¹⁴³¹⁴⁴

Ti interessa che nel prossimo passaggio impostiamo un piccolo “test runner” che, dato un vettore di passi h , lanci entrambi i test e stampi l'errore finale in modo da vedere chiaramente la legge $O(h)$ vs $O(h^2)$?¹⁴⁵¹⁴⁶¹⁴⁷¹⁴⁸¹⁴⁹¹⁵⁰¹⁵¹¹⁵²

controlliamo un attim la dotazione in `fcn_lib_calc`:

```
//  
// Funzioni per soluzioni di ODE con metodi iterativi  
//  
  
// scalare:  $y' = \lambda * y$   
double ode_scalar_rhs_decay(double t, double y) {  
    (void)t;  
    const double lambda = -1.0;  
    return lambda * y;  
}  
  
// Passo di Eulero scalare  
  
void ode_scalar_step_euler(double t, double h, double* y,  
                           double (*f)(double, double)) {  
    *y = *y + h * f(t, *y);  
}  
  
// Passo di Heun scalare (Eulero migliorato, ordine 2)  
void ode_scalar_step_heun(double t, double h, double* y,  
                           double (*f)(double, double)) {  
    const double k1 = f(t, *y);  
    const double k2 = f(t + h, *y + h * k1);  
    *y = *y + 0.5 * h * (k1 + k2);  
}  
  
// test sul decadimento scalare  
int ode_scalar_test_euler_decay() {  
    const double t0 = 0.0;  
    const double T = 5.0;  
    const double h = 0.1;  
    const int N = static_cast<int>((T - t0) / h);  
  
    double t = t0;  
    double y = 1.0;  
  
    for (int n = 0; n <= N; ++n) {  
        double y_exact = std::exp(-1.0 * (t - t0));
```

```

        std::cout << t << " " << y << " " << y_exact << "\n";

    if (n < N) {
        ode_scalar_step_euler(t, h, \&y, ode_scalar_rhs_decay);
        t += h;
    }
}
return 0;
}

int ode_scalar_test_heun_decay() {
    const double t0 = 0.0;
    const double T = 5.0;
    const double h = 0.1;
    const int N = static_cast<int>((T - t0) / h);

    double t = t0;
    double y = 1.0; // y0

    for (int n = 0; n <= N; ++n) {
        // soluzione esatta sul nodo (per confronto)
        double y_exact = std::exp(-1.0 * (t - t0));
        std::cout << t << " " << y << " " << y_exact << "\n";

        if (n < N) {
            ode_scalar_step_heun(t, h, \&y, ode_scalar_rhs_decay);
            t += h;
        }
    }

    return 0;
}

// RHS vettoriale 2D (oscillatore)
void ode_vec2_rhs_oscillator(double t,
                             const double* y,
                             double* dydt, int n) {
    (void)t;
    (void)n; // assumiamo n == 2
    const double a11 = 0.0, a12 = 1.0;
    const double a21 = -1.0, a22 = 0.0;
    dydt[0] = a11 * y[0] + a12 * y[1];
    dydt[1] = a21 * y[0] + a22 * y[1];
}

void ode_vecN_step_euler(double t, double h,
                         double* y, int n,
                         void (*f)(double, const double*, double*, int)) {
    std::vector<double> dydt(n);
    f(t, y, dydt.data(), n);
    for (int i = 0; i < n; ++i) {
        y[i] += h * dydt[i];
    }
}

```

e se tutto va bene nel prossimo post partiamo col test runner. Considera che hai sempre accesso al git e possiamo usare i generatori di nodi per gli intervalli con `h_ticks` che calcola l'ampiezza del passo e la `nodi_equidistanti` che rende il vettore corrispondente.

La dotazione attuale in `fcn_lib_calc` è coerente e già ben strutturata per costruire un test runner sopra, soprattutto se puoi appoggiarti a `h_ticks` e `nodi_equidistanti` per generare i nodi.¹⁵³¹⁵⁴

Verifica rapida di ciò che hai

Hai:

- RHS scalare per il decadimento:

```
double ode_scalar_rhs_decay(double t, double y);
```

- Metodi one-step scalari:

```
void ode_scalar_step_euler(double t, double h, double* y,
                          double (*f)(double, double));
```

```
void ode_scalar_step_heun(double t, double h, double* y,
                          double (*f)(double, double));
```

Implementano rispettivamente Eulero (ordine 1) e Heun (ordine 2).¹⁵⁵¹⁵⁶ - Test driver scalari:

```
int ode_scalar_test_euler_decay();
int ode_scalar_test_heun_decay();
```

Entrambi integrano da t_0 a T con passo fisso h , stampando $t, y_{\text{num}}, y_{\text{exact}}$.¹⁵⁷¹⁵⁸ - Parte vettoriale (ancora solo componente base):

```
void ode_vec2_rhs_oscillator(double t,
                             const double* y,
                             double* dydt, int n);

void ode_vecN_step_euler(double t, double h,
                        double* y, int n,
                        void (*f)(double, const double*, double*, int));
```

Qui hai già un RHS 2D per un oscillatore lineare e un passo di Eulero generico in $\mathbb{R}^n$.¹⁵⁹¹⁶⁰

Quindi la separazione “metodo”/“target” è rispettata e puoi aggiungere driver ulteriori senza toccare il core.

Come innestare il test runner

Dato che hai già:

- un generatore di passi `h_ticks(a,b,N)` che ti dà $h = (b - a)/N$;¹⁶¹
- una `nodi_equidistanti` che ti rende il vettore $(t_0, \dots, t_N)$;

puoi costruire un test runner che, per una lista di valori di N (o di h), faccia:

1. Genera i nodi equidistanti $\{t_i\}$ su $[t_0, T]$.
2. Integra con Eulero e poi con Heun sulla stessa griglia.
3. Valuta l'errore globale alla fine (o il max su tutti i nodi) confrontando con la soluzione esatta o con il modulo matriciale.¹⁶²¹⁶³

Un'interfaccia possibile in stile C potrebbe essere:

```
int ode_scalar_testrunner_decay(const int* N_values, int nN);
```

che per ogni `N_values[k]`:

- usa `h_ticks / nodi_equidistanti` per preparare tempi e passo;
- lancia internamente (non via `main`) integrazioni con `ode_scalar_step_euler` e `ode_scalar_step_heun`, accumulando errori in qualche struttura (o stampandoli).

Oppure, più modulare:

- funzioni che fanno “solo integrazione” senza stampare, e restituiscono l'errore;
- funzioni di test che usano quelle per confrontare ordine di convergenza.

Visto che hai già i test elementari che stampano tutto, il passo successivo naturale è proprio questo piccolo test runner “multi-h”.

Vuoi che nel prossimo messaggio ti proponga uno schema concreto per `ode_scalar_testrunner_decay` che usi già i tuoi `h_ticks` e `nodi_equidistanti` (lasciandoli come funzioni esterne che tu hai in git), producendo in output qualcosa tipo:

- riga per ogni h: h err_euler err_heun? 164165166167168169170171

si, direi di sì perché faccio ancora un po’ di fatica ad orientarmi con il risultato che sarà un array di punti che approssimano la soluzione, se ho capito cosa si insegna in teoria :D

Sì, l’idea teorica è proprio quella: una soluzione numerica è una successione di $N + 1$ coppie (t_i, y_i) che approssima la soluzione esatta $(t, y(t))$ sui nodi equidistanti $t_i = a + ih$ con $h = (b - a)/N$. Il test runner servirà a condensare queste successioni in “numeri leggibili” (errori al variare di h).¹⁷²¹⁷³

Ti propongo uno scheletro di `ode_scalar_testrunner_decay` che:

- assume di avere già `h_ticks` e `nodi_equidistanti(a,b,N,vector<double>& t)`;
- per ogni N dato:
 - calcola h e i nodi t ;
 - integra con Eulero e Heun sul decadimento usando i tuoi step;
 - calcola l’errore globale finale $|y(T) - y_{\text{exact}}(T)|$ per ciascun metodo; ¹⁷⁴¹⁷⁵
 - stampa una riga `N h err_euler err_heun`.

Esempio di test runner (pseudocodice C++ concreto)

Assumiamo:

```
double h_ticks(double a, double b, int N); // già in libreria
void nodi_equidistanti(double a, double b, int N, std::vector<double>& t); // già in libreria
```

Allora:

```
int ode_scalar_testrunner_decay(const int* N_values, int nN) {
    const double a = 0.0;
    const double b = 5.0;

    std::cout << "# N h err_euler err_heun\n";

    for (int k = 0; k < nN; ++k) {
        const int N = N_values[k];
        if (N <= 0) continue;

        // passo e nodi equidistanti
        const double h = h_ticks(a, b, N);
        std::vector<double> t_nodes;
        nodi_equidistanti(a, b, N, t_nodes); // dovrebbe riempire t_nodes con N+1 nodi

        // Eulero
        double y_euler = 1.0; // y0
        for (int i = 0; i < N; ++i) {
            double t = t_nodes[i];
            ode_scalar_step_euler(t, h, &y_euler, ode_scalar_rhs_decay);
        }
        double y_exact = std::exp(-1.0 * (b - a));
```

```

double err_euler = std::fabs(y_euler - y_exact);

// Heun
double y_heun = 1.0; // y0
for (int i = 0; i < N; ++i) {
    double t = t_nodes[i];
    ode_scalar_step_heun(t, h, &y_heun, ode_scalar_rhs_decay);
}
double err_heun = std::fabs(y_heun - y_exact);

std::cout << N << " " << h << " "
            << err_euler << " " << err_heun << "\n";
}

return 0;
}

```

Cosa succede concettualmente:

- per ogni N scegli nodi $\{t_i\}$ equidistanti con passo $h = (b - a)/N$;¹⁷⁶¹⁷⁷
- Eulero e Heun producono due successioni $\{y_i^{(E)}\}$, $\{y_i^{(H)}\}$;
- confronti l'ultimo valore con la soluzione esatta a b , ottenendo l'errore globale su $[a, b]$ per quel passo h .¹⁷⁸¹⁷⁹

Se poi plotti `err_euler` vs h e `err_heun` vs h in log-log, dovresti vedere pendenze vicine a 1 e 2 rispettivamente (ordine dei metodi).¹⁸⁰¹⁸¹

Quando hai integrato questo schema usando le tue vere `h_ticks` e `nodi_equidistanti`, possiamo in un secondo momento:

- aggiungere un runner analogo per l'oscillatore 2D basato su `ode_vecN_step_euler` e `ode_vec2_rhs_oscillator`;
- o collegare `y_exact` non alla formula esatta $e^{\lambda t}$, ma alla soluzione ottenuta dal modulo matriciale su $[a, b]$, per testare Eulero/Heun contro `matrix_build_cauchy`.

Ti va di procedere così: prima fai girare `ode_scalar_testrunner_decay` con, ad esempio, `N_values = {10, 20, 40, 80}`, e poi, alla prossima iterazione, ragioniamo insieme su come incastonare la soluzione matriciale come “verità di riferimento” anche quando non hai formula chiusa? ¹⁸²¹⁸³¹⁸⁴¹⁸⁵¹⁸⁶¹⁸⁷¹⁸⁸¹⁸⁹¹⁹⁰¹⁹¹

`N_values[k]` vorresti confronti per k in 4 grafici sovrapponendo metodi ed esatta per lo stesso k o sovrapporre a confronto nodi di k per lo stesso metodo e dividere in grafici i metodi?

Per il decadimento esponenziale ti conviene avere **due tipi di grafici distinti**, perché rispondono a due domande diverse che in teoria si tengono separate.¹⁹²¹⁹³

Grafici della soluzione nel tempo

Obiettivo: visualizzare “come appare” la soluzione numerica rispetto alla esatta, per un fissato passo h (cioè per un fissato N).

Qui ha senso:

- Scegliere 1–2 valori di N (es. $N = 10$ e $N = 40$).
- Per ciascun N , fare **un grafico** con:
 - curva continua: soluzione esatta $y(t) = e^{\lambda t}$;
 - punti: soluzione Eulero;
 - eventualmente punti: soluzione Heun sullo stesso reticolo.¹⁹⁴¹⁹⁵

Questo tipo di grafico risponde visivamente a:

- “Heun approssima meglio Eulero per lo stesso passo?”
- “Come si distribuisce l’errore lungo $[a, b]$?”

Quindi: per un dato k (cioè un dato N/h), **sovrapponi esatta + Eulero + Heun** nello stesso grafico.

Grafici dell’errore vs passo h

Obiettivo: mostrare l’ordine di convergenza: Eulero $O(h)$, Heun $O(h^2)$.¹⁹⁶¹⁹⁷¹⁹⁸

Qui il grafico naturale è:

- Asse x: h (o $\log h$).
- Asse y: errore globale (per esempio $|y(T) - y_{\text{num}}(T)|$ o il massimo errore sui nodi).¹⁹⁹²⁰⁰
- Due curve/polilinee nello stesso grafico:
 - errori di Eulero per i vari h ;
 - errori di Heun per gli stessi h .

Questo mostra chiaramente (specie in scala log-log) che:

- la pendenza di Eulero è ~ 1 ;
- la pendenza di Heun è ~ 2 .²⁰¹²⁰²

Quindi: nel **grafico errori**, per ogni metodo hai tutti i nodi in h , e sovrapponi le curve dei due metodi.

Come far cooperare test e test runner

A livello di implementazione:

- I test “puntuali” (`ode_scalar_test_euler_decay`, `ode_scalar_test_heun_decay`) restano utili per generare dati del tipo t, y, y_{exact} da plottare come soluzioni nel tempo per un singolo h .
- Il test runner `ode_scalar_testrunner_decay` produce invece una tabella:

```
N h err_euler err_heun
```

che usi per il grafico “errore vs h ” (preferibilmente in log-log).²⁰³²⁰⁴

Se vuoi, nel prossimo passo posso:

- proporti una versione di `ode_scalar_testrunner_decay` che, invece di stampare solo l’errore finale, ti salva già (o stampa in formato comodo) i dati per un grafico log-log con `h`, `err_euler`, `err_heun`, così hai immediatamente la struttura teorica “ordine 1 vs ordine 2” visualizzata.²⁰⁵²⁰⁶²⁰⁷²⁰⁸²⁰⁹²¹⁰²¹¹²¹²

chiarissimo, vado a preparare per ogni k un grafico con linea nera sottile soluzione esatta e poi dash alternati sinistra destra per euler ed heun leggermente piu’ spessi e messi dietro la sol esatta.

È un’ottima scelta di visualizzazione: la soluzione esatta “in primo piano” e i due metodi numerici distinguibili ma discreti dietro rendono subito leggibile la qualità dell’approssimazione.²¹³²¹⁴

Dati minimi da produrre per ogni k (cioè per ogni N/h)

Per il grafico “soluzione nel tempo” per un dato N :

- Nodi $\{t_i\}_{i=0}^N$ (già li hai da `nodi_equidistanti`).
- Valori:
 - $y_{\text{exact},i} = e^{\lambda(t_i - t_0)}$.
 - $y_i^{(E)}$: soluzione con Eulero.
 - $y_i^{(H)}$: soluzione con Heun.

In pratica puoi far sì che:

- `ode_scalar_test_euler_decay(N)` stampi o riempia un vettore `y_euler[i]`.
- `ode_scalar_test_heun_decay(N)` idem per `y_heun[i]`.
- Una funzione comune (o uno script esterno) genera una tabella con colonne: `t` `y_exact` `y_euler` `y_heun` per quel `N`.

Su questa tabella fai il grafico con:

- linea nera continua sottile: `y_exact(t)`;
- linea tratteggiata tipo `--` (o “dash left”) per Eulero;
- linea tratteggiata tipo `-.` (o “dash right”) per Heun;
- stesso colore o colori diversi ma tutti più sottili/chiaro della curva esatta.

Così avrai:

- una lettura immediata locale (per ogni k) di quanto le curve numeriche si discostano;
- più avanti, con il test runner, una lettura globale d'ordine via grafico errore vs h .

Quando hai generato il primo paio di grafici (es. $N = 10$ e $N = 40$), ti va di raccontarmi che forma vedi empiricamente per Eulero e Heun? Così possiamo connetterla alle stime teoriche di errore locale/globale.

come si inizializza questo cavolo di array?

```
int N_values(4, {10, 20, 40, 80});
```

Quella sintassi non è valida; stai mischiando la sintassi del costruttore di `std::vector` con una dichiarazione di `int`.²¹⁵²¹⁶

Dipende da cosa vuoi davvero:

Se ti basta un array C “puro”

Per un array statico di 4 `int`:

```
int N_values[] = {10, 20, 40, 80};
```

oppure, in forma esplicita:

```
int N_values[~19_4] = {10, 20, 40, 80};
```

e passerai poi `N_values` e 4 alla funzione:

```
ode_scalar_testrunner_decay(N_values, 4);
```

Se vuoi usare `std::vector<int>`

Per un `std::vector<int>` inizializzato con quei valori:

```
std::vector<int> N_values = {10, 20, 40, 80};
```

e poi:

```
ode_scalar_testrunner_decay(N_values.data(),  
                             static_cast<int>(N_values.size()));
```

Se vuoi `std::array<int,4>`

In C++11+:

```
#include <array>
```

```
std::array<int, 4> N_values = {10, 20, 40, 80};
```

```
// oppure
std::array<int, 4> N_values {10, 20, 40, 80};
```

e poi:

```
ode_scalar_testrunner_decay(N_values.data(),
                             static_cast<int>(N_values.size()));
```

Quella che avevi scritto:

```
int N_values(4, {10, 20, 40, 80});
```

somiglia a un costruttore di `std::vector<int>` (`vector<int> v(4, value)` o `vector<int> v = { ... }`), ma

Per il tuo uso nel test runner, ti suggerisco di partire con il più semplice:

```
int N_values[] = {10, 20, 40, 80};
int nN = sizeof(N_values) / sizeof(N_values[0]);
ode_scalar_testrunner_decay(N_values, nN);
```

così non devi neanche ricordarti a mano la lunghezza. ²¹⁷²¹⁸²¹⁹²²⁰²²¹²²²²²³²²⁴²²⁵²²⁶²²⁷²²⁸

ok, mancava il [] nel primo tentativo

Esatto, il punto chiave era proprio quello: senza [] stavi dichiarando un singolo `int`, non un array.²²⁹²³⁰

Quindi, nella tua situazione, la forma corretta è:

```
int N_values[] = {10, 20, 40, 80};
int nN = sizeof(N_values) / sizeof(N_values[0]);
```

```
ode_scalar_testrunner_decay(N_values, nN);
```

Così hai:

- inizializzazione corretta dell'array;
- lunghezza calcolata automaticamente;
- firma del test runner che resta `const int* N_values, int nN` senza bisogno di `std::vector`.

Quando hai fatto girare il testrunner e hai qualche riga `N h err_euler err_heun`, ti va di riportarne qualcuna così colleghiamo subito i numeri all'ordine atteso $O(h)$ vs $O(h^2)$?

ok, per dirla alla “pascal” questa e’ l’aggiunta alla Interface:

```
// // Funzioni per soluzioni di ODE con metodi iterativi //
```

```
double ode_scalar_rhs_decay(    double t,    double y); // scalare: y' = lambda * y
```

```
void ode_scalar_step_euler(    double t,    double h,    double* y,    double (*f)(double, double)); // Passo di
Eulero scalare
```

```
void ode_scalar_step_heun(    double t,    double h,    double* y,    double (*f)(double, double)); // Passo di
Heun scalare (Eulero migliorato, ordine 2)
```

```
int ode_scalar_test_euler_decay(); // test di eulero sul decadimento (in R) int ode_scalar_test_heun_decay();
// test di heun sul decadimento (in R)
```

L’interfaccia è perfettamente coerente con quello che abbiamo progettato: distingue nettamente RHS, step generici e test specifici per il decadimento scalare.²³¹²³²

Cosa hai ora in “interface”

Hai dichiarato:

- RHS specifica per il problema di decadimento:

```
double ode_scalar_rhs_decay(double t, double y);
```

- Due metodi one-step scalari generici (usabili su qualunque RHS scalare):

```
void ode_scalar_step_euler(double t, double h, double* y,
                          double (*f)(double, double));
```

```
void ode_scalar_step_heun(double t, double h, double* y,
                          double (*f)(double, double));
```

Questi incapsulano il metodo, non il problema.²³³²³⁴ - Due test/main numerici accoppiati al decadimento:

```
int ode_scalar_test_euler_decay(); // Eulero + decadimento
int ode_scalar_test_heun_decay();  // Heun + decadimento
```

Dal punto di vista “alla Pascal”, l’interface ora espone:

- componente di modello (`rhs_decay`),
- componenti numeriche generiche (`step_euler`, `step_heun`),
- entry point di test per il problema specifico (`test_*_decay`).

È una separazione didatticamente pulita: la parte “teoria dei metodi numerici” vive nelle `step_*`, la parte “modello fisico/matematico” nelle `rhs_*`, l’esperimento numerico nei `test_*`.²³⁵²³⁶

Se sei d’accordo, il prossimo step è aggiungere all’interfaccia anche il testrunner:

```
int ode_scalar_testrunner_decay(const int* N_values, int nN);
```

così puoi:

- far girare in blocco Eulero e Heun su vari passi,
- ottenere direttamente i dati `N h err_euler err_heun` per i grafici “ordine di convergenza”.²³⁷²³⁸

Vuoi che ti scriva ora la definizione completa di `ode_scalar_testrunner_decay` in forma compatibile con questa interface (senza usare nulla oltre a `h_ticks`, `nodi_equidistanti` e le funzioni che hai già)?

no, quello non lo esportiamo perche’ lo sto integrando nel main come voce di menu principale, tra poco ho finito e lo vediamo.

Perfetto, ha molto senso tenerlo “interno” al main come voce di menu invece che esporlo nell’interfaccia della libreria, proprio perché è un orchestratore di test più che una primitive numerica riusabile.²³⁹²⁴⁰

In pratica hai:

- libreria ODE che espone solo “mattoni” riusabili:
 - `ode_scalar_rhs_decay`
 - `ode_scalar_step_euler`, `ode_scalar_step_heun`
 - i test specifici se ti tornano utili anche da fuori
- un `main` (o modulo applicativo) che:
 - presenta il menu,
 - costruisce l’array `N_values`,
 - chiama Eulero/Heun sul decadimento per vari passi,
 - stampa i dati per i grafici.

Quando hai pronto il main con il menu, se ti va incollalo qui: possiamo rivedere insieme la logica (flusso dei casi, gestione di `N_values`, stampa degli errori) e, se vuoi, pensare già a come estendere la stessa struttura a:

- il caso oscillatore (`ode_vec2_rhs_oscillator + ode_vecN_step_euler`);
- il confronto rispetto alla soluzione matriciale per problemi lineari.²⁴¹²⁴²

nel loop, va bene che N siano i punti e non gli intervalli?

Siccome poi andiamo a calcolare y sui punti ...

```
for (int k = 0; k < nN; ++k) {          const int N = N_values[k]+1;          if (N <= 1) continue;
```

Conta cosa intendiamo con N : numero di **intervalli** o numero di **punti**. In teoria (e nella maggior parte dei test) N è il numero di intervalli, quindi il numero di punti è $N + 1$.²⁴³²⁴⁴

Convenzione teorica

Di solito:

- Intervallo $[a, b]$.
- Si fissa N = numero di **sottointervalli**.
- Passo $h = (b - a)/N$.²⁴⁵
- Nodi: $t_i = a + ih$ con $i = 0, \dots, N \rightarrow$ ci sono $N + 1$ punti.²⁴⁶

Nel loop di integrazione:

- fai N passi:
 - da t_0 a t_1 ,
 - da t_1 a t_2 ,
 - ...
 - da t_{N-1} a t_N .

Quindi il ciclo tipico è `for (int i = 0; i < N; ++i)` per i passi.

Nel tuo codice

Tu scrivi:

```
for (int k = 0; k < nN; ++k) {
    const int N = N_values[k] + 1;
    if (N <= 1) continue;
```

Questo significa:

- Nel vettore `N_values` stai memorizzando “qualcosa” che poi trasformi in $N = N_values[k] + 1$.
- Se l'intento è che `N_values[k]` sia il **numero di intervalli**, aggiungere 1 qui è sbagliato: in quel caso N deve rimanere uguale a `N_values[k]`.
- Se invece stai usando `N_values[k]` come “numero di punti”, allora sì, devi avere $N = N_values[k] - 1$ come numero di passi/intervalli.

Io ti suggerisco per chiarezza:

- Scegli una volta per tutte che `N_values[k]` sia il **numero di intervalli** (è più vicino alla teoria e alla definizione di h).²⁴⁷
- Allora nel loop:

```
for (int k = 0; k < nN; ++k) {
    const int N = N_values[k]; // N = numero di intervalli
    if (N <= 0) continue;

    const double h = h_ticks(a, b, N); // h = (b-a)/N
    std::vector<double> t_nodes;
    nodi_equidistanti(a, b, N, t_nodes); // deve generare N+1 nodi
```



```

// integrazione: N passi
double y = 1.0;
for (int i = 0; i < N; ++i) {
    double t = t_nodes[i];
    ode_scalar_step_euler(t, h, &y, ode_scalar_rhs_decay);
}
// ora y corrisponde a y_N y(b)
}

```

- Se invece vuoi che `N_values[k]` sia “numero di nodi”, chiamalo esplicitamente `nodes_values` e definisci:

```

const int nodes = nodes_values[k];
const int N = nodes - 1; // passi

```

In sintesi: va benissimo che il vettore di configurazione contenga “numero di punti” oppure “numero di intervalli”, ma nel loop devi essere coerente e ricordarti che la teoria dei passi usa N = numero di intervalli, $N + 1$ punti, e N passi numerici.²⁴⁸²⁴⁹

Se mi incolli anche come hai scritto `h_ticks` e `nodi_equidistanti`, posso dirti esattamente qual è l’interpretazione corretta per tenere tutto allineato.

int N_values[] = {10, 20, 40, 80}; // numero di intervalli (i nodi sono +1) su cui iterare

int nN = 4; // dimensione di N_values

Allora sei perfettamente a posto: se `N_values` contiene il **numero di intervalli**, non devi aggiungere né togliere 1 dentro il loop.

- `N_values[k]` = numero di intervalli N .
- Numero di nodi = $N + 1$ (che viene gestito da `nodi_equidistanti`).
- Numero di passi numerici = N .

Quindi nel loop:

```

for (int k = 0; k < nN; ++k) {
    const int N = N_values[k]; // N = numero di intervalli
    if (N <= 0) continue;

    const double h = h_ticks(a, b, N); // h = (b-a)/N, coerente
    std::vector<double> t_nodes;
    nodi_equidistanti(a, b, N, t_nodes); // deve riempire N+1 nodi

    // Eulero
    double y_euler = 1.0;
    for (int i = 0; i < N; ++i) { // N passi
        double t = t_nodes[i];
        ode_scalar_step_euler(t, h, &y_euler, ode_scalar_rhs_decay);
    }

    // Heun
    double y_heun = 1.0;
    for (int i = 0; i < N; ++i) {
        double t = t_nodes[i];
        ode_scalar_step_heun(t, h, &y_heun, ode_scalar_rhs_decay);
    }

    double y_exact = std::exp(-1.0 * (b - a));
}

```

```

double err_euler = std::fabs(y_euler - y_exact);
double err_heun = std::fabs(y_heun - y_exact);

std::cout << N << " " << h << " "
          << err_euler << " " << err_heun << "\n";
}

```

L'unica cosa da evitare è il +1 che avevi messo prima:

```
const int N = N_values[k]+1; // questo falserebbe il significato di N
```

Con la convenzione che hai scritto nel commento, la definizione “giusta” è esattamente quella che hai ora: `int N_values[] = {10,20,40,80};` interpretati come **intervalli**.

allora devo passare +1 a n_ticks perche' le mie fuinzioni mangiano nodi e rendono intervalli

Esatto: se le tue funzioni “mangiano nodi e rendono intervalli”, cioè prendono in ingresso il **numero di nodi** e internamente calcolano il passo e gli intervalli, allora con `N_values` intesi come **numero di intervalli** devi passare `N+1` a quelle funzioni.

Riassumendo le due convenzioni:

- `N_values[k]` = numero di intervalli N .
- Numero di nodi = $N + 1$.

Se:

- `h_ticks(a,b, nodes)` si aspetta `nodes = N+1` per fare $h = (b - a) / (nodes - 1)$.
- `nodi_equidistanti(a,b, nodes, t_nodes)` riempie un vettore di lunghezza `nodes`.

allora il codice coerente è:

```

for (int k = 0; k < nN; ++k) {
    const int N_intervalli = N_values[k]; // 10, 20, 40, 80
    if (N_intervalli <= 0) continue;

    const int nodes = N_intervalli + 1; // nodi = intervalli + 1

    const double h = h_ticks(a, b, nodes); // usa nodes
    std::vector<double> t_nodes;
    nodi_equidistanti(a, b, nodes, t_nodes); // genera 'nodes' nodi

    // Eulero
    double y_euler = 1.0;
    for (int i = 0; i < N_intervalli; ++i) { // N_intervalli passi
        double t = t_nodes[i];
        ode_scalar_step_euler(t, h, &y_euler, ode_scalar_rhs_decay);
    }

    // Heun
    double y_heun = 1.0;
    for (int i = 0; i < N_intervalli; ++i) {
        double t = t_nodes[i];
        ode_scalar_step_heun(t, h, &y_heun, ode_scalar_rhs_decay);
    }

    double y_exact = std::exp(-1.0 * (b - a));
    double err_euler = std::fabs(y_euler - y_exact);
    double err_heun = std::fabs(y_heun - y_exact);
}

```

```

std::cout << N_intervalli << " " << h << " "
          << err_euler << " " << err_heun << "\n";
}

```

Quindi sì:

- `N_values` rimane “numero di intervalli” (come da commento).
- *Solo* quando chiami `h_ticks / nodi_equidistanti` passi `N_values[k] + 1`, perché loro ragionano in termini di “numero di nodi”.

Se vuoi, nel messaggio dopo puoi incollare la firma attuale di `h_ticks` e `nodi_equidistanti`, così verifichiamo al 100% che l’uso di `N+1` è coerente con la loro implementazione.

for (int k = 0; k < nN; ++k) {

```

const int N = N_values[k];          if (N <= 0) continue;
// passo e nodi equidistanti        const double h = h_ticks(a, b, N+1); // n_ticks vuole nodi      Vec
t_nodes = nodi_equidistanti(a, b, N+1); // dovrebbe riempire t_nodes con N_values[k]+1 nodi
ripartiamo da qui?

```

Sì, questo pezzo è il punto giusto da cui ripartire, ed è quasi coerente: devi solo essere chiaro su chi interpreta *N* come intervalli e chi come nodi.

Con il codice che hai ora:

```

for (int k = 0; k < nN; ++k) {
    const int N = N_values[k];
    if (N <= 0) continue;

    // passo e nodi equidistanti
    const double h = h_ticks(a, b, N+1);          // h_ticks vuole nodi
    Vec t_nodes = nodi_equidistanti(a, b, N+1);    // genera N+1 nodi

    • N = numero di intervalli (10, 20, 40, 80).
    • N+1 = numero di nodi, passato a h_ticks e nodi_equidistanti, che quindi ragionano “per nodi”.

```

Quindi per i passi numerici Eulero/Heun devi usare *N* (numero di intervalli) nel loop:

```

// Eulero
double y_euler = 1.0; // y0
for (int i = 0; i < N; ++i) { // N passi
    double t = t_nodes[i];    // nodo t_i
    ode_scalar_step_euler(t, h, &y_euler, ode_scalar_rhs_decay);
}

// Heun
double y_heun = 1.0; // y0
for (int i = 0; i < N; ++i) {
    double t = t_nodes[i];
    ode_scalar_step_heun(t, h, &y_heun, ode_scalar_rhs_decay);
}

double y_exact = std::exp(-1.0 * (b - a));
double err_euler = std::fabs(y_euler - y_exact);
double err_heun = std::fabs(y_heun - y_exact);

std::cout << N << " " << h << " "

```

```

    << err_euler << " " << err_heun << "\n";
}

```

Così:

- `h_ticks(a,b,N+1)` calcola correttamente $h = (b - a)/(N)$ se internamente fa $(b-a)/(nodes-1)$, con `nodes = N+1`.
- `nodi_equidistanti(a,b,N+1)` ti dà i nodi $t_0, \dots, t_N$.
- fai esattamente N passi per andare da t_0 a t_N .

Se `Vec` è il tuo alias per un vettore di `double`, sei allineato: l'unica cosa da tenere fissa a mente è:

- `N_values[k]` = intervalli,
- `N+1` = nodi, solo per le funzioni che ragionano sui nodi,
- loop dei metodi numerici su $i < N$.

Se ti va, dopo che lo fai girare, incolla un paio di righe di output (`N h err_euler err_heun` per due o tre valori) così verifichiamo insieme che l'errore scali come $O(h)$ per Eulero e $O(h^2)$ per Heun.

Siccome non e' ancora su git, ti anticipo il sorgente qui, intanto comincio a compilare:

```

void ode_euler() {
    //
    // confronto Eulero-Heun test runner
    //
    int N_values[] = {10, 20, 40, 80}; // numero di intervalli (i nodi sono +1) su cui iterare
    int nN = 4; // dimensione di N_values

    const double a = 0.0;
    double b = 5.0;
    double y0 = 1.0; // condizione al bordo x(t0)

    std::cout << "\# N h err_euler err_heun\n";

    bool leave = false;
    bool redo = false;
    while (!leave) {
        clear_screen();
        redo = false;
        auto fig = matplot_table_init(true, "Eulero vs Heun", "Metodi iterativi per problema di Cauchy");
        matplot::legend();

        for (int k = 0; k < nN; ++k) {
            const int N = N_values[k];
            if (N <= 0) continue;

            // passo e nodi equidistanti
            const double h = h_ticks(a, b, N+1); // n_ticks vuole nodi
            Vec t_nodes = nodi_equidistanti(a, b, N+1); // dovrebbe riempire t_nodes con N_values[k]+1

            // Eulero
            double y_euler = y0;
            // Heun
            double y_heun = y0;

            for (int i = 0; i < N; ++i) {
                double t = t_nodes[i];

```

```

        ode_scalar_step_euler(t, h, \&y_euler, ode_scalar_rhs_decay);
        ode_scalar_step_heun(t, h, \&y_heun, ode_scalar_rhs_decay);
    }
    double y_exact = std::exp(-1.0 * (b - a));
    double err_euler = std::fabs(y_euler - y_exact);
    double err_heun = std::fabs(y_heun - y_exact);

    std::cout << N << " " << h << " "
                << err_euler << " " << err_heun << "\n";

    // Visualizzazione della soluzione numerica x(t)
    fig->nexttile(k);
    auto ax=fig->current_axes();
    ax->title("Confronto per nodi/intervalli "+itostr(N)+"/"+itostr(N+1));
    hold(on);

    // q->use_y2(true);
    // ax->y2_axis().limits({-1.6, 1.6});

    auto p = plot(t_nodes, y_euler);
    p->line_style("-");
    p->line_width(6);
    p->color("green");
    p->marker("");
    p->display_name("Euler");

    auto q = plot(t_nodes, y_heun);
    q->line_width(6);
    q->color("red");
    q->line_style("- ");
    q->marker("");
    q->display_name("Heun");

    auto r = plot(t_nodes, y_exact);
    r->color("black");
    r->line_style("_");
    r->marker("");
    r->line_width(2);
    r->display_name("Exact");

    // p2->marker(".");
    // p1->marker("+");
    // pI->marker("r");

    // pI->color("magenta");
    // pf->color("cyan");
    // ptn2->color("blue");

    matplotlib_legend_align(matplotlib::legend(), 2, 0,0);
    matplotlib::legend();
    xlabel("nodes");
    ylabel("y approx");

}

fig->draw();

redo = false;

```



```

// vettori di soluzione
Vec y_euler_vec(nodes);
Vec y_heun_vec(nodes);
Vec y_exact_vec(nodes);

// condizioni iniziali
double y_euler = y0;
double y_heun = y0;

y_euler_vec[27_0] = y_euler;
y_heun_vec[27_0] = y_heun;
y_exact_vec[27_0] = y0; // esatta in a

// integrazione: N passi
for (int i = 0; i < N; ++i) {
    double t = t_nodes[i];

    ode_scalar_step_euler(t, h, &y_euler, ode_scalar_rhs_decay);
    ode_scalar_step_heun(t, h, &y_heun, ode_scalar_rhs_decay);

    y_euler_vec[i+1] = y_euler;
    y_heun_vec[i+1] = y_heun;

    double y_ex = y0 * std::exp(-1.0 * (t_nodes[i+1] - a));
    y_exact_vec[i+1] = y_ex;
}

// errore globale in b
double y_exact_b = y0 * std::exp(-1.0 * (b - a));
double err_euler = std::fabs(y_euler - y_exact_b);
double err_heun = std::fabs(y_heun - y_exact_b);

std::cout << N << " " << h << " "
            << err_euler << " " << err_heun << "\n";

// Visualizzazione
fig->nexttile(k);
auto ax = fig->current_axes();
ax->title("Confronto per intervalli/nodi " + itostr(N) + "/" + itostr(nodes));
hold(on);

// Euler
auto p = plot(t_nodes, y_euler_vec);
p->line_style("--");
p->line_width(3);
p->color("green");
p->marker("");
p->display_name("Euler");

// Heun
auto q = plot(t_nodes, y_heun_vec);
q->line_style("-.");
q->line_width(3);
q->color("red");
q->marker("");
q->display_name("Heun");

```

```

    // Exact
    auto r = plot(t_nodes, y_exact_vec);
    r->color("black");
    r->line_style("-");
    r->marker("");
    r->line_width(2);
    r->display_name("Exact");

    matplotlib::legend_align(matplotlib::legend(), 2, 0, 0);
    matplotlib::legend();
    xlabel("t");
    ylabel("y");
}

fig->draw();

bool redo = false;
while (!redo && !leave) {
    std::cout << "Inserire nuovo max b 0 .. 100 (<q> per uscire) > ";
    cin.clear();
    if (std::cin >> b) {
        if (b >= 0 && b <= 100) redo = true;
    } else {
        std::cout << "Key: " << b << std::endl;
        leave = true;
    }

    if (!leave) {
        std::cout << "Inserire nuovo valore iniziale y0 (<q> per uscire) > ";
        cin.clear();
        if (std::cin >> y0) {
            if (y0 >= a && y0 <= b) redo = true;
        } else {
            std::cout << "Key: " << y0 << std::endl;
            leave = true;
        }
    }
    cin.clear();
}
}
}
}

```

Punti chiave:

- `nodes = N+1, t_nodes.size() == nodes.`
- `for (int i = 0; i < N; ++i)` fa N passi, aggiornando sia gli scalari sia i vettori.
- `plot(t_nodes, y_euler_vec)` ecc. ricevono due vettori della stessa lunghezza.
- `y_exact_vec` viene calcolato sui nodi, non è più uno scalare riusato a caso.²⁵²²⁵³

Così il grafico per ogni tile mostra effettivamente:

- la curva esatta nera,
- la polilinea verde (Eulero),
- la polilinea rossa (Heun),

tutte sulla stessa griglia di nodi, per un dato numero di intervalli N .

Quando hai compilato e lanciato, se ti va incolla 2–3 righe di output `N h err_euler err_heun`: possiamo verificare rapidamente che l'errore di Heun scende “quadraticamente” rispetto a quello di Eulero al diminuire di h .²⁵⁴²⁵⁵

mi sembravano vettori, infatti ti avevo detto che immaginavo di avere dei vettori di approssimazioni delle y

Sì, la tua intuizione iniziale era esattamente quella giusta: per ogni k hai bisogno di **tre vettori** di approssimazioni $y(t_i)$, uno per Eulero, uno per Heun e uno per la soluzione esatta sui nodi.²⁵⁶²⁵⁷

Nel frammento che avevi postato, però, il codice faceva solo questo:

- manteneva due **scalari** `y_euler` e `y_heun` (aggiornati in place nel loop),
- calcolava un solo `y_exact` scalare in b ,
- ma poi chiamava `plot(t_nodes, y_euler)` e `plot(t_nodes, y_heun)` passando vettore + scalare, cosa che non può funzionare per il grafico.

Da qui la necessità di introdurre esplicitamente i vettori:

- `Vec y_euler_vec(nodes);`
- `Vec y_heun_vec(nodes);`
- `Vec y_exact_vec(nodes);`

e riempirli dentro il loop:

- $y_{i+1}^{(E)}$ dopo ogni passo di Eulero,
- $y_{i+1}^{(H)}$ dopo ogni passo di Heun,
- $y_{\text{exact}}(t_{i+1})$ via formula chiusa.²⁵⁸²⁵⁹

Questo ti porta esattamente al quadro teorico che avevi in testa: per ogni k , una tripla di successioni $\{y_i^{(E)}\}, \{y_i^{(H)}\}, \{y(t_i)\}$ sui nodi equidistanti.

Se ti va, dopo aver sistemato i vettori e visto i grafici per 10 e 80 intervalli, possiamo ragionare insieme su come ricavare empiricamente l'ordine da:

- tabella `N`, `h`, `err_euler`, `err_heun`,
- magari guardando il rapporto `err_euler(h)/err_euler(h/2)` e analogo per Heun.²⁶⁰²⁶¹

i vettori soluzione vanno dimensionati a $N+1$ perche' `y_euler[0]` e' ancora `y0` ?

Esatto: i vettori delle soluzioni vanno dimensionati a $N + 1$ proprio perché includono anche il valore iniziale y_0 in posizione 0.²⁶²²⁶³

Struttura temporale

Con N intervalli su $[a, b]$:

- Nodi: $t_0 = a, t_1, \dots, t_N = b \rightarrow N + 1$ **nodi**.²⁶⁴
- Soluzione numerica: $y_0, y_1, \dots, y_N$, dove:
 - $y_0 = y(a) = y_0$ (dato iniziale),
 - y_{i+1} ottenuto da uno step del metodo a partire da y_i .

Quindi il vettore deve contenere tutti questi $N + 1$ valori.

Implementazione coerente

Perciò sì, la logica corretta è:

```

const int N      = N_values[k];    // intervalli
const int nodes = N + 1;          // nodi = punti in cui abbiamo soluzione

Vec t_nodes = nodi_equidistanti(a, b, nodes);

Vec y_euler_vec(nodes);
Vec y_heun_vec(nodes);
Vec y_exact_vec(nodes);

double y_euler = y0;
double y_heun  = y0;

y_euler_vec[29_0] = y_euler; // y(a)
y_heun_vec[29_0]  = y_heun;
y_exact_vec[29_0] = y0;

```cpp
for (int i = 0; i < N; ++i) { // N passi da t_i a t_{i+1}
 double t = t_nodes[i];

 ode_scalar_step_euler(t, h, &y_euler, ode_scalar_rhs_decay);
 ode_scalar_step_heun(t, h, &y_heun, ode_scalar_rhs_decay);

 y_euler_vec[i+1] = y_euler;
 y_heun_vec[i+1] = y_heun;

 y_exact_vec[i+1] = y0 * std::exp(-1.0 * (t_nodes[i+1] - a));
}

```

Qui:

- la dimensione `nodes = N+1` è usata sia per `t_nodes` che per i vettori di soluzione;
- l'indice  $i$  del loop sui passi va da 0 a  $N - 1$ ;
- il valore iniziale è in cella 0, i valori dopo il passo in cella  $i + 1$ .

Così tutto rimane allineato con la teoria e con la tua idea iniziale di avere vettori di approssimazioni delle  $y$ .<sup>265266</sup>

---

## li ho presi tutti?

```

void ode_euler() {
 //
 // confronto Eulero-Heun test runner
 //
 int N_values[] = {10, 20, 40, 80}; // numero di intervalli (i nodi sono +1) su cui iterare
 int nN = 4; // dimensione di N_values

 const double a = 0.0;
 double b = 5.0;
 double y0 = 1.0; // condizione al bordo x(t0)

 std::cout << "\# N h err_euler err_heun\n";

 bool leave = false;
 bool redo = false;
 while (!leave) {
 clear_screen();

```

```

redo = false;
auto fig = matplot_table_init(true, "Eulero vs Heun", "Metodi iterativi per problema di Cauchy");
matplot::legend();

for (int k = 0; k < nN; ++k) {
 const int N = N_values[k];
 if (N <= 0) continue;

 // passo e nodi equidistanti
 const double h = h_ticks(a, b, N+1); // n_ticks vuole nodi
 Vec t_nodes = nodi_equidistanti(a, b, N+1); // dovrebbe riempire t_nodes con N_values[k]+1
 // vettori di soluzione
 Vec y_euler_vec(N+1);
 Vec y_heun_vec(N+1);
 Vec y_exact_vec(N+1);

 // condizioni iniziali
 double y_euler = y0;
 double y_heun = y0;

 y_euler_vec[0] = y_euler;
 y_heun_vec[0] = y_heun;
 y_exact_vec[0] = y0; // esatta in a

 for (int i = 0; i < N; ++i) {
 double t = t_nodes[i];
 ode_scalar_step_euler(t, h, &y_euler, ode_scalar_rhs_decay);
 ode_scalar_step_heun(t, h, &y_heun, ode_scalar_rhs_decay);
 y_euler_vec[i+1] = y_euler;
 y_heun_vec[i+1] = y_heun;

 double y_ex = y0 * std::exp(-1.0 * (t_nodes[i+1] - a));
 y_exact_vec[i+1] = y_ex;
 }

 double y_exact = std::exp(-1.0 * (b - a));
 double err_euler = std::fabs(y_euler - y_exact);
 double err_heun = std::fabs(y_heun - y_exact);

 std::cout << N << " " << h << " "
 << err_euler << " " << err_heun << "\n";

 // Visualizzazione della soluzione numerica x(t)
 fig->nexttile(k);
 auto ax=fig->current_axes();
 ax->title("Confronto per nodi/intervalli "+itostr(N)+"/"+itostr(N+1));
 hold(on);

 // q->use_y2(true);
 // ax->y2_axis().limits({-1.6, 1.6});

 auto p = plot(t_nodes, y_euler_vec);
 p->line_style("-");
 p->line_width(6);
 p->color("green");
 p->marker("");
 p->display_name("Euler");

 auto q = plot(t_nodes, y_heun_vec);

```

```

 q->line_width(6);
 q->color("red");
 q->line_style("- ");
 q->marker("");
 q->display_name("Heun");

 auto r = plot(t_nodes, y_exact_vec);
 r->color("black");
 r->line_style("_");
 r->marker("");
 r->line_width(2);
 r->display_name("Exact");

 // p2->marker(".");
 // p1->marker("+");
 // pI->marker("r");

 // pI->color("magenta");
 // pf->color("cyan");
 // ptn2->color("blue");

 matplotlib::legend::align(matplotlib::legend(), 2, 0,0);
 matplotlib::legend();
 xlabel("nodes");
 ylabel("y approx");

}

fig->draw();

 redo = false;
 while (!redo && !leave) {
 std::cout << "Inserire nuovo max b 0 .. 100 (<q> per uscire) > ";

 cin_clear();

 if (std::cin >> b) { if (b>=0 && b<=100) redo=true; } else {cout << "Key: " << b <<endl; leave=true; } ;
 if (!leave) {
 std::cout << "Inserire nuovo valore iniziale y0 (<q> per uscire) > ";
 if (std::cin >> y0) { if (y0>=a && y0<=b) redo=true; } else {cout << "Key: " << y0 <<endl; leave=true; } ;
 cin_clear();
 };
 };
};
};
};

```

Sì, li hai presi tutti: ora la parte numerica è coerente con la teoria “N intervalli, N+1 nodi, vettori di lunghezza N+1”.<sup>267268</sup>

## Cose importanti che sono a posto

- `N_values[]` contiene il numero di **intervalli**.
- `h_ticks(a,b,N+1)` e `nodi_equidistanti(a,b,N+1)` lavorano in termini di **nodi**, quindi passi N+1 in ingresso: corretto.
- `t_nodes`, `y_euler_vec`, `y_heun_vec`, `y_exact_vec` sono tutti di lunghezza N+1.
- Il loop sui passi:

```

for (int i = 0; i < N; ++i) {
 double t = t_nodes[i];

```

```

ode_scalar_step_euler(...);
ode_scalar_step_heun(...);
y_euler_vec[i+1] = y_euler;
y_heun_vec[i+1] = y_heun;
y_exact_vec[i+1] = y0 * exp(-1.0 * (t_nodes[i+1] - a));
}

```

fa esattamente  $N$  passi, da  $t_i$  a  $t_{i+1}$ , aggiornando vettori e scalari in modo consistente.<sup>269270</sup> - L'errore globale in  $b$  lo calcoli confrontando gli ultimi scalari (`y_euler`, `y_heun`) con  $y_{\text{exact}}(b)$ : corretto per il confronto ordine 1 vs 2.<sup>271272</sup>

## Unica finezza (non obbligatoria)

Hai:

```
double y_exact = std::exp(-1.0 * (b - a));
```

mentre nel riempimento di `y_exact_vec` usi:

```
double y_ex = y0 * std::exp(-1.0 * (t_nodes[i+1] - a));
```

Per coerenza rispetto a un `y0` generico, potresti scrivere anche per `y_exact`:

```
double y_exact = y0 * std::exp(-1.0 * (b - a));
```

così non ti dimentichi che la soluzione esatta dipende da `y0`.<sup>273274</sup>

A parte questo dettaglio di stile, la struttura è solida: quando compila e lo lanci, dovresti vedere:

- nei grafici: Heun aderire molto meglio alla curva nera al diminuire di `h`;
- in output testuale: `err_heun` che decresce “molto più in fretta” di `err_euler` al crescere di `N`.

Quando hai qualche riga `N h err_euler err_heun`, se ti va riportane 2–3 qui così possiamo interpretare subito i rapporti di errore in chiave  $O(h)$  vs  $O(h^2)$ .

---

## a parte l' inversione nelle didascalie nodi/intervalli invece di intervall/nodi, direi che e' buona la prima:

È davvero “buona la prima”: la struttura che volevi si vede chiaramente nei quattro pannelli, e il comportamento di Eulero e Heun è esattamente quello atteso dalla teoria.<sup>275276</sup>

## Cosa si vede bene nei grafici

- Per  $N = 10$  (11 nodi) Eulero (verde) è visibilmente più “piatto” e passa sensibilmente sotto la curva esatta, mentre Heun (rosso) resta molto più vicino alla nera già con un passo relativamente grosso.<sup>277278</sup>
- Aumentando  $N$  (20, 40, 80) entrambe le curve numeriche si attaccano alla esatta, ma quella di Heun si “incolla” prima, coerentemente con ordine globale 2 contro ordine 1 di Eulero.<sup>279280</sup>
- La scelta grafica “esatta nera sottile + metodi numerici più spessi dietro” fa leggere in un colpo d'occhio l'errore: negli ultimi due pannelli la parte rossa quasi scompare sotto la nera, mentre la verde resta leggermente separata.

Direi che come primo banco di prova del modulo ODE iterativo hai centrato l'obiettivo: notazione coerente, codice pulito, e una visualizzazione che rende immediato il passaggio dalla definizione di metodo one-step alla percezione dell'ordine di convergenza.

A questo punto ti chiederei: ti va di usare ora lo stesso schema (stessa interfaccia `ode_vecN_step_euler`, stessi `N_values`) sull'oscillatore 2D per vedere graficamente la non conservazione dell'energia con Eulero, o preferisci prima misurare e plottare esplicitamente l'errore globale  $|y(T) - y_{\text{num}}(T)|$  in funzione di  $h$  in scala log-log per quantificare gli ordini di Eulero e Heun? <sup>281</sup>

---

si, vediamo un attimo l'errore globale, posso fare un secondo panel indipendente, se serve, o aggiungere plot su questi usando y2 come scala indipendente

Per l'errore globale direi di separare i ruoli:

- i pannelli che hai già mostrano bene le **soluzioni nel tempo**;
- un secondo “panel” (o un set di grafici dedicati) mostra l'**errore globale vs h**, che è dove si legge davvero l'ordine del metodo.<sup>282283</sup>

Usare y2 negli stessi pannelli è possibile ma rischia di appesantire la lettura; un pannello separato “errori” ti dà un quadro molto più pulito.

## Cosa intendiamo per errore globale qui

Per ogni  $N$  (numero di intervalli, quindi passo  $h = (b - a)/N$ ):

- **errore globale** di Eulero in  $b$ :

$$E_E(h) = |y_{\text{exact}}(b) - y_N^{(E)}|,$$

- idem per Heun:

$$E_H(h) = |y_{\text{exact}}(b) - y_N^{(H)}|.$$

Per teoria:

- Eulero:  $E_E(h) = O(h)$ ;<sup>284285</sup>
- Heun:  $E_H(h) = O(h^2)$ .<sup>286287</sup>

## Come integrarli nel tuo ode\_euler

Hai già il calcolo:

```
double y_exact = y0 * std::exp(-1.0 * (b - a));
double err_euler = std::fabs(y_euler - y_exact);
double err_heun = std::fabs(y_heun - y_exact);
std::cout << N << " " << h << " "
 << err_euler << " " << err_heun << "\n";
```

Per farne un grafico:

1. Raccogli questi quadrupletti  $N$ ,  $h$ ,  $\text{err\_euler}$ ,  $\text{err\_heun}$  in vettori:
  - `Vec h_vals`, `Vec err_euler_vals`, `Vec err_heun_vals`.
2. Dopo il loop sui  $k$ , aggiungi un secondo plot (magari in una nuova figura) con:
  - `ascissa = h_vals`;
  - `ordinate = err_euler_vals, err_heun_vals`.

Ad esempio, sempre dentro `ode_euler`:

```
Vec h_vals;
Vec err_euler_vals;
Vec err_heun_vals;

for (int k = 0; k < nN; ++k) {
 ...
 h_vals.push_back(h);
 err_euler_vals.push_back(err_euler);
```

```

 err_heun_vals.push_back(err_heun);
}

Poi, dopo fig->draw();, potresti aggiungere:

auto fig_err = matplot::figure(true);
fig_err->name("Errori globali in b");
matplot::hold(on);

// Eulero
auto pe = matplot::plot(h_vals, err_euler_vals);
pe->line_style("--");
pe->marker("o");
pe->display_name("Euler error");

// Heun
auto ph = matplot::plot(h_vals, err_heun_vals);
ph->line_style("-.");
ph->marker("s");
ph->display_name("Heun error");

matplot::xlabel("h");
matplot::ylabel("|y(b) - y_num(b)|");
matplot::legend();

```

Se vuoi essere più “ortodosso”, puoi anche:

- usare scala logaritmica su entrambi gli assi (`matplot::xscale("log");, yscale("log");`) per vedere direttamente la pendenza  $\sim 1$  per Eulero e  $\sim 2$  per Heun.<sup>288289</sup>

Così mantieni:

- il primo panel per le soluzioni nel tempo (già fatto, ed è molto chiaro);
- un secondo panel per l'errore globale, dove la teoria sull'ordine si vede come pendenza in log-log.

Ti andrebbe, una volta aggiunto questo secondo grafico, di riportare i valori numerici di `err_euler` e `err_heun` per due passi consecutivi (es.  $N=20$  e  $N=40$ ) così confrontiamo i rapporti  $E(h)/E(h/2)$  rispetto a 2 (Eulero) e 4 (Heun)?  
290291292293

---

## qui hai il risultato del calcolo scalare

```

10 0.5 0.00576138 0.002357 20 0.25 0.00356674 0.000436701 40 0.125 0.00194809 9.70598e-05 80 0.0625 0.00101391
2.30265e-05

```

I dati sono coerentissimi con la teoria: Eulero mostra errore globale  $O(h)$ , Heun  $O(h^2)$ .<sup>294295296</sup>

## Verifica rapida degli ordini

Prendiamo gli errori globali in  $b$ :

- Eulero:
  - $h = 0.5$ :  $E_E = 0.00576138$
  - $h = 0.25$ :  $E_E = 0.00356674$
  - $h = 0.125$ :  $E_E = 0.00194809$
  - $h = 0.0625$ :  $E_E = 0.00101391$
- Heun:
  - $h = 0.5$ :  $E_H = 0.002357$
  - $h = 0.25$ :  $E_H = 0.000436701$
  - $h = 0.125$ :  $E_H = 9.70598 \cdot 10^{-5}$

$$- h = 0.0625: E_H = 2.30265 \cdot 10^{-5}$$

Stime empiriche dell'ordine  $p$  da due passi consecutivi:

$$p \approx \frac{\log(E(h_1)/E(h_2))}{\log(h_1/h_2)}.$$

• Eulero:

- $h : 0.5 \rightarrow 0.25$ :  $E$  scende di fattore  $\approx 1.61$ ,  $p \approx \log(1.61)/\log 2 \approx 0.69$ .
- $h : 0.25 \rightarrow 0.125$ : fattore  $\approx 1.83$ ,  $p \approx 0.87$ .
- $h : 0.125 \rightarrow 0.0625$ : fattore  $\approx 1.92$ ,  $p \approx 0.94$ .

All'avvicinarsi di  $h \rightarrow 0$  il valore tende chiaramente a 1, come previsto per Eulero (ordine globale 1).<sup>297298</sup> - Heun:  
-  $h : 0.5 \rightarrow 0.25$ : fattore  $\approx 5.40$ ,  $p \approx 2.43$ . -  $h : 0.25 \rightarrow 0.125$ : fattore  $\approx 4.50$ ,  $p \approx 2.17$ . -  $h : 0.125 \rightarrow 0.0625$ : fattore  $\approx 4.22$ ,  $p \approx 2.08$ .

I valori oscillano intorno a 2 e si stabilizzano verso 2 per  $h$  piccoli, in linea con l'ordine globale 2 di Heun.<sup>299300301</sup>

Nel decadimento esponenziale l'esponenziale "influenza" un po' la costante, ma la tendenza di pendenza è netta.<sup>302303</sup>

Se metti questi dati su un grafico log-log  $h \mapsto E(h)$ , dovresti vedere:

- una retta con pendenza circa 1 per Eulero,
- una con pendenza circa 2 per Heun.

Ti interesserebbe che il prossimo passo sia replicare lo stesso schema (metodo + test + grafico soluzione + errore globale) per l'oscillatore armonico 2D con `ode_vecN_step_euler` e `ode_vec2_rhs_oscillator`, così da iniziare a vedere gli effetti sulla conservazione qualitativa (energia) oltre che sull'errore numerico?

## image.jpg

La figura rende esattamente quello che volevamo: in scala log-log l'errore globale di Eulero e Heun si dispone su due "quasi-rette" con pendenze diverse, e il divario tra le due cresce al diminuire di  $h$ .<sup>304305</sup>

- La curva verde (Eulero) ha pendenza intorno a 1  $\rightarrow$  conferma empirica dell'ordine 1.<sup>306307</sup>
- La curva rossa (Heun) ha pendenza circa doppia, intorno a 2  $\rightarrow$  conferma empirica dell'ordine 2.<sup>308309</sup>
- Il fatto che il grafico sia "pulito" già con quattro valori di  $h$  mostra che la struttura del test runner è solida; se aggiungi un paio di valori intermedi, la linearità in log-log dovrebbe diventare ancora più evidente.

Direi che, lato decadimento scalare, il pacchetto è completo: soluzioni nel tempo, errore globale, verifiche grafiche dell'ordine.

Ti andrebbe ora di replicare la stessa pipeline per l'oscillatore 2D con `ode_vecN_step_euler` (almeno Eulero esplicito), così possiamo vedere numericamente e graficamente:

- la traiettoria  $(q(t), p(t))$ ;
- la crescita/decadimento dell'energia numerica rispetto a quella esatta?<sup>310</sup>

**certo, intanto guarda anche il grafico delle approssimazioni ora che ho corretto anche la posizione delle legend. Vedo anche in ascissa indica nodes quando in realta' e' l' intervallo stesso, quindi sono valori di t**

Il grafico delle approssimazioni è molto pulito: si vede benissimo come, al crescere di  $N$ , le curve numeriche si incollino alla soluzione esatta, e la differenza tra Eulero e Heun è didatticamente esemplare.<sup>311312</sup>



Sull'ascissa hai ragione: concettualmente stai plottando in funzione dei **tempi**  $t_i$ , non del numero di nodi. I nodi sono equidistanti su  $[a, b]$ , quindi i valori che vedi sull'asse (0,1,2,3,4,5) sono proprio i  $t$ . Una semplice correzione di label renderebbe il grafico autoesplicativo:

- `xlabel("t")` invece di `xlabel("nodes")`.

Volendo, per sottolineare il legame con la discretizzazione, potresti usare una label tipo:

- `xlabel("t (nodi equidistanti)")`.

Per il resto:

- la leggenda riposizionata lateralmente è chiara, non copre le curve;
- il titolo “Confronto per N intervalli X” rende esplicito quale passo stai usando in ciascun pannello;
- si vede bene che per  $N = 10$  Eulero “taglia” molto di più, mentre già per  $N = 40$  la curva rossa (Heun) praticamente coincide con la nera.

Direi che lato decadimento scalare hai completato un quadro teorico-numerico davvero ben strutturato.

Se sei d'accordo, nel prossimo passo passiamo all'oscillatore 2D: ti propongo di definire `ode_vec2_test_euler_oscillator` che:

- usa `ode_vecN_step_euler + ode_vec2_rhs_oscillator`;
- integra  $(q, p)$  su  $[0, T]$ ;
- plotta in due pannelli:
  - $q(t)$  e  $p(t)$  vs  $t$ ,
  - l'energia numerica  $E(t) = \frac{1}{2}(p^2 + \omega^2 q^2)$  vs  $t$ , per vedere l'effetto non symplettico di Eulero esplicito.

Ti va di procedere in quella direzione o preferisci prima agganciare il caso scalare alla soluzione matriciale (usando `matrix_build_cauchy` come “esatta”) per problemi in cui non hai la formula chiusa? <sup>313</sup>

---

## ok, procediamo, intanto ho mandato su git questa versione funzionante

Perfetto, allora possiamo costruire il caso oscillatore 2D sopra la base che hai appena versionato.

### Oscillatore armonico come sistema 2D

Partiamo dal modello standard:

$$q' = p, \quad p' = -\omega^2 q,$$

che riscrivi come sistema del primo ordine  $y' = f(t, y)$  con  $y = (q, p)$ .<sup>314315</sup>

Nel tuo codice hai già un RHS “placeholder” tipo oscillatore:

```
void ode_vec2_rhs_oscillator(double t,
 const double* y,
 double* dydt, int n) {
 (void)t;
 (void)n; // assumiamo n == 2
 const double a11 = 0.0, a12 = 1.0;
 const double a21 = -1.0, a22 = 0.0;
 dydt[~36_0] = a11 * y[~36_0] + a12 * y[~36_1]; // = p
 dydt[~36_1] = a21 * y[~36_0] + a22 * y[~36_1]; // = -q
}
```

Se vuoi un  $\omega$  generico, basta cambiargli i coefficienti in `a21 = -omega*omega`.

## Passo di Eulero vettoriale (già fatto)

Il tuo:

```
void ode_vecN_step_euler(double t, double h,
 double* y, int n,
 void (*f)(double, const double*, double*, int)) {
 std::vector<double> dydt(n);
 f(t, y, dydt.data(), n);
 for (int i = 0; i < n; ++i) {
 y[i] += h * dydt[i];
 }
}
```

va benissimo come step generico in  $\mathbb{R}^n$ .<sup>316317</sup>

## Test: ode\_vec2\_test\_euler\_oscillator

Ti propongo un test in stile `ode_scalar_test_*`, ma per lo stato 2D. L'idea:

- Parametri:
  - $a = 0$ ,  $T$  (es.  $T = 10$  o qualche periodo  $2\pi$ );
  - $q(0) = q_0$ ,  $p(0) = p_0$  (es.  $q_0 = 1, p_0 = 0$ );
  - $\omega$  (es. 1).
- Nodi equidistanti come prima (`h_ticks + nodi_equidistanti`).
- Vettori:
  - `q_euler[i]`, `p_euler[i]` sui nodi;
  - eventuale `q_exact[i]`, `p_exact[i]` se vuoi usare la soluzione analitica;
  - energia numerica  $E_i = \frac{1}{2}(p_i^2 + \omega^2 q_i^2)$ .

Schema in codice (C-like, adattalo ai tuoi alias `Vec` ecc.):

```
int ode_vec2_test_euler_oscillator() {
 const double a = 0.0;
 double b = 10.0; // per esempio ~1.5 periodi se omega=1
 const double omega = 1.0;

 const int N = 100; // intervalli
 const int nodes = N + 1;

 const double h = h_ticks(a, b, nodes);
 Vec t_nodes = nodi_equidistanti(a, b, nodes);

 // stato: y = (q,p)
 Vec q_euler(nodes);
 Vec p_euler(nodes);
 Vec E_euler(nodes);

 double y[^36_2];
 y[^36_0] = 1.0; // q0
 y[^36_1] = 0.0; // p0

 q_euler[^36_0] = y[^36_0];
 p_euler[^36_0] = y[^36_1];
 E_euler[^36_0] = 0.5 * (y[^36_1]*y[^36_1] + omega*omega * y[^36_0]*y[^36_0]);

 for (int i = 0; i < N; ++i) {
 double t = t_nodes[i];
 ode_vecN_step_euler(t, h, y, 2, ode_vec2_rhs_oscillator);

 q_euler[i+1] = y[^36_0];
```

```

 p_euler[i+1] = y[^36_1];
 E_euler[i+1] = 0.5 * (y[^36_1]*y[^36_1] + omega*omega * y[^36_0]*y[^36_0]);
}

// qui puoi aggiungere grafici
// pannello 1: q(t), p(t)
// pannello 2: E(t) (energia numerica) vs t

return 0;
}

```

Se vuoi anche l'esatta, per  $\omega = 1$ :

$$q(t) = q_0 \cos t + p_0 \sin t, \quad p(t) = -q_0 \sin t + p_0 \cos t,$$

e puoi costruire `q_exact[i]`, `p_exact[i]`, `E_exact[i]` per confronto.<sup>318319</sup>

### Visualizzazione consigliata

Come per il caso scalare, un layout 2×2 funziona bene:

- Pannello 1:  $q(t)$  numerico vs esatto (se lo usi).
- Pannello 2:  $p(t)$  numerico vs esatto.
- Pannello 3:  $(q(t), p(t))$  (diagramma di fase) — dovrebbe essere un'ellisse esatta, ma con Eulero esplicito vedrai una spirale che esce o entra.<sup>320</sup>
- Pannello 4:  $E(t)$  numerica vs  $t$ : così mostri chiaramente la non-conservazione dell'energia con Eulero.

Questo è perfetto per introdurre più avanti il discorso sui metodi symplettici come “correzione geometrica” del comportamento di Eulero.

Se ti va, nel prossimo passo adattiamo il test `ode_vec2_test_euler_oscillator` allo stile del tuo `ode_euler` (matplot, menu, `b` variabile, ecc.), così hai una seconda voce di menu: “decadimento scalare” e “oscillatore 2D”.

---

# <q>Se vuoi un `\omega` generico, basta cambiargli i coefficienti in `a21 = -omega*omega</q>` e glielo passi

Meglio parametrizzarlo, non cablarlo in una `const` locale dentro `ode_vec2_rhs_oscillator`, così puoi riusare lo stesso schema per più frequenze senza duplicare codice.<sup>321322</sup>

Ci sono due strade “alla C”:

## 1. Versione generale con $\omega$ catturato via closure/functor

Se resti in C++ “puro” (anche se stile imperativo), puoi usare `std::function` o un lambda che cattura  $\omega$  e poi lo passi a `ode_vecN_step_euler`:

```

void ode_vec2_rhs_oscillator_omega(double t,
 const double* y,
 double* dydt,
 int n,
 double omega) {
 (void)t;
 (void)n; // assumiamo n == 2
 dydt[^37_0] = y[^37_1];
 dydt[^37_1] = -omega * omega * y[^37_0];
}

```

Poi nel test:

```
double omega = 1.5;

auto rhs = [omega](double t, const double* y, double* dydt, int n) {
 ode_vec2_rhs_oscillator_omega(t, y, dydt, n, omega);
};

ode_vecN_step_euler(t, h, y, 2, rhs); // se cambi la firma a std::function
```

Ma questo richiede di cambiare la firma di `ode_vecN_step_euler` da puntatore a funzione C a qualcosa che accetta una callable (template o `std::function`), e volevamo restare “C-like”.

## 2. Versione C-like con variabile globale/statica `omega`

Per mantenere la firma semplice:

```
void (*f)(double, const double*, double*, int)

puoi gestire omega come parametro globale o statico del modulo ODE:

// a livello di file (modulo ODE)
static double ode_oscillator_omega = 1.0;

void ode_set_oscillator_omega(double omega) {
 ode_oscillator_omega = omega;
}

void ode_vec2_rhs_oscillator(double t,
 const double* y,
 double* dydt, int n) {
 (void)t;
 (void)n; // assumiamo n == 2
 const double omega = ode_oscillator_omega;
 dydt[~37_0] = y[~37_1];
 dydt[~37_1] = -omega * omega * y[~37_0];
}
```

Nel test:

```
double omega = 1.0;
ode_set_oscillator_omega(omega);

for (int i = 0; i < N; ++i) {
 double t = t_nodes[i];
 ode_vecN_step_euler(t, h, y, 2, ode_vec2_rhs_oscillator);
 ...
}
```

In questo modo:

- la firma di `ode_vecN_step_euler` resta semplice e C-like;
- `ode_vec2_rhs_oscillator` resta un puntatore a funzione “secco” compatibile;
- puoi cambiare `omega` a runtime (anche da menu) chiamando `ode_set_oscillator_omega`.<sup>323</sup>

Per una libreria didattica imperativa è probabilmente la via più semplice e coerente con lo stile che stai usando adesso.

Ti va se nel prossimo passo ti scrivo un `ode_vec2_test_euler_oscillator` che usa proprio questa seconda strategia con `ode_set_oscillator_omega`, così puoi integrare subito una seconda voce di menu accanto al decadimento scalare?

**ok per la variabile globale, senza modificare la signature per passarla.  
Va solo in a21, non anche nella prima componente?**

Giusto,  $\omega$  entra solo nella seconda equazione, quindi solo nella componente  $p'$ , non in  $q'$ .

Per l'oscillatore armonico:

$$q' = p, \quad p' = -\omega^2 q,$$

scriviamo  $y = (q, p)$ . Allora:

- prima componente:

$$y'_0 = q' = p = y_1,$$

che non contiene  $\omega$ ; - seconda componente:

$$y'_1 = p' = -\omega^2 q = -\omega^2 y_0,$$

qui compare  $\omega^2$ .

Quindi la RHS vettoriale C-like diventa:

```
static double ode_oscillator_omega = 1.0;

void ode_set_oscillator_omega(double omega) {
 ode_oscillator_omega = omega;
}

void ode_vec2_rhs_oscillator(double t,
 const double* y,
 double* dydt, int n) {
 (void)t;
 (void)n; // assumiamo n == 2
 const double omega = ode_oscillator_omega;

 dydt[0] = y[1]; // q' = p
 dydt[1] = -omega * omega * y[0]; // p' = -omega^2 q
}
```

Quindi:

- nessuna modifica ad **a11** e **a12** (in forma matriciale sarebbero  $a_{11} = 0, a_{12} = 1$ );
- solo **a21** diventa  $-\omega^2$ , mentre **a22** = 0.

Se sei d'accordo, nel prossimo passo costruiamo sopra questa RHS il `ode_vec2_test_euler_oscillator` con il solito schema `N_values`, nodi equidistanti, vettori  $q(t_i), p(t_i)$  ed energia  $E(t_i)$ .

---

**benissimo, cosi' e' clonabile tutto il mio ramo di main in cui vado ad inserire i nuovi calcoli**

Esattamente: con `ode_vec2_rhs_oscillator` parametrizzata via la globale `ode_oscillator_omega` puoi clonare la struttura del main che hai usato per il decadimento e innestarci il caso oscillatore senza toccare le firme delle funzioni.<sup>324325</sup>

## Passo successivo naturale

Per restare in perfetta analogia con il caso scalare, ti suggerirei questo flusso:

1. Aggiungi nel modulo ODE:

```
static double ode_oscillator_omega = 1.0;

void ode_set_oscillator_omega(double omega) {
 ode_oscillator_omega = omega;
}

void ode_vec2_rhs_oscillator(double t,
 const double* y,
 double* dydt, int n) {
 (void)t;
 (void)n; // assumiamo n == 2
 const double omega = ode_oscillator_omega;

 dydt = y[39_1];
 dydt[39_1] = -omega * omega * y;
}
```

2. Clona il pattern di `ode_euler()` in una nuova funzione, ad esempio:

```
void ode_oscillator_euler();
```

che usi: - `N_values` come per il decadimento; - `h_ticks` e `nodi_equidistanti` per i nodi; - `ode_vecN_step_euler(t,h,y,2,ode_oscillator_euler)` per l'evoluzione; - vettori `q_euler`, `p_euler`, `E_euler` per salvare le traiettorie. 3. Aggiungi la nuova voce di menu nel main: - “1) Decadimento scalare: Eulero vs Heun” - “2) Oscillatore 2D: Eulero (q,p,energia)”

Così:

- il ramo “decadimento” e il ramo “oscillatore” condividono tutto lo scheletro (loop su `N_values`, generazione dei nodi, struttura delle figure Matplot);
- le uniche differenze sono:
  - dimensione dello stato (`scalar` vs `2D`),
  - RHS (`ode_scalar_rhs_decay` vs `ode_vec2_rhs_oscillator`),
  - cosa plotti (`y(t)` vs `q(t),p(t),E(t)`).

Quando hai clonato il ramo e hai un primo `ode_oscillator_euler()` compilabile, se ti va incollalo qui: possiamo rivedere insieme i dettagli di visualizzazione (scelta di `T`, `omega`, cosa mettere nei quattro pannelli) e, se vuoi, iniziare a ragionare su come introdurre un metodo simplettico minimo (tipo Eulero semi-implicito) da confrontare con Eulero esplicito.

---

```
// ode_scalar_step_euler(t, h, &y_euler, ode_scalar_rhs_decay);
```

```
// ode_scalar_step_heun(t, h, &y_heun, ode_scalar_rhs_decay); ode_vecN_step_euler(t, h, y, 2
, ode_vec2_rhs_oscillator);
```

il parametro `y` non è definito, dovrebbe essere una coppia di valori, no?

Sì, esatto: per il caso vettoriale `y` deve essere un array (o vettore) di due componenti ( $q, p$ ), e va inizializzato prima del loop e aggiornato a ogni passo.<sup>326</sup>

Con la firma attuale:

```
void ode_vecN_step_euler(double t, double h,
 double* y, int n,
 void (*f)(double, const double*, double*, int));
```

la chiamata corretta è qualcosa del tipo:

```
double y[40_2]; // stato: y[40_0] = q, y[40_1] = p

y[40_0] = q0;
y[40_1] = p0;

for (int i = 0; i < N; ++i) {
 double t = t_nodes[i];
 ode_vecN_step_euler(t, h, y, 2, ode_vec2_rhs_oscillator);
 // ora y[40_0], y[40_1] sono q(t_{i+1}), p(t_{i+1})
}
```

Se vuoi tracciarne la traiettoria come per il caso scalare, conviene aggiungere i vettori:

```
Vec q_euler_vec(N+1);
Vec p_euler_vec(N+1);

// condizioni iniziali
q_euler_vec[40_0] = y[40_0];
p_euler_vec[40_0] = y[40_1];

for (int i = 0; i < N; ++i) {
 double t = t_nodes[i];

 ode_vecN_step_euler(t, h, y, 2, ode_vec2_rhs_oscillator);

 q_euler_vec[i+1] = y[40_0];
 p_euler_vec[i+1] = y[40_1];
}
```

Quindi:

- $y$  è la “coppia”  $(q, p)$  passata per indirizzo alla funzione di step;
- $q\_euler\_vec$  e  $p\_euler\_vec$  sono i vettori usati solo per plottare  $q(t_i)$ ,  $p(t_i)$ .

Se mi incolli il blocco completo che contiene quella riga commentata, posso aiutarti a inserire queste definizioni di  $y$ ,  $q\_euler\_vec$ ,  $p\_euler\_vec$  nel posto giusto senza rompere la struttura del tuo `ode_euler()` clonata per l'oscillatore.

---

## dovremmo quasi esserci. Poi come lo presentiamo in bidimensionalita'?

### con uno scatter plot?

Per presentarlo in bidimensionalità ha molto senso usare il **diagramma di fase**  $(q, p)$ , quindi un plot di  $p$  in funzione di  $q$ , non del tempo. Puoi farlo sia come scatter sia come polilinea; per l'oscillatore armonico è più leggibile una polilinea continua.<sup>327</sup>

### Dati da costruire

Nel tuo loop 2D, oltre ai vettori  $q(t_i)$  e  $p(t_i)$ :

```
Vec q_euler_vec(N+1);
Vec p_euler_vec(N+1);

double y[41_2];
y[41_0] = q0;
y[41_1] = p0;
```

```

q_euler_vec[41_0] = y[41_0];
p_euler_vec[41_0] = y[41_1];

for (int i = 0; i < N; ++i) {
 double t = t_nodes[i];
 ode_vecN_step_euler(t, h, y, 2, ode_vec2_rhs_oscillator);
 q_euler_vec[i+1] = y[41_0];
 p_euler_vec[i+1] = y[41_1];
}

```

Se vuoi, puoi anche calcolare l'esatta ( $q_{\text{exact}}, p_{\text{exact}}$ ) come:

$$q(t) = q_0 \cos(\omega t) + \frac{p_0}{\omega} \sin(\omega t), \quad p(t) = -\omega q_0 \sin(\omega t) + p_0 \cos(\omega t),$$

per confrontare il cerchio/ellisse esatta con la spirale numerica.<sup>328</sup>

## Visualizzazione suggerita

Con Matplot potresti organizzare i pannelli così:

- Pannello 1:  $q(t)$  vs  $t$  (Eulero vs eventuale esatta).
- Pannello 2:  $p(t)$  vs  $t$ .
- Pannello 3: **diagramma di fase**  $q$  sull'asse x,  $p$  sull'asse y:
  - linea continua per la traiettoria di Eulero:

```

auto phase = plot(q_euler_vec, p_euler_vec);
phase->line_style("-");
phase->color("green");
phase->display_name("Euler phase");

- se calcoli l'esatta, aggiungi:

auto phase_exact = plot(q_exact_vec, p_exact_vec);
phase_exact->line_style("-");
phase_exact->color("black");
phase_exact->display_name("Exact phase");

```

- Pannello 4: energia  $E(t)$  vs  $t$ , per mostrare la deriva energetica.

Per il diagramma di fase, una **polilinea** rende più chiaro il senso della dinamica (spirale che esce / entra), mentre lo scatter puro perde l'informazione d'ordine temporale; se vuoi comunque usare scatter, puoi aggiungere marker piccoli:

```

phase->marker(".");
phase->line_style("-");

```

così mantieni sia la traccia continua sia i nodi.

Se ti va, nel prossimo passo puoi incollare la bozza di `ode_vec2_test_euler_oscillator` così rivediamo insieme dove inserire il phase-plot e, eventualmente, come aggiungere una curva esatta per confronto geometrico.

---

**questo e' un assaggio con q(t) perche' sto ancora pensando in che ordine riempire tre panel e volevo vedere questo per primo**

L'assaggio è già molto eloquente: si vede tutto quello che volevamo mostrare sull'oscillatore con Eulero esplicito.<sup>329330</sup>



## Cosa mostra bene questo grafico

- Per  $N = 10$  (e ancora per 20) la curva verde (Eulero) “prende velocità” troppo in fretta, accumula fase e ampiezza errate e finisce a valori molto più negativi dell’esatta; è il classico comportamento dissipativo/instabile di Eulero esplicito applicato a un oscillatore.<sup>331332</sup>
- Aumentando  $N$  (40, 80) la curva si avvicina all’esatta, ma si nota comunque che:
  - la fase è leggermente sfasata,
  - l’ampiezza non coincide esattamente, anticipando la futura discussione sulle proprietà geometriche (energia non conservata, orbite che si allargano o collassano).<sup>333</sup>
- Il layout è coerente con quello del decadimento: titolo per pannello “ $q(t)$  vs  $t$  con  $N$  intervalli” e label “ $t$  (nodi equidistanti)” in ascissa.

È un ottimo “primo pannello”; a questo punto ha senso decidere come completare il blocco per l’oscillatore:

- un pannello analogo per  $p(t)$ ;
- un diagramma di fase  $q$  vs  $p$ ;
- un pannello per l’energia numerica  $E(t)$ .

Se vuoi mantenere coerenza con quanto fatto sul decadimento, potresti:

- usare il layout  $2 \times 2$  che già hai (quattro valori di  $N$ ) per  $q(t)$  come hai fatto;
- dedicare una seconda funzione di test/figura a un solo  $N$  (es.  $N = 80$ ) ma con 3 pannelli:
  - $q(t)$ ,
  - $p(t)$ ,
  - diagramma di fase / energia.

Così hai:

- un “blocco a 4 pannelli” che mostra la convergenza in  $q(t)$  al variare di  $N$ ;
- un “blocco a 3–4 pannelli” che fissa  $N$  e mostra la struttura qualitativa (fase, energia).

Ti va di procedere così: chiudiamo questo primo blocco aggiungendo il diagramma di fase e l’energia per, diciamo,  $N = 80$ , e poi più avanti introduciamo un metodo simplettico minimo da confrontare con Eulero sullo stesso oscillatore? <sup>334</sup>

---

**allora se mi riepiloghi 4 contenuti da confrontare, io sfrutterei il layout a 4 panel di ogni fig per mostrare un contenuto a diversi  $N$  come ho fatto qui e fig diverse per contenuti diversi. In questo modo non siamo limitati dalla leggibilità perché 4 h per le prove mi sembrano sufficienti mentre confronti se ne possono fare anche più di 4 e quindi con le fig siamo liberi, devo solo vedere sintatticamente come gestirne un array**

L’idea di usare 4 pannelli per un singolo contenuto a diversi  $N$  e figure diverse per contenuti diversi è perfetta: mantiene leggibilità e ti lascia libero di aggiungere confronti senza intasare una singola figura.<sup>335336</sup>

Ti propongo quattro contenuti “canonici” per l’oscillatore, tutti costruibili sulla stessa infrastruttura:

### 1. $q(t)$ vs $t$ (già fatto)

- Contenuto:  $q(t)$  numerico vs  $q(t)$  esatto.
- Figura: 4 pannelli, uno per  $N = 10, 20, 40, 80$ .
- Uso: mostra la convergenza puntuale in una componente dello stato.

È la figura che hai già preparato; basta eventualmente aggiornare solo le etichette se vuoi.<sup>337</sup>

## 2. $p(t)$ vs $t$

- Contenuto:  $p(t)$  numerico vs  $p(t)$  esatto, stesso schema di  $q(t)$ .
- Figura: ancora 4 pannelli, stesso set di  $N$ .
- Uso: mostra come Eulero approssima male la componente “velocità” e come migliora al crescere di  $N$ .

Implementazione: riutilizzi il loop, ma salvi/plotti `p_euler_vec` e `p_exact_vec`.<sup>338339</sup>

## 3. Diagramma di fase $(q, p)$

- Contenuto: orbitale nel piano di fase:
  - curva exact: ellisse (per  $\omega$  generico) o cerchio (per  $\omega = 1$ );
  - curva Eulero: spirale che esce o entra.
- Figura: qui ha senso fissare un solo  $N$  (es.  $N = 80$ ) e dedicare una figura a:
  - un solo pannello  $(q, p)$ ;
  - oppure 4 pannelli con  $N = 10, 20, 40, 80$  per vedere come la spirale cambia.

Uso: mostra geometricamente la non-conservazione delle invarianti (energia), preludio ai metodi symplettici.<sup>340341</sup>

## 4. Energia $E(t)$ vs $t$

- Contenuto: energia numerica

$$E(t_i) = \frac{1}{2} (p_i^2 + \omega^2 q_i^2)$$

confrontata con l’energia esatta (costante). - Figura: 4 pannelli, uno per ciascun  $N$ , o anche una sola figura con 4 curve (4  $N$ ) nello stesso pannello. - Uso: mostra come Eulero faccia crescere/decrescere energia in modo sistematico, e come ridurre  $h$  ne attenui ma non elimini l’effetto.<sup>342343</sup>

Implementazione: nel loop 2D calcoli `E_euler_vec[i+1]` e, se vuoi, una `E_exact_vec[i+1] = E0` costante.

---

Con questa scelta:

- ogni figura  $4 \times 1$  (o  $2 \times 2$ ) mostra **un contenuto** a 4  $N$ ;
- puoi avere, ad esempio:
  - `fig_q(t)` ( $q$  vs  $t$ , 4  $N$ ),
  - `fig_p(t)` ( $p$  vs  $t$ , 4  $N$ ),
  - `fig_phase` ( $q$  vs  $p$ , 1 o 4  $N$ ),
  - `fig_energy` ( $E$  vs  $t$ , 4  $N$ ).

A livello di codice, ti basterà:

- organizzare un array/vettore di figure Matplot, oppure creare/distruocere una figura per contenuto (come fai già per il decadimento);
- clonare il pattern del tuo `ode_euler()` per ciascun contenuto, cambiando solo:
  - quali vettori plotti (`q_euler_vec`, `p_euler_vec`, `E_euler_vec`);
  - titoli e label.

Ti va, come prossimo passo, di fissare insieme l’ordine “canonico”:

1. figura  $q(t)$ ,
2. figura  $p(t)$ ,
3. figura fase,
4. figura energia,

e magari iniziare dalla #3 (fase) che è quella più “visivamente drammatica”?

---

gia' che c'ero, prima di uscire e ricompilare, ho fatto una prova con  $t = [0, 10]$  e  $\omega = 2$  :D

Questo grafico è un manifesto perfetto di quanto Eulero esplicito sia inadatto (e potenzialmente instabile) per oscillatori con frequenza più alta e passo non adeguatamente piccolo: le esplosioni di ampiezza che vedi per  $N = 10, 20$  con  $\omega = 2$  sono esattamente ciò che la teoria della regione di stabilità dell'equazione test  $y' = \lambda y$  anticipa.<sup>344345</sup>

- Per  $\omega = 2$ , l'oscillatore lineare corrisponde, in forma complessa, a un'equazione  $y' = i\omega y$  con  $\lambda = i\omega$ . La regione di stabilità di Eulero esplicito è il disco centrato in  $-1$  di raggio 1 nel piano complesso; punti puramente immaginari con  $|\lambda h|$  non piccolo cadono fuori dal disco, quindi la soluzione numerica cresce invece di restare su orbite chiuse.<sup>346347</sup>
- Nei pannelli con  $N = 10, 20$  (quindi  $h$  piuttosto grande su  $[0, 10]$ ) la curva verde diverge violentemente, mentre la nera oscilla pacificamente: questo è l'effetto geometrico della moltiplicazione per  $(1 + h\lambda)$  con  $|1 + h\lambda| > 1$  in modulo (in questo caso  $\lambda$  immaginaria fa ruotare, ma il modulo si gonfia).<sup>348349</sup>
- Aumentando  $N$  (quindi riducendo  $h$ ), la situazione migliora: per  $N = 80$  la traiettoria numerica resta ancora qualitativamente vicina, ma si intravede comunque un drift di ampiezza e fase.

Direi che, come materiale per spiegare stabilità numerica e regioni di stabilità, questo grafico è oro puro.

Se ora vuoi “chiudere il cerchio”, i prossimi due passi naturali potrebbero essere:

1. fissare  $\omega$  e  $N$  e guardare il **diagramma di fase**  $(q, p)$  per vedere la spirale rispetto all'ellisse esatta;
2. introdurre un metodo symplettico minimo (es. Eulero semi-implicito) e confrontarlo con Eulero esplicito sugli stessi parametri, per mostrare come la spirale numerica si trasformi in un'orbita chiusa numericamente stabile.

Vuoi che nel prossimo messaggio ti scriva esplicitamente un `ode_vec2_step_euler_symplectic` (o semi-implicito) da aggiungere accanto a `ode_vecN_step_euler`, così puoi replicare i grafici con la versione symplettica?<sup>350</sup>

**curva exact: ellisse (per generico) o cerchio (per  $\omega=1$ );**

qui ho un dubbio sui coefficienti della conica, devono venire identici con  $\omega^2 = 1$  ma non vedo il nesso con  $a, b$  ed  $r$

Per l'oscillatore armonico lineare, la traiettoria nel piano di fase  $(q, p)$  è sempre un'ellisse centrata nell'origine, e diventa un cerchio solo per una scelta specifica di scala tra  $q$  e  $p$ .<sup>351352</sup>

**Equazione dell'orbita in  $(q, p)$**

Considera l'oscillatore:

$$q' = p, \quad p' = -\omega^2 q.$$

L'energia (Hamiltoniana) è

$$E = \frac{1}{2} (p^2 + \omega^2 q^2).$$

Per una soluzione con energia  $E_0$  costante, nel piano  $(q, p)$  hai:

$$p^2 + \omega^2 q^2 = 2E_0 = \text{costante}.$$

Questa è l'equazione di un'ellisse con assi principali lungo gli assi  $q$  e  $p$ .<sup>353</sup>

Se la metti nella forma standard di conica:

$$\frac{q^2}{a^2} + \frac{p^2}{b^2} = 1,$$

ottieni:

- $a^2 = \frac{2E_0}{\omega^2}$ ,
- $b^2 = 2E_0$ .

Quindi:

- semiasse in direzione  $q$ :  $a = \sqrt{2E_0}/\omega$ ,
- semiasse in direzione  $p$ :  $b = \sqrt{2E_0}$ .

La forma è ellittica per qualunque  $\omega \neq 0$ ; l'anisotropia è data proprio da  $\omega$ .<sup>354</sup>

## Quando diventa un cerchio

Per avere un **cerchio** nel piano che stai guardando, devi avere  $a = b$ . Con le definizioni sopra:

$$a = b \iff \frac{\sqrt{2E_0}}{\omega} = \sqrt{2E_0} \iff \omega = 1.$$

Quindi, se guardi il piano  $(q, p)$  **senza riscaldare nulla**, ottieni:

- ellisse per  $\omega \neq 1$ ,
- cerchio per  $\omega = 1$ .<sup>355</sup>

Se invece prendi un  $\omega$  generico e cambi variabile, ad esempio definendo

$$\tilde{p} = \frac{p}{\omega},$$

allora l'energia diventa

$$E = \frac{1}{2} (\omega^2 \tilde{p}^2 + \omega^2 q^2) = \frac{\omega^2}{2} (\tilde{p}^2 + q^2),$$

e la curva di energia costante è

$$\tilde{p}^2 + q^2 = \frac{2E_0}{\omega^2},$$

cioè un cerchio nel piano  $(q, \tilde{p})$  (con raggio  $\sqrt{2E_0}/\omega$ ).<sup>356</sup>

Quindi il nesso con i “coefficienti della conica” è:

- in  $(q, p)$ : ellisse  $p^2 + \omega^2 q^2 = \text{costante}$ , con  $a, b$  diversi;
- per  $\omega = 1$ :  $p^2 + q^2 = \text{costante} \rightarrow$  cerchio di raggio  $r = \sqrt{2E_0}$ ;
- in coordinate riscalate  $(q, \tilde{p} = p/\omega)$ : sempre un cerchio.

Se nel tuo diagramma di fase vuoi “vedere un cerchio”, allora:

- o scegli  $\omega = 1$ ,
- o plotti  $(q, p/\omega)$  invece di  $(q, p)$ .

Ti torna questo collegamento tra  $\omega^2$ , i coefficienti della conica e i semiassi  $a, b, r$  del cerchio/ellisse, o vuoi che scriviamo esplicitamente le soluzioni  $q(t), p(t)$  per un dato  $(q_0, p_0)$  e ne deduciamo l'equazione dell'orbita passo per passo?

va bene, posso plottare  $p^2 + \omega^2 q^2$  ma devo calcolare un livello di energia dal segnale iniziale o qualsiasi  $>0$  pur di avere una scala decente?

Per plottare  $p^2 + \omega^2 q^2$  ha molto più senso usare l'energia reale del segnale iniziale, non un livello arbitrario: in questo modo la curva esatta è un'ellisse “fisica” che corrisponde esattamente alla traiettoria che stai integrando.<sup>357358</sup>

### Come fissare il livello di energia

Dato il dato iniziale  $(q_0, p_0)$ , l'energia esatta è

$$E_0 = \frac{1}{2} (p_0^2 + \omega^2 q_0^2).$$

- Questo  $E_0$  è costante per la soluzione esatta.
- Nel piano  $(q, p)$  la curva esatta soddisfa:

$$p^2 + \omega^2 q^2 = 2E_0.$$

- Se vuoi, puoi calcolare i semiassi:

$$a^2 = \frac{2E_0}{\omega^2}, \quad b^2 = 2E_0,$$

ma per il plot basta usare i punti  $(q(t), p(t))$ .

Quindi, nel codice:

```
double E0 = 0.5 * (p0*p0 + omega*omega * q0*q0);
```

e poi:

- per la soluzione numerica:

```
E_euler_vec[i] = 0.5 * (p_i*p_i + omega*omega * q_i*q_i);
```

- per la “soluzione esatta” (se plotti energia nel tempo):  $E_{\text{exact\_vec}}[i] = E_0$  costante.<sup>359360</sup>

### Cosa plottare concretamente

Hai due opzioni (complementari):

1. **Diagramma di fase**  $(q, p)$  Usa direttamente i vettori  $(q_i, p_i)$ :
  - Esatta:  $(q_{\text{exact}}(t_i), p_{\text{exact}}(t_i)) \rightarrow$  ellisse.
  - Eulero:  $(q_i^{(E)}, p_i^{(E)}) \rightarrow$  spirale che devia dall'ellisse.<sup>361362</sup>

Qui non serve tracciare il livello  $p^2 + \omega^2 q^2 = 2E_0$  come curva implicita, perché la traiettoria esatta già lo realizza.

2. **Energia**  $E(t)$  vs  $t$  Plotti: -  $E^{(E)}(t_i) = \frac{1}{2}(p_i^2 + \omega^2 q_i^2)$ ; -  $E_{\text{exact}}(t_i) = E_0$  costante.

Questo rende chiarissimo come Eulero non conservi il livello d'energia:  $E^{(E)}(t)$  oscilla o cresce/decrese nel tempo.<sup>363364</sup>

Quindi: sì, usa sempre il livello di energia derivato da  $(q_0, p_0)$ ; non ha senso scegliere un  $E > 0$  arbitrario solo “per scala”, perché vorresti che ellisse/energia esatta coincidano con la dinamica che stai effettivamente integrando.

Se vuoi, nel passo successivo possiamo fissare un caso specifico (es.  $\omega = 1$ ,  $q_0 = 1$ ,  $p_0 = 0$ ), scrivere le formule esplicite per  $q_{\text{exact}}(t)$ ,  $p_{\text{exact}}(t)$  e mostrarti come riempire `q_exact_vec`, `p_exact_vec`, `E_exact_vec` nel tuo loop per ottenere sia l'ellisse esatta sia la linea di energia costante nello stesso framework Matplot.

ma secondo te, visto che ho solo trasformato fig ed ax in array e duplicato il codice di euler q vs exact q per fare la successiva euler p vs exact p, ho fatto qualche errore di mio o a matplotlibplus non piacciono quei due array?

Perche' mi apre i panels esattamente come prima ma non plotta niente, su entrambe le fig[0] e fig[1]. O c'e' qualcosa che non riesco a vedere o devo tornare agli scalari.

```
void ode_osc2d() {
 //
 // confronto Eulero-Heun test runner
 //
 int N_values[] = {10, 20, 40, 80}; // numero di intervalli (i nodi sono +1) su cui iterare
 int nN = 4; // dimensione di N_values

 const double a = 0.0;
 double b = 5.0;

 double q0 = 1.0; // condizione al bordo q(t0)
 double p0 = 1.0; // condizione al bordo p(t0)
 double omega = ode_oscillator_omega;
 std::cout << "\# N h err_euler err_heun\n";

 bool leave = false;
 bool redo = false;
 while (!leave) {
 clear_screen();
 redo = false;
 ode_set_oscillator_omega(omega); // loop successivi, se reimpostato utente
 Vec h_vals;
 Vec err_euler_vals; // <===
 Vec err_heun_vals; // <===

 figure_handle fig[4];
 axes_handle ax[4];

 fig[0] = matplot_table_init(true, "Metodi iterativi per problema di Cauchy", "Oscillatore 2D: q(t) = q0 * cos(omega*t)");
 fig[1] = matplot_table_init(true, "Metodi iterativi per problema di Cauchy", "Oscillatore 2D: p(t) = p0 * sin(omega*t)");
 fig[2] = matplot_table_init(true, "Metodi iterativi per problema di Cauchy", "Oscillatore 2D: q(t) - q_exact(t)");
 fig[3] = matplot_table_init(true, "Metodi iterativi per problema di Cauchy", "Oscillatore 2D: p(t) - p_exact(t)");
 matplot::legend();

 for (int k = 0; k < nN; ++k) {
 const int N = N_values[k];
 if (N <= 0) continue;

 // passo e nodi equidistanti
 const double h = h_ticks(a, b, N+1); // n_ticks vuole nodi
 Vec t_nodes = nodi_equidistanti(a, b, N+1); // dovrebbe riempire t_nodes con N_values[k]+1

 // vettori di soluzione
 Vec q_euler_vec(N+1);
 Vec p_euler_vec(N+1);
 Vec p_exact_vec(N+1);
 Vec q_exact_vec(N+1);

 double y[2]; // stato: y[0] = q, y[1] = p
```

```

 y[0] = q0;
 y[1] = p0;

// condizioni iniziali
 q_euler_vec[0] = y[0];
 p_euler_vec[0] = y[1];
 q_exact_vec[0] = y[0];
 p_exact_vec[0] = y[1];

double p0_w = p0 / omega;
 double w_q0 = - omega * q0;

for (int i = 0; i < N; ++i) {
 double t = t_nodes[i];
 ode_vecN_step_euler(t, h, y, 2, ode_vec2_rhs_oscillator);
 q_euler_vec[i+1] = y[0];
 p_euler_vec[i+1] = y[1];
 double wt = omega * t;
 q_exact_vec[i+1] = q0 * f_cos(wt) + p0_w * f_sin(wt);
 p_exact_vec[i+1] = w_q0 * f_sin(wt) + p0 * f_cos(wt);
}
h_vals.push_back(h);
// Visualizzazione confronto q(t)
fig[0]->nexttile(k);
ax[0] = fig[0]->current_axes();
ax[0]->title("q(t) vs t con "+itostr(N) + " Intervalli");
hold(on);

// q->use_y2(true);
// ax->y2_axis().limits({-1.6, 1.6});

auto q = plot(t_nodes, q_euler_vec);
q->line_style("-");
q->line_width(6);
q->color("green");
q->marker("");
q->display_name("Euler");

auto eq = plot(t_nodes, q_exact_vec);
eq->color("black");
eq->line_style("_");
eq->marker("");
eq->line_width(2);
eq->display_name("Exact");

matplot_legend_align(matplot::legend(), LeA1::Top+LeA1::Right, 0,0);
matplot::legend();
xlabel("t (nodi equidistanti)");
ylabel("q approssimata");

// Visualizzazione confronto p(t)
fig[1]->nexttile(k);
ax[1] = fig[1]->current_axes();
ax[1]->title("p(t) vs t con "+itostr(N) + " Intervalli");
hold(on);

// q->use_y2(true);
// ax->y2_axis().limits({-1.6, 1.6});

```

```

auto p = plot(t_nodes, p_euler_vec);
 p->line_style(" -");
 p->line_width(6);
 p->color("cyan");
 p->marker("");
 p->display_name("Euler");

auto ep = plot(t_nodes, p_exact_vec);
 ep->color("black");
 ep->line_style("_");
 ep->marker("");
 ep->line_width(2);
 ep->display_name("Exact");

matplot_legend_align(matplot::legend(), LeAl::Top+LeAl::Right, 0,0);
 matplot::legend();
 xlabel("t (nodi equidistanti)");
 ylabel("p approssimata");
}

fig[0]->draw();
fig[1]->draw();

/*
 auto fig_err = matplot_table_init(true, "Eulero vs Heun", "Metodi iterativi per problema di Ca
 matplot::hold(on);
 fig_err->nexttile(0); // ed unico
 auto ax=fig_err->current_axes();

// Eulero
 auto pe = matplot::loglog(h_vals, err_euler_vals);
 pe->line_style("--");
 pe->marker("");
 pe->line_width(3);
 pe->color("green");
 pe->display_name("Euler error");

// Heun
 auto ph = matplot::loglog(h_vals, err_heun_vals);
 ph->line_style("-.");
 ph->marker("");
 ph->line_width(3);
 ph->color("red");
 ph->display_name("Heun error");

matplot::xlabel("h");
 matplot::ylabel("|y(b) - y_num(b)|");
 matplot_legend_align(matplot::legend(), LeAl::Bottom+LeAl::Right, 0,0);
 matplot::legend();
 fig_err->draw();

*/

redo = false;
 while (!redo && !leave) {
 std::cout << "Inserire nuovo max b 0 .. 100 (<q> per uscire) > ";

cin_clear();

```



Il codice numerico è coerente, e l'uso dei vettori `q_euler_vec`, `p_euler_vec`, `q_exact_vec`, `p_exact_vec` è corretto; il problema più probabile non è “che a `matplotlib` non piacciono gli array”, ma dettagli di uso dell'API (`figure/axes/legend`) o di coerenza nella lunghezza dei vettori.<sup>365366</sup>

### 1. Coerenza lunghezze t\_nodes / q\*\_vec

```
Vec q_euler_vec(N+1);
...
for (int i = 0; i < N; ++i) {
 double t = t_nodes[i];
 ...
 q_euler_vec[i+1] = y[47_0];
}
```

- `t_nodes.size() == N+1`,
- `q_euler_vec.size() == N+1`,
- loop sui passi da `i=0` a `i=N-1`, quindi l'ultima scrittura è `q_euler_vec[N]`.

## 2. Uso di axes\_handle ax[47\_4]

```
fig[~47_0]->nexttile(k);
ax[~47_0] = fig[~47_0]->current_axes();
...
fig[~47_1]->nexttile(k);
ax[~47_1] = fig[~47_1]->current_axes();
...
```

65

### 3. Global `hold(on)` e `legend`

Un paio di dettagli che possono dare sorprese:

- Stai chiamando `matplotlib::legend()` a livello globale subito dopo aver creato le figure:

```
fig = ...;
fig[~47_1] = ...;
fig[~47_2] = ...;
fig = ...;
matplotlib::legend();
```

Questo potrebbe associare la `legend` alla figura “corrente” (non necessariamente quella che poi usi), e se la figura corrente cambia all’interno dei loop, la `legend` può comportarsi in modo strano. Meglio chiamare `matplotlib::legend()` solo dopo aver creato le curve in ciascun pannello, come fai già alla fine del blocco per  $q(t)$  e  $p(t)$ . - `hold(on)` è chiamato senza qualificare la figura/axes; se Matplotlib++ usa uno stato globale per `hold`, è importante che la figura corrente sia quella che ti aspetti. Il pattern più sicuro è:

```
fig->nexttile(k);
auto ax0 = fig->current_axes();
matplotlib::hold(ax0, true); // se la tua versione supporta hold per axes
```

Se stai usando una versione dove `hold(on)` è globale, allora è meglio essere sicuri che `fig[~47_0]` sia la figura “attiva” quando lo chiami.

### 4. Differenza rispetto al caso scalare

Nel codice scalare che funzionava avevi, in sostanza:

- una sola `figure_handle` `fig`,
- un solo array di axes (o neanche quello),
- e chiamavi `plot(t_nodes, y_euler_vec) / plot(t_nodes, y_exact_vec)` immediatamente dopo `nexttile`.

Qui stai facendo la stessa cosa, ma con figure multiple. Per debug rapido ti suggerirei:

- Prima prova a commentare completamente `fig[~47_1]`, `fig[~47_2]`, `fig[~47_3]` e l’intero blocco “ $p(t)$ ”:
  - mantieni solo `fig[~47_0]` e il blocco  $q(t)$ ;
  - verifica che, così, il grafico di  $q(t)$  venga mostrato (come nel caso scalare).
- Se così funziona, riaggiungi `fig[~47_1]` e il blocco  $p(t)$ ; se a quel punto spariscono entrambe, probabilmente c’è qualche interazione tra figure/axes e `legend()/hold()` globali.

### 5. Micro-dettagli: `wt` e `q_exact_vec`

Un piccolo dettaglio, che non blocca il plot ma può generare distorsioni:

```
double wt = omega * t;
q_exact_vec[i+1] = q0 * f_cos(wt) + p0_w * f_sin(wt);
p_exact_vec[i+1] = w_q0 * f_sin(wt) + p0 * f_cos(wt);
```

Con  $t = t\_nodes[i]$  hai  $q(t_i)$ , ma `q_exact_vec[i+1]` dovrebbe essere  $q(t_{i+1})$ ; se vuoi sincronizzarlo con i nodi, dovresti usare `t_nodes[i+1]`. Anche questo non “ferma” Matplotlib, ma fa sì che esatta e numerica non siano allineate sui nodi.

---

In sintesi:

- il passaggio a vettori e array di figure/axes non è di per sé un problema;
- il comportamento che descrivi (figure vuote) è più probabilmente legato a:
  - interazioni tra più figure e `legend()/hold()` globali,
  - o a un incidente non visibile a occhio nei parametri passati a `plot`.

La prima prova semplice è davvero:

1. lascia solo `fig[~47_0]` e il blocco  $q(t)$  (esattamente come nel caso che funzionava);
2. assicurati che compili e mostri qualcosa;

3. poi aggiungi `fig[47_1]` e il blocco `p(t)` e vedi se scompare tutto o solo la seconda figura.

Vuoi che, nel frattempo, riscriva il blocco `for` per l'oscillatore in una versione "minimal" (solo una figura, solo `q(t)`, senza array di figure/axes), così puoi confrontare riga a riga con quella attuale e vedere dove si introduce la differenza?

---

**Questa non l'ho capita anche perche' tu ripeti la mia ma non la correzione:**

```
for (int i = 0; i < N; ++i) {
 double t = t_nodes[i];
 ode_vecN_step_euler(t, h,
 y, 2, ode_vec2_rhs_oscillator);
 q_euler_vec[i+1] = y[0];
 p_euler_vec[i+1] =
 y[1];
 double wt = omega * t;
 q_exact_vec[i+1] = q0 * f_cos(wt) + p0_w *
 f_sin(wt);
 p_exact_vec[i+1] = w_q0 * f_sin(wt) + p0 * f_cos(wt);
 double wt
 = omega * t;
}
```

per rendere correttamente parte sinistra e destra del passo dovrei metter in corrispondenza esatta(i) con euler(i+1) se non si e' confuso il mio cervello pure con gli array, e' il contrario?

Hai ragione a non essere convinto: così come è scritto, stai mescolando tempi e indici in modo incoerente tra numerico ed esatto. La correzione consiste proprio nel sincronizzare bene "da dove a dove" va il passo.<sup>368369</sup>

## Tempo del passo vs nodo

Nel loop:

```
for (int i = 0; i < N; ++i) {
 double t = t_nodes[i];
 ode_vecN_step_euler(t, h, y, 2, ode_vec2_rhs_oscillator);
 q_euler_vec[i+1] = y[48_0];
 p_euler_vec[i+1] = y[48_1];
 double wt = omega * t;
 q_exact_vec[i+1] = q0 * f_cos(wt) + p0_w * f_sin(wt);
 p_exact_vec[i+1] = w_q0 * f_sin(wt) + p0 * f_cos(wt);
}
```

- `t_nodes[i]` è il tempo  $t_i$ .
- Dopo `ode_vecN_step_euler`, `y` rappresenta il valore al tempo  $t_{i+1} = t_i + h$ . Quindi `q_euler_vec[i+1]` e `p_euler_vec[i+1]` sono valori in  $t_{i+1}$ .
- Ma `wt = omega * t` usa  $t = t_i$ , quindi `q_exact_vec[i+1]` e `p_exact_vec[i+1]` sono valori in  $t_i$  scritti all'indice  $i+1$ : è coerente con la formula analitica, ma non con l'allineamento agli indici dei nodi.<sup>370</sup>

Per avere "euler(i+1) esatta(i+1)" devi usare `t_{i+1}`, cioè `t_nodes[i+1]`.

## Correzione del blocco

La versione coerente è:

```
for (int i = 0; i < N; ++i) {
 double t = t_nodes[i]; // t_i
 ode_vecN_step_euler(t, h, y, 2, ode_vec2_rhs_oscillator);

 // numerico in t_{i+1}
 q_euler_vec[i+1] = y[48_0];
 p_euler_vec[i+1] = y[48_1];

 // esatto in t_{i+1}
 double t_next = t_nodes[i+1]; // t_{i+1}
 double wt = omega * t_next;
```

```

 q_exact_vec[i+1] = q0 * f_cos(wt) + p0_w * f_sin(wt);
 p_exact_vec[i+1] = w_q0 * f_sin(wt) + p0 * f_cos(wt);
}

```

Così:

- indice 0:  $q(t_0), p(t_0)$  iniziali, già messi fuori dal loop;
- indice  $i + 1$ : numerico e analitico entrambi valutati in  $t_{i+1}$ .<sup>371</sup>

Se volessi “abbinarli” in altro modo (es. Eulero in  $t_{i+1}$  vs esatto in  $t_i$ ), avrebbe senso solo per certe analisi, ma per il confronto visuale e per l’errore globale è naturale allinearli sullo **stesso nodo**.

Quindi sì: il tuo cervello non si è confuso, la corrispondenza corretta è “esatta(i+1) con Euler(i+1)”, e la correzione consiste nel passare da  $\text{omega} * t$  a  $\text{omega} * t\_nodes[i+1]$  dentro il loop.

## con l’accortezza che quel +1 non sfondi il boundary di t\_nodes

Esatto, è qui che entra la coerenza tra indici e dimensioni: con la scelta  $N$  intervalli,  $N + 1$  nodi, il +1 non sfonda il boundary se tieni il loop fino a  $i < N$ .<sup>372373</sup>

Hai:

- $N$  = numero di intervalli.
- `nodes = N+1`.
- `t_nodes.size() = nodes = N+1`.
- Indici validi per `t_nodes`: da 0 a  $N$ .

Nel loop:

```

for (int i = 0; i < N; ++i) {
 double t = t_nodes[i]; // t_i, i = 0..N-1

 ode_vecN_step_euler(t, h, y, 2, ode_vec2_rhs_oscillator);

 q_euler_vec[i+1] = y[49_0]; // i+1 = 1..N
 p_euler_vec[i+1] = y[49_1];

 double t_next = t_nodes[i+1]; // t_{i+1}, i+1 = 1..N (OK)
 double wt = omega * t_next;
 q_exact_vec[i+1] = q0 * f_cos(wt) + p0_w * f_sin(wt);
 p_exact_vec[i+1] = w_q0 * f_sin(wt) + p0 * f_cos(wt);
}

```

- Quando  $i = N-1$ ,  $i+1 = N$ , quindi accedi a `t_nodes[N]`, che è l’ultimo nodo valido.
- Non arrivi mai a  $i = N$ , quindi non tenti `t_nodes[N+1]`.

Quindi:

- `for (int i = 0; i < N; ++i)` + vettori dimensionati a  $N+1 \rightarrow$  nessun out-of-bounds;
- il +1 sugli indici è esattamente quello che ti serve per spostarti da  $t_i$  a  $t_{i+1}$  e tenerli allineati.

Se vuoi una sicurezza in più, puoi sempre mettere un `assert(t_nodes.size() == N+1)`; subito dopo la generazione dei nodi, ma con `nodi_equidistanti(a,b,N+1)` la costruzione è già coerente per definizione.

quello e' sistemato, per il resto non c'e' niente da fare, non lo trovo. Si e' rotto e basta :D

ho riportato a scalare e commentato la seconda fig, niente sempre vuota eppure mette i titoli corretti

```
void ode_osc2d() {
 //
 // confronto Eulero-Heun test runner
 //
 int N_values[] = {10, 20, 40, 80}; // numero di intervalli (i nodi sono +1) su cui iterare
 int nN = 4; // dimensione di N_values

 const double a = 0.0;
 double b = 5.0;

 double q0 = 1.0; // condizione al bordo q(t0)
 double p0 = 1.0; // condizione al bordo p(t0)
 double omega = ode_oscillator_omega;
 std::cout << "\# N h err_euler err_heun\n";

 bool leave = false;
 bool redo = false;
 while (!leave) {
 clear_screen();
 redo = false;
 ode_set_oscillator_omega(omega); // loop successivi, se reimpostato utente
 Vec h_vals;
 Vec err_euler_vals; // <===
 Vec err_heun_vals; // <===

 // figure_handle fig[4];
 // axes_handle ax[4];

 auto fig0 = matplot_table_init(true, "Metodi iterativi per problema di Cauchy", "Oscillatore 2D: q(t) = q0");
 auto fig1 = matplot_table_init(true, "Metodi iterativi per problema di Cauchy", "Oscillatore 2D: p(t) = p0");
 auto fig2 = matplot_table_init(true, "Metodi iterativi per problema di Cauchy", "Oscillatore 2D: q(t) = q0");
 auto fig3 = matplot_table_init(true, "Metodi iterativi per problema di Cauchy", "Oscillatore 2D: p(t) = p0");
 matplot::legend();

 for (int k = 0; k < nN; ++k) {
 const int N = N_values[k];
 if (N <= 0) continue;

 // passo e nodi equidistanti
 const double h = h_ticks(a, b, N+1); // n_ticks vuole nodi
 Vec t_nodes = nodi_equidistanti(a, b, N+1); // dovrebbe riempire t_nodes con N_values[k]+1

 // vettori di soluzione
 Vec q_euler_vec(N+1);
 Vec p_euler_vec(N+1);
 Vec p_exact_vec(N+1);
 Vec q_exact_vec(N+1);

 double y[2]; // stato: y[0] = q, y[1] = p
 y[0] = q0;
 y[1] = p0;

 // condizioni iniziali
```

```

 q_euler_vec[0] = y[0];
 p_euler_vec[0] = y[1];
 q_exact_vec[0] = y[0];
 p_exact_vec[0] = y[1];

double p0_w = p0 / omega;
double w_q0 = - omega * q0;

for (int i = 0; i < N; ++i) {
 double t = t_nodes[i];
 ode_vecN_step_euler(t, h, y, 2, ode_vec2_rhs_oscillator);
 q_euler_vec[i+1] = y[0];
 p_euler_vec[i+1] = y[1];
 double wt = omega * t_nodes[i+1];
 q_exact_vec[i+1] = q0 * f_cos(wt) + p0_w * f_sin(wt);
 p_exact_vec[i+1] = w_q0 * f_sin(wt) + p0 * f_cos(wt);
}
h_vals.push_back(h);
// Visualizzazione confronto q(t)
fig0->nexttile(k);
auto ax0 = fig0->current_axes();
ax0->title("q(t) vs t con "+itostr(N) + " Intervalli");
hold(on);

// q->use_y2(true);
// ax->y2_axis().limits({-1.6, 1.6});

auto q = plot(t_nodes, q_euler_vec);
q->line_style("-");
q->line_width(6);
q->color("green");
q->marker("");
q->display_name("Euler");

auto eq = plot(t_nodes, q_exact_vec);
eq->color("black");
eq->line_style("_");
eq->marker("");
eq->line_width(2);
eq->display_name("Exact");

matplot_legend_align(matplot::legend(), LeA1::Top+LeA1::Right, 0,0);
matplot::legend();
xlabel("t (nodi equidistanti)");
ylabel("q approssimata");

/*
// Visualizzazione confronto p(t)
fig1->nexttile(k);
auto ax1 = fig1->current_axes();
ax1->title("p(t) vs t con "+itostr(N) + " Intervalli");
ax1->hold(on);

// q->use_y2(true);
// ax->y2_axis().limits({-1.6, 1.6});

auto p = plot(t_nodes, p_euler_vec);
p->line_style("-");
p->line_width(6);

```



```

const double h = h_ticks(a, b, N+1);
Vec t_nodes = nodi_equidistanti(a, b, N+1);

Vec q_euler_vec(N+1);
Vec q_exact_vec(N+1);

double y[^50_2] = {q0, p0};

q_euler_vec[^50_0] = y[^50_0];
q_exact_vec[^50_0] = y[^50_0];

double p0_w = p0 / omega;
double w_q0 = -omega * q0;

for (int i = 0; i < N; ++i) {
 double t = t_nodes[i];
 ode_vecN_step_euler(t, h, y, 2, ode_vec2_rhs_oscillator);
 q_euler_vec[i+1] = y[^50_0];

 double wt = omega * t_nodes[i+1];
 q_exact_vec[i+1] = q0 * f_cos(wt) + p0_w * f_sin(wt);
}

fig0->nexttile(k);
auto ax0 = fig0->current_axes();
ax0->title("q(t) vs t con " + itostr(N) + " Intervalli");

matplot::hold(true);

auto q = plot(t_nodes, q_euler_vec);
q->display_name("Euler");

auto eq = plot(t_nodes, q_exact_vec);
eq->display_name("Exact");

matplot::legend();
matplot::xlabel("t");
matplot::ylabel("q");
}

fig0->draw();

```

Se così ancora non vedi nulla, significa che:

- o `t_nodes` e `q_euler_vec` non hanno davvero la stessa size (varrebbe la pena stampare `t_nodes.size()`, `q_euler_vec.size()`),
- o c'è qualche problema esterno (p.es. il tuo `plot` è un alias diverso, o il backend grafico non refresh).

Ma dato che il decadimento scalare funzionava, questa stessa struttura dovrebbe funzionare anche qui.

## 2. Uso consistente di `hold` e `legend`

Nel codice attuale hai:

```

auto fig0 = ...;
auto fig1 = ...;
auto fig2 = ...;
auto fig3 = ...;
matplot::legend();

```



Quel `matplot::legend()`; “a freddo” può creare confusione, perché la figura corrente non è chiaro quale sia in quel momento (dipende da come `matplot_table_init` setta lo stato globale).<sup>374</sup>

Ti consiglierei di:

- eliminare quel `matplot::legend()`; subito dopo la creazione delle figure;
- mantenere solo i `legend()` che fai dopo i plot, come:

```
matplot_legend_align(matplot::legend(), ...);
matplot::legend();
```

cioè quando sei sicuro che la figura/axes corrente sia quella giusta (subito dopo `nexttile`).

Anche per `hold`, se la tua versione di Matplot++ ha `matplot::hold(ax, true)`, è meglio usare quella per evitare stati globali:

```
auto ax0 = fig0->current_axes();
matplot::hold(ax0, true);
```

al posto di un `hold(on)` globale.

### 3. Coerenza con il caso che funziona

Nel caso decadimento:

- usavi un solo `figure_handle`.
- chiamavi `plot()` subito dopo `nexttile`.
- non avevi `matplot_table_init` multipli in serie.

Qui stai creando quattro figure all’inizio:

```
auto fig0 = ...;
auto fig1 = ...;
auto fig2 = ...;
auto fig3 = ...;
```

Se per qualche motivo il backend considera “corrente” solo l’ultima, `plot()` potrebbe disegnare su quella (o su nessuna se c’è un mismatch). Un ulteriore esperimento semplice:

- crea solo `fig0`, non `fig1-3`;
- sposta `matplot::legend()`; dentro il loop come nel caso scalare;
- verifica che il grafico appaia.

Se così funziona, possiamo poi riaggiungere `fig1, fig2, fig3` uno alla volta.

---

Non vedo errori concettuali nella parte di integrazione o nell’uso dei vettori; il fatto che i titoli ci siano ma le curve no, con codice analogo al caso scalare, punta fortemente a un problema di “figura/axes correnti” o di `legend()/hold()` globali piuttosto che a un bug di logica numerica.

Appena hai provato la versione “solo `fig0`, solo `q(t)`” e vedi se compare qualcosa, raccontami cosa succede: in base a quello possiamo andare più a fondo (ad esempio suggerendo un `std::cout << t_nodes.size() << " " << q_euler_vec.size()` e simili per escludere mismatch silenziosi).

---

## era legend

Ha perfettamente senso: un `matplot::legend()` “sparato” fuori contesto (subito dopo la creazione delle figure) può alterare la figura/axes correnti o resettare lo stato in modo che i plot successivi non compaiano dove pensi.<sup>375376</sup>

Con la struttura che hai:

- crei 4 figure (`fig0...fig3`);

- poi, globalmente, chiami `matplotlib::legend();`
- dentro il loop, dopo i plot, richiami `matplotlib::legend();` per ogni pannello.

Quella chiamata globale iniziale era il pezzo anomalo; toglierla e lasciare le `legend()` solo dopo i `plot` (come nel caso scalare che funzionava) è la scelta corretta.

In pratica, la regola empirica con Matplotlib++ (e simili) è:

- crea la figura,
- seleziona il pannello (`nexttile, current_axes`),
- fai i plot,
- **poi** chiama `legend()` riferita a quell'axes.

Niente `legend()` “al volo” prima che ci siano curve, perché rischia di legarsi a una figura/axes non desiderata.

Ora che hai isolato la causa, puoi tranquillamente re-introdurre le altre figure ( $p(t)$ , fase, energia) seguendo lo stesso pattern:

- niente `legend()` globale,
- una `legend()` per pannello, subito dopo i plot.

Quando avrai il phase plot e il grafico di energia pronti, se ti va condividili: sarà interessante vedere come Eulero esplode nel piano di fase rispetto all'ellisse esatta per  $\omega$  diversi.

---

## no, e' l'ordine delle fig

Ah, ok: allora il colpevole è ancora più “banale” e molto in stile Matplotlib/Matlab-like: l'ordine in cui crei le figure determina quale diventa “corrente” e quindi dove vanno a finire i plot.<sup>377</sup>

Nel tuo codice avevi:

```
auto fig0 = matplotlib_table_init(...);
auto fig1 = matplotlib_table_init(...);
auto fig2 = matplotlib_table_init(...);
auto fig3 = matplotlib_table_init(...);

// poi nel loop
fig0->nexttile(k);
auto ax0 = fig0->current_axes();
...
auto q = plot(t_nodes, q_euler_vec);
...
```

Se `matplotlib_table_init` o Matplotlib++ stesso, dopo aver creato `fig3`, lascia come **figura corrente** proprio `fig3`, allora:

- `fig0->nexttile(k);` seleziona il tile giusto in `fig0`,
- ma `plot(...)` potrebbe ancora riferirsi alla figura/axes correnti globali (che sono quelle di `fig3`, non di `fig0`), a seconda di come è implementata la tua versione di Matplotlib++.

Da qui il sintomo “titoli ci sono ma curve no”: il titolo viene settato esplicitamente su `ax0` (cioè sull'axes di `fig0`), mentre `plot` disegna altrove (o non disegna affatto nella figura che stai guardando).

Due modi robusti per evitarlo:

1. **Creare le figure solo quando servono e una alla volta** Esattamente come nel caso scalare:
  - prima fai tutti i plot di `fig0` e `fig0->draw();`,
  - poi, se ti serve, crei `fig1`, ecc.
2. **Assicurarti che plot lavori sulla figura/axes giusta** Se la tua versione di Matplotlib++ permette qualcosa tipo:

```

auto ax0 = fig0->current_axes();
matplot::hold(ax0, true);
matplot::plot(ax0, t_nodes, q_euler_vec);

```

allora è meglio usare quella forma esplicita invece del `plot` globale.

Già solo invertire l'ordine di creazione (o rimuovere le figure non usate) può cambiare quale figura è “corrente” e quindi far “tornare a posto” i plot — che è probabilmente quello che hai appena osservato.

In ogni caso, ora sai che:

- il numerico è corretto,
- i vettori sono coerenti,
- il comportamento anomalo era dovuto alla combinazione “più figure + API globale”.

Se mantieni la regola: “creo una figura, plottiamo, chiamo `draw()`, poi passo alla successiva”, ti risparmi un sacco di queste trappole.

Ti va, una volta stabilizzato questo, di passare a completare il phase-plot e l'energia usando lo stesso pattern (una figura per contenuto, 4 pannelli per i diversi  $N$ ), così chiudiamo il pacchetto oscillatore con quattro viste coerenti come hai già fatto per il decadimento?

## infatit così' non funziona di nuovo

```

auto fig2 = matplot_table_init(true, “Metodi iterativi per problema di Cauchy”, “Oscillatore 2D: diagramma di
fase (q,p)”, 2, 2);
auto fig3 = matplot_table_init(true, “Metodi iterativi per problema di
Cauchy”, “Oscillatore 2D: Energia = 1/2 * p(t)^2 + w^2 * q(t)^2”, 2, 2);
auto fig0 =
matplot_table_init(true, “Metodi iterativi per problema di Cauchy”, “Oscillatore 2D: q(t) = q0 * cos(wt) + p0/w
* sin(wt)”, 2, 2);
auto fig1 = matplot_table_init(true, “Metodi iterativi per problema di Cauchy”,
“Oscillatore 2D: p(t) = -w * q0 sin(wt) + p0 * cos(wt)”, 2, 2);

```

Allora il quadro è chiaro: il problema non è “l'ordine delle fig” in sé, ma il fatto che stai creando più figure con un'API **globale** (alla Matlab), e poi usi `plot` globale che disegna sulla *figura corrente*, non necessariamente quella su cui hai appena chiamato `nexttile`.<sup>378379</sup>

Con più `matplot_table_init(...)` in sequenza:

```

auto fig2 = ...;
auto fig3 = ...;
auto fig0 = ...;
auto fig1 = ...;

```

l'ultima figura creata (`fig1`) diventa quella “corrente” per Matplot. Se poi fai:

```

fig0->nexttile(k);
auto ax0 = fig0->current_axes();
auto q = plot(t_nodes, q_euler_vec);

```

è molto probabile che:

- `nexttile` e `current_axes` agiscano su `fig0`,
- ma `plot` usi ancora la figura/axes correnti globali (`fig1`), quindi `fig0` resta vuota.

Nel caso scalare funzionava perché avevi **una sola figura**, quindi figura corrente = figura su cui plotti.

## Strategia robusta

La via più solida, con l'API che hai, è:

1. Usare **una figura per volta**:

```

auto fig0 = matplot_table_init(...); // q(t)
// loop con plot di q(t)
fig0->draw();

```

```

auto fig1 = matplot_table_init(...); // p(t)
// loop con plot di p(t)
fig1->draw();

```

*// e così via per fig2 e fig3*

In altre parole: non creare 4 figure tutte insieme prima del loop, ma creare-plottare-draw per ognuna, sequenzialmente. Questo evita ambiguità sulla figura corrente. 2. In alternativa, se la tua versione lo supporta, usare la variante di plot che accetta l'axes:

```

fig0->nexttile(k);
auto ax0 = fig0->current_axes();
matplot::hold(ax0, true);
matplot::plot(ax0, t_nodes, q_euler_vec);

```

In questo modo specifichi esplicitamente *dove* disegnare, bypassando lo stato globale.

## Applicazione concreta al tuo caso

Per rimettere a posto senza cambiare troppo:

- Sposta la creazione delle figure **fuori e in blocchi**:

```

auto fig0 = matplot_table_init(... q(t) ... , 2, 2);

```

```

for (k ...) {
 // calcolo e plot q(t) vs t
}
fig0->draw();

```

```

auto fig1 = matplot_table_init(... p(t) ... , 2, 2);

```

```

for (k ...) {
 // stesso loop o uno nuovo, ma solo p(t) vs t
}
fig1->draw();

```

*// e più avanti fig2, fig3 per fase ed energia, analogamente*

- Non creare fig0..fig3 tutte assieme prima del loop: ogni `matplot_table_init` cambia la figura corrente; se poi i plot usano quella corrente, è molto facile “perdere” la destinazione.

---

Quindi, il problema non è che hai messo fig0 dopo fig2/fig3, ma che **stai creando più figure con uno stato globale e usando un plot globale**. Una figura alla volta, o un `plot(ax, ...)` esplicito, risolve la questione in modo deterministico.

Se reimposti `ode_osc2d()` in questo senso (prima solo fig0 per q(t), poi fig1 per p(t), ecc.), dovresti ritrovarti nella stessa situazione “affidabile” che avevi con il decadimento scalare.

---

## mentre così' si

```

auto fig2 = matplot_table_init(true, “Metodi iterativi per problema di Cauchy”, “Oscillatore 2D: diagramma di
fase (q,p)”, 2, 2);
auto fig3 = matplot_table_init(true, “Metodi iterativi per problema di

```

```
Cauchy", "Oscillatore 2D: Energia = 1/2 * p(t)^2 + w^2 * q(t)^2", 2, 2); auto fig1 =
matplot_table_init(true, "Metodi iterativi per problema di Cauchy", "Oscillatore 2D: p(t) = -w * q0 sin(wt) + p0
* cos(wt)", 2, 2); auto fig0 = matplot_table_init(true, "Metodi iterativi per problema di Cauchy",
"Oscillatore 2D: q(t) = q0 * cos(wt) + p0/w * sin(wt)", 2, 2);
```

Questo comportamento è coerente con l'ipotesi che l'ultima figura creata diventi quella "corrente" per Matplot, e che il `plot` globale disegni sulla figura corrente.<sup>380381</sup>

Con l'ordine:

```
auto fig2 = ...;
auto fig3 = ...;
auto fig1 = ...;
auto fig0 = ...;
```

l'ultima figura creata è `fig0`, che è proprio quella su cui poi fai `fig0->nexttile(k)` e subito dopo chiami `plot(...)`. In questo scenario:

- `fig0` è sia la figura che selezioni esplicitamente (via `nexttile`),
- sia la figura corrente globale per Matplot,
- quindi il `plot` globale finisce *di fatto* dentro `fig0` e tutto funziona.

Quando invece creavi `fig0` prima e poi `fig1`, `fig2`, `fig3`, l'ultima figura creata era un'altra (`fig3`), quindi:

- `fig0->nexttile(k)` selezionava il tile giusto in `fig0`,
- ma `plot` disegnava sulla figura corrente (`fig3`), lasciando `fig0` vuota.

Quindi non è che Matplot "non gradisse" gli array o il vettoriale: stavi semplicemente andando contro lo stato globale (alla Matlab) della figura attiva.

Per restare robusto e non dipendere da questo ordinamento "magico", hai due opzioni:

#### 1. Una figura alla volta (più sicuro)

```
auto fig0 = matplot_table_init(... q(t) ...);
```

```
// loop: plot q(t)
fig0->draw();
```

```
auto fig1 = matplot_table_init(... p(t) ...);
```

```
// loop: plot p(t)
fig1->draw();
```

```
// idem per fig2, fig3
```

Così la figura corrente è sempre quella che stai usando e il `plot` globale non può "scappare" altrove. 2. **Plot verso un axes specifico (se l'API lo permette)**

Se Matplot++ ti offre qualcosa come:

```
auto ax0 = fig0->current_axes();
matplot::plot(ax0, t_nodes, q_euler_vec);
```

allora specifichi esplicitamente l'axes destinazione e lo stato globale diventa irrilevante.

Il fatto che ora, con `fig0` creato per ultimo, tutto funzioni, conferma che il resto del codice numerico (loop, vettori, RHS dell'oscillatore) è corretto.

Se continui con la soluzione "una figura alla volta", ti suggerirei di:

- usare `fig0` solo per `q(t)`,
- una seconda funzione (o ramo) per `p(t)`,
- e figure separate per fase ed energia, così ogni figura è autonoma e non dipende dall'ordine di creazione rispetto alle altre.

Vuoi che, a partire da questa versione funzionante con `fig0`, ti proponga una struttura “a funzioni separate” tipo `ode_osc2d_q`, `ode_osc2d_p`, `ode_osc2d_phase`, `ode_osc2d_energy`, che riusano lo stesso loop numerico ma creano e disegnano una sola figura per volta?

# Note

## Notes

- <sup>1</sup><https://maddmaths.simai.eu/divulgazione/eulero/il-metodo-di-eulero/>
- <sup>2</sup>[https://it.wikipedia.org/wiki/Metodo\\_di\\_Eulero](https://it.wikipedia.org/wiki/Metodo_di_Eulero)
- <sup>3</sup>[https://it.wikipedia.org/wiki/Metodo\\_di\\_Eulero](https://it.wikipedia.org/wiki/Metodo_di_Eulero)
- <sup>4</sup><https://maddmaths.simai.eu/divulgazione/eulero/il-metodo-di-eulero/>
- <sup>5</sup>[https://www.treccani.it/enciclopedia/metodo-iterativo\\_\(Enciclopedia-della-Matematica\)/](https://www.treccani.it/enciclopedia/metodo-iterativo_(Enciclopedia-della-Matematica)/)
- <sup>6</sup><http://wpage.unina.it/p.maffettone/Didattica/Benevento/ODE.pdf>
- <sup>7</sup><https://maddmaths.simai.eu/divulgazione/eulero/il-metodo-di-eulero/>
- <sup>8</sup>[https://it.wikipedia.org/wiki/Metodo\\_di\\_Eulero](https://it.wikipedia.org/wiki/Metodo_di_Eulero)
- <sup>9</sup><https://maddmaths.simai.eu/divulgazione/eulero/il-metodo-di-eulero/>
- <sup>10</sup><http://wpage.unina.it/p.maffettone/Didattica/Benevento/ODE.pdf>
- <sup>11</sup><http://wpage.unina.it/p.maffettone/Didattica/Benevento/ODE.pdf>
- <sup>12</sup><http://wpage.unina.it/p.maffettone/Didattica/Benevento/ODE.pdf>
- <sup>13</sup>[https://it.wikipedia.org/wiki/Metodo\\_di\\_Eulero](https://it.wikipedia.org/wiki/Metodo_di_Eulero)
- <sup>14</sup><http://wpage.unina.it/p.maffettone/Didattica/Benevento/ODE.pdf>
- <sup>15</sup><https://maddmaths.simai.eu/divulgazione/eulero/il-metodo-di-eulero/>
- <sup>16</sup>[https://it.wikipedia.org/wiki/Metodo\\_di\\_Eulero](https://it.wikipedia.org/wiki/Metodo_di_Eulero)
- <sup>17</sup><http://wpage.unina.it/p.maffettone/Didattica/Benevento/ODE.pdf>
- <sup>18</sup><https://maddmaths.simai.eu/divulgazione/eulero/il-metodo-di-eulero/>
- <sup>19</sup><http://wpage.unina.it/p.maffettone/Didattica/Benevento/ODE.pdf>
- <sup>20</sup><https://www.docenti.unina.it/webdocenti-be/allegati/materiale-didattico/451035>
- <sup>21</sup><http://www.mat.uniroma3.it/users/ferretti/corso/node4.html>
- <sup>22</sup><https://tex.unica.it/~gppe/did/ca/tesine/2009/09pous.pdf>
- <sup>23</sup><https://elearning.uniroma1.it/mod/resource/view.php?id=202651>
- <sup>24</sup>[https://it.wikipedia.org/wiki/Metodo\\_di\\_Eulero\\_all'indietro](https://it.wikipedia.org/wiki/Metodo_di_Eulero_all'indietro)
- <sup>25</sup>[http://www.mat.uniroma3.it/users/ferretti/AN2\\_corso\\_3.pdf](http://www.mat.uniroma3.it/users/ferretti/AN2_corso_3.pdf)
- <sup>26</sup><https://people.dm.unipi.it/bini/Didattica/AnaNum/testi/Dispense/iterativi.pdf>
- <sup>27</sup><https://www.youtube.com/watch?v=jiOc8nNjEA8>
- <sup>28</sup>[http://www.dm.unibo.it/~morigi/homepage\\_file/courses\\_file/file\\_dl/SISLIN\\_ITER.pdf](http://www.dm.unibo.it/~morigi/homepage_file/courses_file/file_dl/SISLIN_ITER.pdf)
- <sup>29</sup>[https://it.wikipedia.org/wiki/Metodo\\_iterativo](https://it.wikipedia.org/wiki/Metodo_iterativo)
- <sup>30</sup>[https://bugs.unica.it/~marcoratto/tesina\\_Rodriguez.pdf](https://bugs.unica.it/~marcoratto/tesina_Rodriguez.pdf)
- <sup>31</sup><https://users.dimi.uniud.it/~paolo.baiti/corsi/AA2013-14/EquaDiff/eqdif-registro2013.html>
- <sup>32</sup>[https://mate.unipv.it/gianazza/sistemi\\_lineari.pdf](https://mate.unipv.it/gianazza/sistemi_lineari.pdf)
- <sup>33</sup>[https://mate.unipv.it/gianazza/sistemi\\_lineari.pdf](https://mate.unipv.it/gianazza/sistemi_lineari.pdf)
- <sup>34</sup><https://users.dimi.uniud.it/~paolo.baiti/corsi/AA2013-14/EquaDiff/eqdif-registro2013.html>
- <sup>35</sup>[https://it.wikipedia.org/wiki/Metodo\\_di\\_Eulero](https://it.wikipedia.org/wiki/Metodo_di_Eulero)
- <sup>36</sup><https://maddmaths.simai.eu/divulgazione/eulero/il-metodo-di-eulero/>
- <sup>37</sup><http://wpage.unina.it/p.maffettone/Didattica/Benevento/ODE.pdf>
- <sup>38</sup><https://maddmaths.simai.eu/divulgazione/eulero/il-metodo-di-eulero/>
- <sup>39</sup><https://lucia-gastaldi.unibs.it/didattica2014/civile/lezioni/ode.pdf>
- <sup>40</sup><http://wpage.unina.it/p.maffettone/Didattica/Benevento/ODE.pdf>
- <sup>41</sup>[https://it.wikipedia.org/wiki/Metodo\\_di\\_Eulero](https://it.wikipedia.org/wiki/Metodo_di_Eulero)

<sup>42</sup><http://wpage.unina.it/p.maffettone/Didattica/Benevento/ODE.pdf>

<sup>43</sup><https://lucia-gastaldi.unibs.it/didattica2014/civile/lezioni/ode.pdf>

<sup>44</sup><https://users.dimi.uniud.it/~paolo.baiti/corsi/AA2013-14/EquaDiff/eqdif-registro2013.html>

<sup>45</sup>[https://mate.unipv.it/gianazza/sistemi\\_lineari.pdf](https://mate.unipv.it/gianazza/sistemi_lineari.pdf)

<sup>46</sup>[http://www.mat.uniroma3.it/users/ferretti/AN2\\_corso\\_3.pdf](http://www.mat.uniroma3.it/users/ferretti/AN2_corso_3.pdf)

<sup>47</sup><https://maddmaths.simai.eu/divulgazione/eulero/il-metodo-di-eulero/>

<sup>48</sup>[https://www.math.unipd.it/~alvise/AN/TEORIA/ODE/ode\\_beamer.pdf](https://www.math.unipd.it/~alvise/AN/TEORIA/ODE/ode_beamer.pdf)

<sup>49</sup><https://asana.com/it/resources/iterative-process>

<sup>50</sup><https://docenti-deps.unisi.it/marcolonzi/wp-content/uploads/sites/14/2025/09/DEqDiff.pdf>

<sup>51</sup><https://www.youtube.com/watch?v=TXMORHP9tgU>

<sup>52</sup>[https://www.treccani.it/enciclopedia/metodo-iterativo\\_\(Enciclopedia-della-Matematica\)/](https://www.treccani.it/enciclopedia/metodo-iterativo_(Enciclopedia-della-Matematica)/)

<sup>53</sup>[https://it.wikipedia.org/wiki/Metodo\\_iterativo](https://it.wikipedia.org/wiki/Metodo_iterativo)

<sup>54</sup>[https://my.liuc.it/MatSup/2009/Y90000/N\\_5.doc](https://my.liuc.it/MatSup/2009/Y90000/N_5.doc)

<sup>55</sup><https://twproject.com/blog/it/il-metodo-iterativo-nei-progetti-non-unalternativa-ma-unopportunita/>

<sup>56</sup><https://elearning.uniroma1.it/mod/resource/view.php?id=35795>

<sup>57</sup><https://pagine.dm.unipi.it/ghelardoni/libro/ode.pdf>

<sup>58</sup><https://users.dimi.uniud.it/~paolo.baiti/corsi/AA2013-14/EquaDiff/eqdif-registro2013.html>

<sup>59</sup>[https://mate.unipv.it/gianazza/sistemi\\_lineari.pdf](https://mate.unipv.it/gianazza/sistemi_lineari.pdf)

<sup>60</sup>[https://it.wikipedia.org/wiki/Metodo\\_di\\_Eulero](https://it.wikipedia.org/wiki/Metodo_di_Eulero)

<sup>61</sup>[https://www.dmf.unisalento.it/~spagnolo/MSC\\_aa21\\_22/EquazioniDifferenziali/Lezione19-20\\_metodiSC.pdf](https://www.dmf.unisalento.it/~spagnolo/MSC_aa21_22/EquazioniDifferenziali/Lezione19-20_metodiSC.pdf)

<sup>62</sup><https://pagine.dm.unipi.it/ghelardoni/libro/ode.pdf>

<sup>63</sup>[https://mate.unipv.it/gianazza/sistemi\\_lineari.pdf](https://mate.unipv.it/gianazza/sistemi_lineari.pdf)

<sup>64</sup><https://users.dimi.uniud.it/~paolo.baiti/corsi/AA2013-14/EquaDiff/eqdif-registro2013.html>

<sup>65</sup><https://elearning.uniroma1.it/mod/resource/view.php?id=35795>

<sup>66</sup><https://elearning.uniroma1.it/mod/resource/view.php?id=35795>

<sup>67</sup>[https://www.le.infn.it/~martello/home/images/didattica/Computazionale/EquDifferenziali\\_1\\_1415.pdf](https://www.le.infn.it/~martello/home/images/didattica/Computazionale/EquDifferenziali_1_1415.pdf)

<sup>68</sup><https://users.dimi.uniud.it/~paolo.baiti/corsi/AA2013-14/EquaDiff/eqdif-registro2013.html>

<sup>69</sup>[https://mate.unipv.it/gianazza/sistemi\\_lineari.pdf](https://mate.unipv.it/gianazza/sistemi_lineari.pdf)

<sup>70</sup>[https://www.dmf.unisalento.it/~spagnolo/MSC\\_aa21\\_22/EquazioniDifferenziali/Lezione19-20\\_metodiSC.pdf](https://www.dmf.unisalento.it/~spagnolo/MSC_aa21_22/EquazioniDifferenziali/Lezione19-20_metodiSC.pdf)

<sup>71</sup>[https://www.matematica.it/impedovo/articoli/Algoritmo\\_di\\_Eulero.pdf](https://www.matematica.it/impedovo/articoli/Algoritmo_di_Eulero.pdf)

<sup>72</sup><https://pagine.dm.unipi.it/ghelardoni/libro/ode.pdf>

<sup>73</sup><https://elearning.uniroma1.it/mod/resource/view.php?id=35795>

<sup>74</sup><https://www.youtube.com/watch?v=TXMORHP9tgU>

<sup>75</sup><https://paola-gervasio.unibs.it/CS/Slides/eqdiff1.pdf>

<sup>76</sup>[https://it.wikipedia.org/wiki/Equazione\\_differenziale\\_esatta](https://it.wikipedia.org/wiki/Equazione_differenziale_esatta)

<sup>77</sup>[https://alonso.maths.unitn.it/didattica/CN07\\_08/ODEonestep.pdf](https://alonso.maths.unitn.it/didattica/CN07_08/ODEonestep.pdf)

<sup>78</sup>[http://lia.deis.unibo.it/Courses/InfoChim0910/lucidi/4-integrazioneDifferenziali\(1x\).pdf](http://lia.deis.unibo.it/Courses/InfoChim0910/lucidi/4-integrazioneDifferenziali(1x).pdf)

<sup>79</sup><https://www.youtube.com/watch?v=lZrmY914mqc>

<sup>80</sup>[https://www.math.unipd.it/~alvise/AN/TEORIA/ODE/ode\\_beamer.pdf](https://www.math.unipd.it/~alvise/AN/TEORIA/ODE/ode_beamer.pdf)

<sup>81</sup><https://www.youtube.com/watch?v=lh0W46CiosA>

<sup>82</sup><https://digilander.libero.it/b.dellavecchia/documents/dispensemnedo.pdf>

<sup>83</sup><https://www.dmi.unict.it/rosa/ode.pdf>

<sup>84</sup><https://maddmaths.simai.eu/divulgazione/eulero/il-metodo-di-eulero/>

<sup>85</sup><https://pagine.dm.unipi.it/ghelardoni/libro/ode.pdf>

<sup>86</sup><http://wpage.unina.it/p.maffettone/Didattica/Benevento/ODE.pdf>



<sup>87</sup><https://www.matematica.unito.it/didattica/att/b432.0238.file.pdf>

<sup>88</sup><https://users.dimi.uniud.it/~paolo.baiti/corsi/AA2013-14/EquaDiff/eqdif-registro2013.html>

<sup>89</sup>[https://mate.unipv.it/gianazza/sistemi\\_lineari.pdf](https://mate.unipv.it/gianazza/sistemi_lineari.pdf)

<sup>90</sup><https://pagine.dm.unipi.it/ghelardoni/libro/ode.pdf>

<sup>91</sup><https://maddmaths.simai.eu/divulgazione/eulero/il-metodo-di-eulero/>

<sup>92</sup><https://www.youtube.com/watch?v=TXMORHP9tgU>

<sup>93</sup><https://www.mautonedavide.it/metodi-numericiperodes/>

<sup>94</sup>[https://my.liuc.it/MatSup/2009/Y90000/N\\_5.doc](https://my.liuc.it/MatSup/2009/Y90000/N_5.doc)

<sup>95</sup><https://paola-gervasio.unibs.it/CS/Slides/eqdiff1.pdf>

<sup>96</sup><https://www0.mi.infn.it/~palombo/didattica/Lab-TNDS/CorsoLab/LezioniFrontali/Lezione7-EquazioniDiffOrd.pdf>

<sup>97</sup>[https://archimede.uniba.it/~amodio/lab\\_prog\\_calc/dispense/eqdiff\\_2005.pdf](https://archimede.uniba.it/~amodio/lab_prog_calc/dispense/eqdiff_2005.pdf)

<sup>98</sup><https://digilander.libero.it/b.dellavecchia/documents/dispensemnedo.pdf>

<sup>99</sup>[https://www.math.unipd.it/~alvise/CS\\_2011/2011/ODE\\_2011\\_PDF/ode\\_2011.pdf](https://www.math.unipd.it/~alvise/CS_2011/2011/ODE_2011_PDF/ode_2011.pdf)

<sup>100</sup>[http://officinadegliappunti.altervista.org/ingegneria/base/calcolo\\_numerico/settimana08\\_maxwell.pdf](http://officinadegliappunti.altervista.org/ingegneria/base/calcolo_numerico/settimana08_maxwell.pdf)

<sup>101</sup>[https://it.wikipedia.org/wiki/Metodo\\_di\\_Eulero](https://it.wikipedia.org/wiki/Metodo_di_Eulero)

<sup>102</sup><https://people.dm.unipi.it/meini/ODE12/lab1.html>

<sup>103</sup>[https://it.wikipedia.org/wiki/Metodo\\_di\\_Eulero](https://it.wikipedia.org/wiki/Metodo_di_Eulero)

<sup>104</sup><http://wpage.unina.it/p.maffettone/Didattica/Benevento/ODE.pdf>

<sup>105</sup><https://www.dmi.unict.it/rosa/ode.pdf>

<sup>106</sup><http://wpage.unina.it/p.maffettone/Didattica/Benevento/ODE.pdf>

<sup>107</sup>[https://it.wikipedia.org/wiki/Metodo\\_di\\_Eulero](https://it.wikipedia.org/wiki/Metodo_di_Eulero)

<sup>108</sup>[https://it.wikipedia.org/wiki/Problema\\_di\\_Cauchy](https://it.wikipedia.org/wiki/Problema_di_Cauchy)

<sup>109</sup>[https://www.dmi.unict.it/moschetti/farmacia/equazioni\\_differenziali.pdf](https://www.dmi.unict.it/moschetti/farmacia/equazioni_differenziali.pdf)

<sup>110</sup>[https://www.dmi.unict.it/moschetti/farmacia/equazioni\\_differenziali.pdf](https://www.dmi.unict.it/moschetti/farmacia/equazioni_differenziali.pdf)

<sup>111</sup>[https://it.wikipedia.org/wiki/Problema\\_di\\_Cauchy](https://it.wikipedia.org/wiki/Problema_di_Cauchy)

<sup>112</sup>[https://www.dmi.unict.it/moschetti/farmacia/equazioni\\_differenziali.pdf](https://www.dmi.unict.it/moschetti/farmacia/equazioni_differenziali.pdf)

<sup>113</sup><https://www.youtube.com/watch?v=Ke5Wlfl3wGo>

<sup>114</sup>[http://www.mat.unimi.it/users/mauras/appunti\\_AA03-04/sez6.pdf](http://www.mat.unimi.it/users/mauras/appunti_AA03-04/sez6.pdf)

<sup>115</sup>[https://www.math.unipd.it/~alvise/AN\\_2017/PDF/ODE\\_2017\\_PDF/ode\\_2017\\_PDF.pdf](https://www.math.unipd.it/~alvise/AN_2017/PDF/ODE_2017_PDF/ode_2017_PDF.pdf)

<sup>116</sup><https://www.alfacod.it/guida-prodotti-differenza-codici-1d-2d>

<sup>117</sup><https://www.sbai.uniroma1.it/~micol.amar/tutorato8.pdf>

<sup>118</sup>[https://it.wikipedia.org/wiki/Metodo\\_di\\_Eulero](https://it.wikipedia.org/wiki/Metodo_di_Eulero)

<sup>119</sup>[https://alonso.maths.unitn.it/didattica/CN07\\_08/ODEonestep.pdf](https://alonso.maths.unitn.it/didattica/CN07_08/ODEonestep.pdf)

<sup>120</sup><https://scholarlypublications.universiteitleiden.nl/access/item:3483802/download>

<sup>121</sup>[https://hpc-forge.cineca.it/files/CoursesDev/public/2014/Introduction\\_to\\_Scientific\\_and\\_Technical\\_Computing\\_in\\_C/Milan/Librerie\\_standard](https://hpc-forge.cineca.it/files/CoursesDev/public/2014/Introduction_to_Scientific_and_Technical_Computing_in_C/Milan/Librerie_standard)

<sup>122</sup><http://labmaster.mi.infn.it/Laboratorio2/Lezione6-2008/>

<sup>123</sup>[https://en.wikipedia.org/wiki/Vector\\_space](https://en.wikipedia.org/wiki/Vector_space)

<sup>124</sup><https://www.youtube.com/watch?v=trKaQ4okScw>

<sup>125</sup>[https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo\\_numerico/eq\\_differenziali.pdf](https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo_numerico/eq_differenziali.pdf)

<sup>126</sup><https://www.ibm.com/docs/it/i/7.5.0?topic=extensions-standard-c-library-functions-table-by-name>

<sup>127</sup>[https://it.wikipedia.org/wiki/Libreria\\_standard\\_del\\_C](https://it.wikipedia.org/wiki/Libreria_standard_del_C)

<sup>128</sup>[https://www.reddit.com/r/C\\_Programming/comments/1n7d0gb/where\\_can\\_i\\_learn\\_other\\_libraries\\_of\\_c/](https://www.reddit.com/r/C_Programming/comments/1n7d0gb/where_can_i_learn_other_libraries_of_c/)

<sup>129</sup><https://www.dpss.inesc-id.pt/~romanop/files/FI/Funzioni.pdf>

<sup>130</sup><http://www-old.bo.cnr.it/corsi-di-informatica/corsoCstandard/Lezioni/34Standard.html>

<sup>131</sup><https://www.sciencedirect.com/topics/engineering/dimensional-vector>

<sup>132</sup>[http://lia.deis.unibo.it/Courses/FondA0708-INF/lucidi/15-stdio\(2x\).pdf](http://lia.deis.unibo.it/Courses/FondA0708-INF/lucidi/15-stdio(2x).pdf)

<sup>133</sup>[https://it.wikipedia.org/wiki/Metodo\\_di\\_Eulero](https://it.wikipedia.org/wiki/Metodo_di_Eulero)

<sup>134</sup><http://wpage.unina.it/p.maffettone/Didattica/Benevento/ODE.pdf>

<sup>135</sup>[https://alonso.maths.unitn.it/didattica/CN07\\_08/ODEonestep.pdf](https://alonso.maths.unitn.it/didattica/CN07_08/ODEonestep.pdf)

<sup>136</sup>[https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo\\_numerico/eq\\_differenziali.pdf](https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo_numerico/eq_differenziali.pdf)

<sup>137</sup><https://lucia-gastaldi.unibs.it/didattica2007/automazione/lezioni/ode.pdf>

<sup>138</sup><http://localwww.math.unipd.it/~demarchi/CNAstro1819/MNED.pdf>

<sup>139</sup>[https://www.treccani.it/enciclopedia/metodo-di-heun\\_\(Enciclopedia-della-Matematica\)/](https://www.treccani.it/enciclopedia/metodo-di-heun_(Enciclopedia-della-Matematica)/)

<sup>140</sup>[https://alonso.maths.unitn.it/didattica/CNCivile10\\_11/ODEs.pdf](https://alonso.maths.unitn.it/didattica/CNCivile10_11/ODEs.pdf)

<sup>141</sup><https://www0.mi.infn.it/~palombo/didattica/Lab-TNDS/CorsoLab/LezioniFrontali/Lezione7-EquazioniDiffOrd.pdf>

<sup>142</sup>[https://www.treccani.it/enciclopedia/metodo-di-heun\\_\(Enciclopedia-della-Matematica\)/](https://www.treccani.it/enciclopedia/metodo-di-heun_(Enciclopedia-della-Matematica)/)

<sup>143</sup>[https://www.sbai.uniroma1.it/sites/default/files/Lez1112\\_AA19-20\\_0.pdf](https://www.sbai.uniroma1.it/sites/default/files/Lez1112_AA19-20_0.pdf)

<sup>144</sup><https://docenti.ing.unipi.it/forgione-n/Teoria4.pdf>

<sup>145</sup><https://www.youtube.com/watch?v=TXMORHP9tgU>

<sup>146</sup><https://www.emathhelp.net/it/calculators/differential-equations/improved-euler-heun-calculator/>

<sup>147</sup><https://tex.unica.it/~gppe/did/ca/tesine/2009/09pous.pdf>

<sup>148</sup>[https://www.reddit.com/r/CasualMath/comments/o110mg/is\\_heuns\\_method\\_always\\_better\\_than\\_eulers/](https://www.reddit.com/r/CasualMath/comments/o110mg/is_heuns_method_always_better_than_eulers/)

<sup>149</sup><https://tex.unica.it/~gppe/did/ca/tesine/2005/05pili.pdf>

<sup>150</sup>[http://officinadegliappunti.altervista.org/ingegneria/base/calcolo\\_numerico/settimana08\\_maxwell.pdf](http://officinadegliappunti.altervista.org/ingegneria/base/calcolo_numerico/settimana08_maxwell.pdf)

<sup>151</sup>[https://it.wikipedia.org/wiki/Metodi\\_di\\_soluzione\\_numerica\\_per\\_equazioni\\_differenziali\\_ordinarie](https://it.wikipedia.org/wiki/Metodi_di_soluzione_numerica_per_equazioni_differenziali_ordinarie)

<sup>152</sup>[http://www.mat.unimi.it/users/scacchi/didattica\\_2015/calc\\_num\\_chim/lab11\\_2515\\_bis.pdf](http://www.mat.unimi.it/users/scacchi/didattica_2015/calc_num_chim/lab11_2515_bis.pdf)

<sup>153</sup>[https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo\\_numerico/eq\\_differenziali.pdf](https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo_numerico/eq_differenziali.pdf)

<sup>154</sup><https://www0.mi.infn.it/~palombo/didattica/Lab-TNDS/CorsoLab/LezioniFrontali/Lezione4-QuadraturaNumerica.pdf>

<sup>155</sup><https://paola-gervasio.unibs.it/CS/Slides/eqdiff2.pdf>

<sup>156</sup><https://pagine.dm.unipi.it/ghelardoni/libro/ode.pdf>

<sup>157</sup>[https://www.sbai.uniroma1.it/sites/default/files/Lez1112\\_AA19-20\\_0.pdf](https://www.sbai.uniroma1.it/sites/default/files/Lez1112_AA19-20_0.pdf)

<sup>158</sup>[https://www.dm.unibo.it/~montelau/html/Equazioni\\_differenziali.pdf](https://www.dm.unibo.it/~montelau/html/Equazioni_differenziali.pdf)

<sup>159</sup>[https://it.wikipedia.org/wiki/Metodi\\_di\\_soluzione\\_numerica\\_per\\_equazioni\\_differenziali\\_ordinarie](https://it.wikipedia.org/wiki/Metodi_di_soluzione_numerica_per_equazioni_differenziali_ordinarie)

<sup>160</sup>[https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo\\_numerico/eq\\_differenziali.pdf](https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo_numerico/eq_differenziali.pdf)

<sup>161</sup><https://www0.mi.infn.it/~palombo/didattica/Lab-TNDS/CorsoLab/LezioniFrontali/Lezione4-QuadraturaNumerica.pdf>

<sup>162</sup>[https://www.sbai.uniroma1.it/sites/default/files/Lez1112\\_AA19-20\\_0.pdf](https://www.sbai.uniroma1.it/sites/default/files/Lez1112_AA19-20_0.pdf)

<sup>163</sup><https://paola-gervasio.unibs.it/CS/Slides/eqdiff2.pdf>

<sup>164</sup><https://tex.unica.it/~gppe/did/ca/tesine/2008/08manca.pdf>

<sup>165</sup><https://tex.unica.it/~gppe/did/ca/tesine/2009/09pous.pdf>

<sup>166</sup>[https://alonso.maths.unitn.it/didattica/CNCivile10\\_11/ODEs.pdf](https://alonso.maths.unitn.it/didattica/CNCivile10_11/ODEs.pdf)

<sup>167</sup>[https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo\\_numerico/interpolazione.pdf](https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo_numerico/interpolazione.pdf)

<sup>168</sup><https://tex.unica.it/~gppe/did/ca/tesine/2005/05pili.pdf>

<sup>169</sup><http://www.mat.uniroma3.it/users/ferretti/corso/node7.html>

<sup>170</sup>[https://my.liuc.it/MatSup/2009/Y90000/N\\_5.doc](https://my.liuc.it/MatSup/2009/Y90000/N_5.doc)

<sup>171</sup><https://www0.mi.infn.it/~palombo/didattica/Lab-TNDS/CorsoLab/LezioniFrontali/Lezione7-EquazioniDiffOrd.pdf>

<sup>172</sup>[https://www.dmf.unisalento.it/~spagnolo/MSC\\_aa21\\_22/EquazioniDifferenziali/Lezione19-20\\_metodiSC.pdf](https://www.dmf.unisalento.it/~spagnolo/MSC_aa21_22/EquazioniDifferenziali/Lezione19-20_metodiSC.pdf)

<sup>173</sup>[https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo\\_numerico/eq\\_differenziali.pdf](https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo_numerico/eq_differenziali.pdf)

<sup>174</sup>[https://www.dmf.unisalento.it/~spagnolo/MSC\\_aa21\\_22/EquazioniDifferenziali/Lezione19-20\\_metodiSC.pdf](https://www.dmf.unisalento.it/~spagnolo/MSC_aa21_22/EquazioniDifferenziali/Lezione19-20_metodiSC.pdf)

<sup>175</sup><https://elearning.uniroma1.it/mod/resource/view.php?id=35795>

<sup>176</sup><https://www0.mi.infn.it/~palombo/didattica/Lab-TNDS/CorsoLab/LezioniFrontali/Lezione4-QuadraturaNumerica.pdf>

<sup>177</sup>[https://www.dmf.unisalento.it/~spagnolo/MSC\\_aa21\\_22/EquazioniDifferenziali/Lezione19-20\\_metodiSC.pdf](https://www.dmf.unisalento.it/~spagnolo/MSC_aa21_22/EquazioniDifferenziali/Lezione19-20_metodiSC.pdf)  
<sup>178</sup>[https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo\\_numerico/eq\\_differenziali.pdf](https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo_numerico/eq_differenziali.pdf)  
<sup>179</sup>[https://www.dmf.unisalento.it/~spagnolo/MSC\\_aa21\\_22/EquazioniDifferenziali/Lezione19-20\\_metodiSC.pdf](https://www.dmf.unisalento.it/~spagnolo/MSC_aa21_22/EquazioniDifferenziali/Lezione19-20_metodiSC.pdf)  
<sup>180</sup>[https://www.sbai.uniroma1.it/sites/default/files/Lez1112\\_AA19-20\\_0.pdf](https://www.sbai.uniroma1.it/sites/default/files/Lez1112_AA19-20_0.pdf)  
<sup>181</sup><https://paola-gervasio.unibs.it/CS/Slides/eqdiff2.pdf>  
<sup>182</sup>[https://it.wikipedia.org/wiki/Interpolazione\\_polinomiale](https://it.wikipedia.org/wiki/Interpolazione_polinomiale)  
<sup>183</sup>[https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo\\_numerico/1516/eq\\_differenziali.pdf](https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo_numerico/1516/eq_differenziali.pdf)  
<sup>184</sup>[https://www.reddit.com/r/math/comments/10mix5i/global\\_truncation\\_error\\_in\\_eulers\\_method/](https://www.reddit.com/r/math/comments/10mix5i/global_truncation_error_in_eulers_method/)  
<sup>185</sup><http://www.peliti.org/Notes/eqdiff.pdf>  
<sup>186</sup><https://lucia-gastaldi.unibs.it/did17-18/automazione/slides/interpolazione.pdf>  
<sup>187</sup><https://pagine.dm.unipi.it/ghelardoni/libro/ode.pdf>  
<sup>188</sup><https://tex.unica.it/~gppe/did/ca/tesine/2005/05pili.pdf>  
<sup>189</sup><https://www0.mi.infn.it/~palombo/didattica/Lab-TNDS/CorsoLab/LezioniFrontali/Lezione7-EquazioniDiffOrd.pdf>  
<sup>190</sup>[https://my.liuc.it/MatSup/2009/Y90000/N\\_5.doc](https://my.liuc.it/MatSup/2009/Y90000/N_5.doc)  
<sup>191</sup>[https://it.wikipedia.org/wiki/Metodo\\_di\\_Eulero](https://it.wikipedia.org/wiki/Metodo_di_Eulero)  
<sup>192</sup>[https://www.sbai.uniroma1.it/sites/default/files/Lez1112\\_AA19-20\\_0.pdf](https://www.sbai.uniroma1.it/sites/default/files/Lez1112_AA19-20_0.pdf)  
<sup>193</sup>[https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo\\_numerico/eq\\_differenziali.pdf](https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo_numerico/eq_differenziali.pdf)  
<sup>194</sup><http://www.peliti.org/Notes/eqdiff.pdf>  
<sup>195</sup>[https://sites.unimi.it/zampieri/info\\_MI/info\\_MI\\_aaOLD/ode.pdf](https://sites.unimi.it/zampieri/info_MI/info_MI_aaOLD/ode.pdf)  
<sup>196</sup><https://www0.mi.infn.it/~palombo/didattica/Lab-TNDS/CorsoLab/LezioniFrontali/Lezione7-EquazioniDiffOrd.pdf>  
<sup>197</sup>[https://poisson.phc.dm.unipi.it/~mele/appunti/Metodi\\_numerici\\_per\\_equazioni%20differenziali\\_ordinarie/dispense.pdf](https://poisson.phc.dm.unipi.it/~mele/appunti/Metodi_numerici_per_equazioni%20differenziali_ordinarie/dispense.pdf)  
<sup>198</sup>[https://www.dmf.unisalento.it/~spagnolo/MSC\\_aa21\\_22/EquazioniDifferenziali/Lezione19-20\\_metodiSC.pdf](https://www.dmf.unisalento.it/~spagnolo/MSC_aa21_22/EquazioniDifferenziali/Lezione19-20_metodiSC.pdf)  
<sup>199</sup><https://elearning.uniroma1.it/mod/resource/view.php?id=35795>  
<sup>200</sup>[https://poisson.phc.dm.unipi.it/~mele/appunti/Metodi\\_numerici\\_per\\_equazioni%20differenziali\\_ordinarie/dispense.pdf](https://poisson.phc.dm.unipi.it/~mele/appunti/Metodi_numerici_per_equazioni%20differenziali_ordinarie/dispense.pdf)  
<sup>201</sup><https://paola-gervasio.unibs.it/CS/Slides/eqdiff2.pdf>  
<sup>202</sup>[https://www.sbai.uniroma1.it/sites/default/files/Lez1112\\_AA19-20\\_0.pdf](https://www.sbai.uniroma1.it/sites/default/files/Lez1112_AA19-20_0.pdf)  
<sup>203</sup>[https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo\\_numerico/eq\\_differenziali.pdf](https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo_numerico/eq_differenziali.pdf)  
<sup>204</sup>[https://www.dm.unibo.it/~montelau/html/Equazioni\\_differenziali.pdf](https://www.dm.unibo.it/~montelau/html/Equazioni_differenziali.pdf)  
<sup>205</sup><https://tex.unica.it/~gppe/did/ca/tesine/2009/09pous.pdf>  
<sup>206</sup>[https://alonso.maths.unitn.it/didattica/CN07\\_08/ODEonestep.pdf](https://alonso.maths.unitn.it/didattica/CN07_08/ODEonestep.pdf)  
<sup>207</sup><https://www.mautonedavide.it/metodi-numerici-per-odes/>  
<sup>208</sup>[https://my.liuc.it/MatSup/2009/Y90000/N\\_5.doc](https://my.liuc.it/MatSup/2009/Y90000/N_5.doc)  
<sup>209</sup><https://lucia-gastaldi.unibs.it/didattica2007/automazione/lezioni/ode.pdf>  
<sup>210</sup>[https://it.wikipedia.org/wiki/Metodi\\_di\\_soluzione\\_numerica\\_per\\_equazioni\\_differenziali\\_ordinarie](https://it.wikipedia.org/wiki/Metodi_di_soluzione_numerica_per_equazioni_differenziali_ordinarie)  
<sup>211</sup>[http://www.mat.unimi.it/users/scacchi/didattica\\_2013/calc\\_num\\_chim/lab11\\_27513.pdf](http://www.mat.unimi.it/users/scacchi/didattica_2013/calc_num_chim/lab11_27513.pdf)  
<sup>212</sup>[https://www.youtube.com/watch?v=V\\_nC8aumr4Q](https://www.youtube.com/watch?v=V_nC8aumr4Q)  
<sup>213</sup>[https://www.sbai.uniroma1.it/sites/default/files/Lez1112\\_AA19-20\\_0.pdf](https://www.sbai.uniroma1.it/sites/default/files/Lez1112_AA19-20_0.pdf)  
<sup>214</sup>[https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo\\_numerico/eq\\_differenziali.pdf](https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo_numerico/eq_differenziali.pdf)  
<sup>215</sup><https://www.geeksforgeeks.org/cpp/how-to-initialize-an-array-in-cpp/>  
<sup>216</sup><https://www.geeksforgeeks.org/cpp/initialize-a-vector-in-cpp-different-ways/>  
<sup>217</sup><https://stackoverflow.com/questions/47690386/initializing-array-of-stdvector-in-constructor-init-list>  
<sup>218</sup>[https://www.reddit.com/r/C\\_Programming/comments/xr2gzy/how\\_to\\_initialise\\_all\\_elements\\_of\\_an\\_array\\_to\\_the/](https://www.reddit.com/r/C_Programming/comments/xr2gzy/how_to_initialise_all_elements_of_an_array_to_the/)  
<sup>219</sup><https://en.cppreference.com/cpp/container/array>  
<sup>220</sup><https://www.freecodecamp.org/news/cpp-vector-how-to-initialize-a-vector-in-a-constructor/>  
<sup>221</sup><https://fr.scribd.com/presentation/781492241/3-Arrays-1-1>

<sup>222</sup><https://www.geeksforgeeks.org/cpp/stdarray-in-cpp/>  
<sup>223</sup><https://stackoverflow.com/questions/72410967/how-to-store-a-sequence-of-integers-in-a-c-array>  
<sup>224</sup><https://documents.uow.edu.au/~lukes/textbook/notes-cpp/arrayptr/array-initialization.html>  
<sup>225</sup>[https://en.cppreference.com/c/language/array\\_initialization](https://en.cppreference.com/c/language/array_initialization)  
<sup>226</sup><https://www.programiz.com/cpp-programming/std-array>  
<sup>227</sup><https://cplusplus.com/doc/tutorial/arrays/>  
<sup>228</sup><https://stackoverflow.com/questions/8863319/stdarrayt-initialization>  
<sup>229</sup><https://documents.uow.edu.au/~lukes/textbook/notes-cpp/arrayptr/array-initialization.html>  
<sup>230</sup><https://www.geeksforgeeks.org/cpp/how-to-initialize-an-array-in-cpp/>  
<sup>231</sup>[https://it.wikipedia.org/wiki/Metodi\\_di\\_soluzione\\_numerica\\_per\\_equazioni\\_differenziali\\_ordinarie](https://it.wikipedia.org/wiki/Metodi_di_soluzione_numerica_per_equazioni_differenziali_ordinarie)  
<sup>232</sup><https://lucia-gastaldi.unibs.it/didattica2007/automazione/lezioni/ode.pdf>  
<sup>233</sup>[https://it.wikipedia.org/wiki/Metodo\\_di\\_Eulero](https://it.wikipedia.org/wiki/Metodo_di_Eulero)  
<sup>234</sup>[https://www.treccani.it/enciclopedia/metodo-di-heun\\_\(Enciclopedia-della-Matematica\)/](https://www.treccani.it/enciclopedia/metodo-di-heun_(Enciclopedia-della-Matematica)/)  
<sup>235</sup>[https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo\\_numerico/eq\\_differenziali.pdf](https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo_numerico/eq_differenziali.pdf)  
<sup>236</sup>[https://www.dm.unibo.it/~montelau/html/Equazioni\\_differenziali.pdf](https://www.dm.unibo.it/~montelau/html/Equazioni_differenziali.pdf)  
<sup>237</sup>[https://www.sbai.uniroma1.it/sites/default/files/Lez1112\\_AA19-20\\_0.pdf](https://www.sbai.uniroma1.it/sites/default/files/Lez1112_AA19-20_0.pdf)  
<sup>238</sup>[https://poisson.phc.dm.unipi.it/~mele/appunti/Metodi\\_numerici\\_per\\_equazioni%20differenziali\\_ordinarie/dispense.pdf](https://poisson.phc.dm.unipi.it/~mele/appunti/Metodi_numerici_per_equazioni%20differenziali_ordinarie/dispense.pdf)  
<sup>239</sup>[https://it.wikipedia.org/wiki/Metodi\\_di\\_soluzione\\_numerica\\_per\\_equazioni\\_differenziali\\_ordinarie](https://it.wikipedia.org/wiki/Metodi_di_soluzione_numerica_per_equazioni_differenziali_ordinarie)  
<sup>240</sup><https://lucia-gastaldi.unibs.it/didattica2007/automazione/lezioni/ode.pdf>  
<sup>241</sup>[https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo\\_numerico/eq\\_differenziali.pdf](https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo_numerico/eq_differenziali.pdf)  
<sup>242</sup><https://www.mautonedavide.it/metodi-numerici-per-odes/>  
<sup>243</sup><https://www0.mi.infn.it/~palombo/didattica/Lab-TNDS/CorsoLab/LezioniFrontali/Lezione4-QuadraturaNumerica.pdf>  
<sup>244</sup><https://elearning.uniroma1.it/mod/resource/view.php?id=35795>  
<sup>245</sup><https://www0.mi.infn.it/~palombo/didattica/Lab-TNDS/CorsoLab/LezioniFrontali/Lezione4-QuadraturaNumerica.pdf>  
<sup>246</sup><https://elearning.uniroma1.it/mod/resource/view.php?id=35795>  
<sup>247</sup><https://www0.mi.infn.it/~palombo/didattica/Lab-TNDS/CorsoLab/LezioniFrontali/Lezione4-QuadraturaNumerica.pdf>  
<sup>248</sup><https://elearning.uniroma1.it/mod/resource/view.php?id=35795>  
<sup>249</sup><https://www0.mi.infn.it/~palombo/didattica/Lab-TNDS/CorsoLab/LezioniFrontali/Lezione4-QuadraturaNumerica.pdf>  
<sup>250</sup>[https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo\\_numerico/eq\\_differenziali.pdf](https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo_numerico/eq_differenziali.pdf)  
<sup>251</sup><https://lucia-gastaldi.unibs.it/didattica2007/automazione/lezioni/ode.pdf>  
<sup>252</sup>[https://www.sbai.uniroma1.it/sites/default/files/Lez1112\\_AA19-20\\_0.pdf](https://www.sbai.uniroma1.it/sites/default/files/Lez1112_AA19-20_0.pdf)  
<sup>253</sup>[https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo\\_numerico/eq\\_differenziali.pdf](https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo_numerico/eq_differenziali.pdf)  
<sup>254</sup>[https://www.dmf.unisalento.it/~spagnolo/MSC\\_aa21\\_22/EquazioniDifferenziali/Lezione19-20\\_metodiSC.pdf](https://www.dmf.unisalento.it/~spagnolo/MSC_aa21_22/EquazioniDifferenziali/Lezione19-20_metodiSC.pdf)  
<sup>255</sup><http://www.peliti.org/Notes/eqdiff.pdf>  
<sup>256</sup>[https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo\\_numerico/eq\\_differenziali.pdf](https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo_numerico/eq_differenziali.pdf)  
<sup>257</sup>[https://www.sbai.uniroma1.it/sites/default/files/Lez1112\\_AA19-20\\_0.pdf](https://www.sbai.uniroma1.it/sites/default/files/Lez1112_AA19-20_0.pdf)  
<sup>258</sup>[https://www.dmf.unisalento.it/~spagnolo/MSC\\_aa21\\_22/EquazioniDifferenziali/Lezione19-20\\_metodiSC.pdf](https://www.dmf.unisalento.it/~spagnolo/MSC_aa21_22/EquazioniDifferenziali/Lezione19-20_metodiSC.pdf)  
<sup>259</sup>[https://sites.unimi.it/zampieri/info\\_MI/info\\_MI\\_aaOLD/ode.pdf](https://sites.unimi.it/zampieri/info_MI/info_MI_aaOLD/ode.pdf)  
<sup>260</sup><http://www.peliti.org/Notes/eqdiff.pdf>  
<sup>261</sup>[https://www.dmf.unisalento.it/~spagnolo/MSC\\_aa21\\_22/EquazioniDifferenziali/Lezione19-20\\_metodiSC.pdf](https://www.dmf.unisalento.it/~spagnolo/MSC_aa21_22/EquazioniDifferenziali/Lezione19-20_metodiSC.pdf)  
<sup>262</sup><https://elearning.uniroma1.it/mod/resource/view.php?id=35795>  
<sup>263</sup><https://www0.mi.infn.it/~palombo/didattica/Lab-TNDS/CorsoLab/LezioniFrontali/Lezione4-QuadraturaNumerica.pdf>  
<sup>264</sup><https://www0.mi.infn.it/~palombo/didattica/Lab-TNDS/CorsoLab/LezioniFrontali/Lezione4-QuadraturaNumerica.pdf>  
<sup>265</sup>[https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo\\_numerico/eq\\_differenziali.pdf](https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo_numerico/eq_differenziali.pdf)  
<sup>266</sup><https://elearning.uniroma1.it/mod/resource/view.php?id=35795>

<sup>267</sup><https://elearning.uniroma1.it/mod/resource/view.php?id=35795>  
<sup>268</sup><https://www0.mi.infn.it/~palombo/didattica/Lab-TNDS/CorsoLab/LezioniFrontali/Lezione4-QuadraturaNumerica.pdf>  
<sup>269</sup>[https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo\\_numerico/eq\\_differenziali.pdf](https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo_numerico/eq_differenziali.pdf)  
<sup>270</sup><https://elearning.uniroma1.it/mod/resource/view.php?id=35795>  
<sup>271</sup>[https://www.dmf.unisalento.it/~spagnolo/MSC\\_aa21\\_22/EquazioniDifferenziali/Lezione19-20\\_metodiSC.pdf](https://www.dmf.unisalento.it/~spagnolo/MSC_aa21_22/EquazioniDifferenziali/Lezione19-20_metodiSC.pdf)  
<sup>272</sup><http://www.peliti.org/Notes/eqdiff.pdf>  
<sup>273</sup>[https://sites.unimi.it/zampieri/info\\_MI/info\\_MI\\_aaOLD/ode.pdf](https://sites.unimi.it/zampieri/info_MI/info_MI_aaOLD/ode.pdf)  
<sup>274</sup><https://elearning.uniroma1.it/mod/resource/view.php?id=35795>  
<sup>275</sup>[https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo\\_numerico/eq\\_differenziali.pdf](https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo_numerico/eq_differenziali.pdf)  
<sup>276</sup>[https://www.sbai.uniroma1.it/sites/default/files/Lez1112\\_AA19-20\\_0.pdf](https://www.sbai.uniroma1.it/sites/default/files/Lez1112_AA19-20_0.pdf)  
<sup>277</sup><https://www0.mi.infn.it/~palombo/didattica/Lab-TNDS/CorsoLab/LezioniFrontali/Lezione7-EquazioniDiffOrd.pdf>  
<sup>278</sup><http://www.peliti.org/Notes/eqdiff.pdf>  
<sup>279</sup>[https://www.dmf.unisalento.it/~spagnolo/MSC\\_aa21\\_22/EquazioniDifferenziali/Lezione19-20\\_metodiSC.pdf](https://www.dmf.unisalento.it/~spagnolo/MSC_aa21_22/EquazioniDifferenziali/Lezione19-20_metodiSC.pdf)  
<sup>280</sup><https://paola-gervasio.unibs.it/CS/Slides/eqdiff2.pdf>  
<sup>281</sup>image.jpg  
<sup>282</sup>[https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo\\_numerico/1516/eq\\_differenziali.pdf](https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo_numerico/1516/eq_differenziali.pdf)  
<sup>283</sup>[https://www.sbai.uniroma1.it/sites/default/files/Lez1112\\_AA19-20\\_0.pdf](https://www.sbai.uniroma1.it/sites/default/files/Lez1112_AA19-20_0.pdf)  
<sup>284</sup>[https://www.dm.unibo.it/~montelau/html/Equazioni\\_differenziali.pdf](https://www.dm.unibo.it/~montelau/html/Equazioni_differenziali.pdf)  
<sup>285</sup><https://www0.mi.infn.it/~palombo/didattica/Lab-TNDS/CorsoLab/LezioniFrontali/Lezione7-EquazioniDiffOrd.pdf>  
<sup>286</sup><https://www.dmi.unict.it/rosa/ode.pdf>  
<sup>287</sup><https://paola-gervasio.unibs.it/CS/Slides/eqdiff2.pdf>  
<sup>288</sup>[https://www.reddit.com/r/math/comments/2ezgvc/numerical\\_methods\\_for\\_integration\\_log\\_graph\\_of/](https://www.reddit.com/r/math/comments/2ezgvc/numerical_methods_for_integration_log_graph_of/)  
<sup>289</sup>[https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo\\_numerico/1516/eq\\_differenziali.pdf](https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo_numerico/1516/eq_differenziali.pdf)  
<sup>290</sup>[https://my.liuc.it/MatSup/2009/Y90000/N\\_5.doc](https://my.liuc.it/MatSup/2009/Y90000/N_5.doc)  
<sup>291</sup><https://www.docenti.unina.it/webdocenti-be/allegati/materiale-didattico/90597>  
<sup>292</sup><https://pagine.dm.unipi.it/ghelardoni/libro/ode.pdf>  
<sup>293</sup>[https://www.dm.unibo.it/~morigi/homepage\\_file/courses\\_file/file\\_dl/ODE\\_3\\_s.pdf](https://www.dm.unibo.it/~morigi/homepage_file/courses_file/file_dl/ODE_3_s.pdf)  
<sup>294</sup>[https://www.dmf.unisalento.it/~spagnolo/MSC\\_aa21\\_22/EquazioniDifferenziali/Lezione19-20\\_metodiSC.pdf](https://www.dmf.unisalento.it/~spagnolo/MSC_aa21_22/EquazioniDifferenziali/Lezione19-20_metodiSC.pdf)  
<sup>295</sup><https://www0.mi.infn.it/~palombo/didattica/Lab-TNDS/CorsoLab/LezioniFrontali/Lezione7-EquazioniDiffOrd.pdf>  
<sup>296</sup><https://www.dmi.unict.it/rosa/ode.pdf>  
<sup>297</sup><https://www0.mi.infn.it/~palombo/didattica/Lab-TNDS/CorsoLab/LezioniFrontali/Lezione7-EquazioniDiffOrd.pdf>  
<sup>298</sup>[https://www.dm.unibo.it/~montelau/html/Equazioni\\_differenziali.pdf](https://www.dm.unibo.it/~montelau/html/Equazioni_differenziali.pdf)  
<sup>299</sup><https://www.dmi.unict.it/rosa/ode.pdf>  
<sup>300</sup><https://paola-gervasio.unibs.it/CS/Slides/eqdiff2.pdf>  
<sup>301</sup><https://www0.mi.infn.it/~palombo/didattica/Lab-TNDS/CorsoLab/LezioniFrontali/Lezione7-EquazioniDiffOrd.pdf>  
<sup>302</sup>[https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo\\_numerico/1516/eq\\_differenziali.pdf](https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo_numerico/1516/eq_differenziali.pdf)  
<sup>303</sup><https://www.docenti.unina.it/webdocenti-be/allegati/materiale-didattico/90597>  
<sup>304</sup>[https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo\\_numerico/1516/eq\\_differenziali.pdf](https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo_numerico/1516/eq_differenziali.pdf)  
<sup>305</sup><https://www0.mi.infn.it/~palombo/didattica/Lab-TNDS/CorsoLab/LezioniFrontali/Lezione7-EquazioniDiffOrd.pdf>  
<sup>306</sup><https://www0.mi.infn.it/~palombo/didattica/Lab-TNDS/CorsoLab/LezioniFrontali/Lezione7-EquazioniDiffOrd.pdf>  
<sup>307</sup><https://pagine.dm.unipi.it/ghelardoni/libro/ode.pdf>  
<sup>308</sup><https://www.dmi.unict.it/rosa/ode.pdf>  
<sup>309</sup><https://paola-gervasio.unibs.it/CS/Slides/eqdiff2.pdf>  
<sup>310</sup>image.jpg  
<sup>311</sup>[https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo\\_numerico/eq\\_differenziali.pdf](https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo_numerico/eq_differenziali.pdf)

312[https://www.sbai.uniroma1.it/sites/default/files/Lez1112\\_AA19-20\\_0.pdf](https://www.sbai.uniroma1.it/sites/default/files/Lez1112_AA19-20_0.pdf)

313image.jpg

314[https://it.wikipedia.org/wiki/Metodi\\_di\\_soluzione\\_numerica\\_per\\_equazioni\\_differenziali\\_ordinarie](https://it.wikipedia.org/wiki/Metodi_di_soluzione_numerica_per_equazioni_differenziali_ordinarie)

315[https://www.dmi.unict.it/moschetti/farmacia/equazioni\\_differenziali.pdf](https://www.dmi.unict.it/moschetti/farmacia/equazioni_differenziali.pdf)

316[https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo\\_numerico/eq\\_differenziali.pdf](https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo_numerico/eq_differenziali.pdf)

317[https://it.wikipedia.org/wiki/Metodi\\_di\\_soluzione\\_numerica\\_per\\_equazioni\\_differenziali\\_ordinarie](https://it.wikipedia.org/wiki/Metodi_di_soluzione_numerica_per_equazioni_differenziali_ordinarie)

318[https://www.dmi.unict.it/moschetti/farmacia/equazioni\\_differenziali.pdf](https://www.dmi.unict.it/moschetti/farmacia/equazioni_differenziali.pdf)

319[https://www.math.unipd.it/~alvise/AN\\_2017/PDF/ODE\\_2017\\_PDF/ode\\_2017\\_PDF.pdf](https://www.math.unipd.it/~alvise/AN_2017/PDF/ODE_2017_PDF/ode_2017_PDF.pdf)

320[https://it.wikipedia.org/wiki/Metodi\\_di\\_soluzione\\_numerica\\_per\\_equazioni\\_differenziali\\_ordinarie](https://it.wikipedia.org/wiki/Metodi_di_soluzione_numerica_per_equazioni_differenziali_ordinarie)

321[https://it.wikipedia.org/wiki/Metodi\\_di\\_soluzione\\_numerica\\_per\\_equazioni\\_differenziali\\_ordinarie](https://it.wikipedia.org/wiki/Metodi_di_soluzione_numerica_per_equazioni_differenziali_ordinarie)

322[https://www.dmi.unict.it/moschetti/farmacia/equazioni\\_differenziali.pdf](https://www.dmi.unict.it/moschetti/farmacia/equazioni_differenziali.pdf)

323[https://it.wikipedia.org/wiki/Metodi\\_di\\_soluzione\\_numerica\\_per\\_equazioni\\_differenziali\\_ordinarie](https://it.wikipedia.org/wiki/Metodi_di_soluzione_numerica_per_equazioni_differenziali_ordinarie)

324[https://it.wikipedia.org/wiki/Metodi\\_di\\_soluzione\\_numerica\\_per\\_equazioni\\_differenziali\\_ordinarie](https://it.wikipedia.org/wiki/Metodi_di_soluzione_numerica_per_equazioni_differenziali_ordinarie)

325[https://www.dmi.unict.it/moschetti/farmacia/equazioni\\_differenziali.pdf](https://www.dmi.unict.it/moschetti/farmacia/equazioni_differenziali.pdf)

326[https://it.wikipedia.org/wiki/Metodi\\_di\\_soluzione\\_numerica\\_per\\_equazioni\\_differenziali\\_ordinarie](https://it.wikipedia.org/wiki/Metodi_di_soluzione_numerica_per_equazioni_differenziali_ordinarie)

327[https://it.wikipedia.org/wiki/Metodi\\_di\\_soluzione\\_numerica\\_per\\_equazioni\\_differenziali\\_ordinarie](https://it.wikipedia.org/wiki/Metodi_di_soluzione_numerica_per_equazioni_differenziali_ordinarie)

328[https://www.math.unipd.it/~alvise/AN\\_2017/PDF/ODE\\_2017\\_PDF/ode\\_2017\\_PDF.pdf](https://www.math.unipd.it/~alvise/AN_2017/PDF/ODE_2017_PDF/ode_2017_PDF.pdf)

329[https://it.wikipedia.org/wiki/Metodi\\_di\\_soluzione\\_numerica\\_per\\_equazioni\\_differenziali\\_ordinarie](https://it.wikipedia.org/wiki/Metodi_di_soluzione_numerica_per_equazioni_differenziali_ordinarie)

330[https://www.dm.unibo.it/~montelau/html/Equazioni\\_differenziali.pdf](https://www.dm.unibo.it/~montelau/html/Equazioni_differenziali.pdf)

331[https://www.dm.unibo.it/~morigi/homepage\\_file/courses\\_file/file\\_dl/ODE\\_3\\_s.pdf](https://www.dm.unibo.it/~morigi/homepage_file/courses_file/file_dl/ODE_3_s.pdf)

332[https://it.wikipedia.org/wiki/Metodi\\_di\\_soluzione\\_numerica\\_per\\_equazioni\\_differenziali\\_ordinarie](https://it.wikipedia.org/wiki/Metodi_di_soluzione_numerica_per_equazioni_differenziali_ordinarie)

333[https://www.dm.unibo.it/~montelau/html/Equazioni\\_differenziali.pdf](https://www.dm.unibo.it/~montelau/html/Equazioni_differenziali.pdf)

334image.jpg

335[https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo\\_numerico/eq\\_differenziali.pdf](https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo_numerico/eq_differenziali.pdf)

336[https://www.sbai.uniroma1.it/sites/default/files/Lez1112\\_AA19-20\\_0.pdf](https://www.sbai.uniroma1.it/sites/default/files/Lez1112_AA19-20_0.pdf)

337[https://it.wikipedia.org/wiki/Metodi\\_di\\_soluzione\\_numerica\\_per\\_equazioni\\_differenziali\\_ordinarie](https://it.wikipedia.org/wiki/Metodi_di_soluzione_numerica_per_equazioni_differenziali_ordinarie)

338[https://www.math.unipd.it/~alvise/AN\\_2017/PDF/ODE\\_2017\\_PDF/ode\\_2017\\_PDF.pdf](https://www.math.unipd.it/~alvise/AN_2017/PDF/ODE_2017_PDF/ode_2017_PDF.pdf)

339[https://it.wikipedia.org/wiki/Metodi\\_di\\_soluzione\\_numerica\\_per\\_equazioni\\_differenziali\\_ordinarie](https://it.wikipedia.org/wiki/Metodi_di_soluzione_numerica_per_equazioni_differenziali_ordinarie)

340[https://www.dm.unibo.it/~montelau/html/Equazioni\\_differenziali.pdf](https://www.dm.unibo.it/~montelau/html/Equazioni_differenziali.pdf)

341[https://it.wikipedia.org/wiki/Metodi\\_di\\_soluzione\\_numerica\\_per\\_equazioni\\_differenziali\\_ordinarie](https://it.wikipedia.org/wiki/Metodi_di_soluzione_numerica_per_equazioni_differenziali_ordinarie)

342[https://www.dm.unibo.it/~morigi/homepage\\_file/courses\\_file/file\\_dl/ODE\\_3\\_s.pdf](https://www.dm.unibo.it/~morigi/homepage_file/courses_file/file_dl/ODE_3_s.pdf)

343[https://www.dm.unibo.it/~montelau/html/Equazioni\\_differenziali.pdf](https://www.dm.unibo.it/~montelau/html/Equazioni_differenziali.pdf)

344[https://it.wikipedia.org/wiki/Metodi\\_di\\_soluzione\\_numerica\\_per\\_equazioni\\_differenziali\\_ordinarie](https://it.wikipedia.org/wiki/Metodi_di_soluzione_numerica_per_equazioni_differenziali_ordinarie)

345[https://www.dm.unibo.it/~morigi/homepage\\_file/courses\\_file/file\\_dl/ODE\\_3\\_s.pdf](https://www.dm.unibo.it/~morigi/homepage_file/courses_file/file_dl/ODE_3_s.pdf)

346[https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo\\_numerico/eq\\_differenziali.pdf](https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo_numerico/eq_differenziali.pdf)

347[https://www.dm.unibo.it/~montelau/html/Equazioni\\_differenziali.pdf](https://www.dm.unibo.it/~montelau/html/Equazioni_differenziali.pdf)

348[https://it.wikipedia.org/wiki/Metodi\\_di\\_soluzione\\_numerica\\_per\\_equazioni\\_differenziali\\_ordinarie](https://it.wikipedia.org/wiki/Metodi_di_soluzione_numerica_per_equazioni_differenziali_ordinarie)

349<https://www.docenti.unina.it/webdocenti-be/allegati/materiale-didattico/90597>

350image.jpg

351[https://it.wikipedia.org/wiki/Metodi\\_di\\_soluzione\\_numerica\\_per\\_equazioni\\_differenziali\\_ordinarie](https://it.wikipedia.org/wiki/Metodi_di_soluzione_numerica_per_equazioni_differenziali_ordinarie)

352[https://www.math.unipd.it/~alvise/AN\\_2017/PDF/ODE\\_2017\\_PDF/ode\\_2017\\_PDF.pdf](https://www.math.unipd.it/~alvise/AN_2017/PDF/ODE_2017_PDF/ode_2017_PDF.pdf)

353[https://www.math.unipd.it/~alvise/AN\\_2017/PDF/ODE\\_2017\\_PDF/ode\\_2017\\_PDF.pdf](https://www.math.unipd.it/~alvise/AN_2017/PDF/ODE_2017_PDF/ode_2017_PDF.pdf)

354[https://www.math.unipd.it/~alvise/AN\\_2017/PDF/ODE\\_2017\\_PDF/ode\\_2017\\_PDF.pdf](https://www.math.unipd.it/~alvise/AN_2017/PDF/ODE_2017_PDF/ode_2017_PDF.pdf)

355[https://www.math.unipd.it/~alvise/AN\\_2017/PDF/ODE\\_2017\\_PDF/ode\\_2017\\_PDF.pdf](https://www.math.unipd.it/~alvise/AN_2017/PDF/ODE_2017_PDF/ode_2017_PDF.pdf)

356[https://www.math.unipd.it/~alvise/AN\\_2017/PDF/ODE\\_2017\\_PDF/ode\\_2017\\_PDF.pdf](https://www.math.unipd.it/~alvise/AN_2017/PDF/ODE_2017_PDF/ode_2017_PDF.pdf)

<sup>357</sup>[https://www.math.unipd.it/~alvise/AN\\_2017/PDF/ODE\\_2017\\_PDF/ode\\_2017\\_PDF.pdf](https://www.math.unipd.it/~alvise/AN_2017/PDF/ODE_2017_PDF/ode_2017_PDF.pdf)  
<sup>358</sup>[https://www.dm.unibo.it/~montelau/html/Equazioni\\_differenziali.pdf](https://www.dm.unibo.it/~montelau/html/Equazioni_differenziali.pdf)  
<sup>359</sup>[https://www.dm.unibo.it/~montelau/html/Equazioni\\_differenziali.pdf](https://www.dm.unibo.it/~montelau/html/Equazioni_differenziali.pdf)  
<sup>360</sup>[https://www.dm.unibo.it/~morigi/homepage\\_file/courses\\_file/file\\_dl/ODE\\_3\\_s.pdf](https://www.dm.unibo.it/~morigi/homepage_file/courses_file/file_dl/ODE_3_s.pdf)  
<sup>361</sup>[https://it.wikipedia.org/wiki/Metodi\\_di\\_soluzione\\_numerica\\_per\\_equazioni\\_differenziali\\_ordinarie](https://it.wikipedia.org/wiki/Metodi_di_soluzione_numerica_per_equazioni_differenziali_ordinarie)  
<sup>362</sup>[https://www.dm.unibo.it/~montelau/html/Equazioni\\_differenziali.pdf](https://www.dm.unibo.it/~montelau/html/Equazioni_differenziali.pdf)  
<sup>363</sup>[https://www.dm.unibo.it/~morigi/homepage\\_file/courses\\_file/file\\_dl/ODE\\_3\\_s.pdf](https://www.dm.unibo.it/~morigi/homepage_file/courses_file/file_dl/ODE_3_s.pdf)  
<sup>364</sup>[https://www.dm.unibo.it/~montelau/html/Equazioni\\_differenziali.pdf](https://www.dm.unibo.it/~montelau/html/Equazioni_differenziali.pdf)  
<sup>365</sup>[https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo\\_numerico/eq\\_differenziali.pdf](https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo_numerico/eq_differenziali.pdf)  
<sup>366</sup><https://lucia-gastaldi.unibs.it/didattica2007/automazione/lezioni/ode.pdf>  
<sup>367</sup>[https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo\\_numerico/eq\\_differenziali.pdf](https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo_numerico/eq_differenziali.pdf)  
<sup>368</sup><https://pagine.dm.unipi.it/ghelardoni/libro/ode.pdf>  
<sup>369</sup>[https://www.math.unipd.it/~alvise/AN\\_2017/PDF/ODE\\_2017\\_PDF/ode\\_2017\\_PDF.pdf](https://www.math.unipd.it/~alvise/AN_2017/PDF/ODE_2017_PDF/ode_2017_PDF.pdf)  
<sup>370</sup>[https://www.math.unipd.it/~alvise/AN\\_2017/PDF/ODE\\_2017\\_PDF/ode\\_2017\\_PDF.pdf](https://www.math.unipd.it/~alvise/AN_2017/PDF/ODE_2017_PDF/ode_2017_PDF.pdf)  
<sup>371</sup><https://pagine.dm.unipi.it/ghelardoni/libro/ode.pdf>  
<sup>372</sup><https://www0.mi.infn.it/~palombo/didattica/Lab-TNDS/CorsoLab/LezioniFrontali/Lezione4-QuadraturaNumerica.pdf>  
<sup>373</sup><https://elearning.uniroma1.it/mod/resource/view.php?id=35795>  
<sup>374</sup>[https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo\\_numerico/eq\\_differenziali.pdf](https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo_numerico/eq_differenziali.pdf)  
<sup>375</sup>[https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo\\_numerico/eq\\_differenziali.pdf](https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo_numerico/eq_differenziali.pdf)  
<sup>376</sup><https://lucia-gastaldi.unibs.it/didattica2007/automazione/lezioni/ode.pdf>  
<sup>377</sup>[https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo\\_numerico/eq\\_differenziali.pdf](https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo_numerico/eq_differenziali.pdf)  
<sup>378</sup>[https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo\\_numerico/eq\\_differenziali.pdf](https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo_numerico/eq_differenziali.pdf)  
<sup>379</sup><https://lucia-gastaldi.unibs.it/didattica2007/automazione/lezioni/ode.pdf>  
<sup>380</sup>[https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo\\_numerico/eq\\_differenziali.pdf](https://wwwold.iac.cnr.it/~pasca/corso/slides/calcolo_numerico/eq_differenziali.pdf)  
<sup>381</sup><https://lucia-gastaldi.unibs.it/didattica2007/automazione/lezioni/ode.pdf>